

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA

NGUYỄN TRƯỞNG MINH HOÀNG

XÂY DỰNG FRAMEWORK TẠO
DỰNG MÔ HÌNH AI CHO CÁC
ỨNG DỤNG THEO DÕI
CHUYỂN ĐỘNG CỦA CON
NGƯỜI

Chuyên ngành: Khoa học Máy tính
Mã ngành: 8480101

ĐỀ ÁN TỐT NGHIỆP

Thành phố Hồ Chí Minh, tháng 08 năm 2026

XÁC NHẬN CỦA GIẢNG VIÊN

Xác nhận chữ ký

LỜI CẢM ƠN

Tôi xin bày tỏ lòng biết ơn chân thành đến tất cả những người đã đóng góp vào việc phát triển và hoàn thiện luận văn này.

Trước tiên, tôi xin gửi lời tri ân sâu sắc nhất đến người hướng dẫn khoa học của tôi, TS. Lê Trọng Nhân, vì sự hướng dẫn quý báu, động viên và phản hồi mang tính xây dựng trong suốt quá trình hình thành đề tài này. Chuyên môn và những hiểu biết sâu sắc của thầy đã đóng vai trò quan trọng trong việc định hướng nghiên cứu của tôi.

Tôi cũng vô cùng biết ơn Trường Đại học Bách Khoa - ĐHQG TP.HCM và Khoa Khoa học và Kỹ thuật Máy tính đã cung cấp môi trường học thuật nuôi dưỡng và các nguồn lực cần thiết để thực hiện nghiên cứu này.

Lời cảm ơn đặc biệt gửi đến các bạn đồng nghiệp và đồng môn đã chia sẻ quan điểm và đưa ra những gợi ý sâu sắc, làm phong phú thêm phạm vi của nghiên cứu này.

Cuối cùng, tôi vô cùng biết ơn gia đình và bạn bè vì sự ủng hộ không ngừng, sự kiên nhẫn và động lực khi tôi đắm mình vào nỗ lực đầy thách thức nhưng bổ ích này.

Đề xuất nghiên cứu này là sự phản ánh của sự hỗ trợ và cống hiến tập thể, và tôi xin chân thành cảm ơn tất cả những người đã đóng góp trực tiếp hoặc gián tiếp vào việc hiện thực hóa nó.

TÓM TẮT

Sự phát triển của học máy và điện toán biên đã cách mạng hóa các ứng dụng theo dõi chuyển động, cho phép các hệ thống thời gian thực, tiết kiệm năng lượng và chính xác cao cho nhiều trường hợp sử dụng trong y tế, phân tích thể thao và tương tác người-máy. Tuy nhiên, việc phát triển các mô hình AI cho theo dõi chuyển động vẫn còn phức tạp, đòi hỏi kiến thức chuyên môn về tiền xử lý dữ liệu, trích xuất đặc trưng, huấn luyện mô hình và triển khai trên thiết bị biên. Bài báo này trình bày một framework toàn diện giúp đơn giản hóa quy trình end-to-end để tạo ra các mô hình AI được tùy chỉnh cho theo dõi chuyển động con người trên các thiết bị biên có tài nguyên hạn chế.

Framework giới thiệu một phương pháp tiền xử lý tương tác mới với tính năng lựa chọn cửa sổ thời gian kéo thả, cho phép người dùng chú thích và phân đoạn dữ liệu chuyển động một cách hiệu quả mà không cần kiến thức lập trình sâu rộng. Hệ thống hỗ trợ nhiều thuật toán học máy bao gồm Random Forest, Support Vector Machines và Neural Networks, với khả năng tự động sinh mã cho nhiều họ thiết bị và môi trường thực thi khác nhau như các bo tương thích Arduino, ARM Cortex-M thuần, ESP-IDF, MicroPython và Zephyr. Kiến trúc tạo mã nhận biết tối ưu hóa xem xét nhiều chiến lược triển khai—độ chính xác, tốc độ, tiêu thụ điện năng và phương án cân bằng—điều chỉnh gói mã sinh ra theo ràng buộc của từng đích triển khai.

Để kiểm chứng framework ở mức thực nghiệm, chúng tôi sử dụng bộ dữ liệu Nhận dạng Hoạt động Con Người (HAR) được thu thập ở 100Hz từ cảm biến IMU 6 trục (LSM6DS3) trên Seeed XIAO nRF52840 Sense như một nền tảng tham chiếu thuộc họ ARM Cortex-M4F. Trong nghiên cứu bằng chứng khái niệm này sử dụng dữ liệu đơn đối tượng trên năm lớp hoạt động, năm thuật toán (Random Forest, SVM, Neural Network, PyTorch MLP, PyTorch 1D-CNN) đạt độ chính xác 97,38–99,13% trên tập kiểm tra, với PyTorch MLP và 1D-CNN đạt cao nhất (99,13%). Quan trọng hơn, cùng một pipeline huấn luyện có thể được ánh xạ sang nhiều đích triển khai thông qua hệ thống generator phân tầng theo mô hình, nền tảng và backend thực thi. Dữ liệu tham khảo trên thiết bị cho thấy mô hình PyTorch 1D-CNN đạt 99,4% độ chính xác triển khai với tỷ lệ từ chối chỉ 0,6%. Phân tích so sánh với các nền tảng thương mại (Edge Impulse, SensiML) cho thấy framework mã nguồn mở của chúng tôi cung cấp tính minh bạch hoàn toàn, tính linh hoạt tiền xử lý vượt trội và chi phí bằng không.

Framework giải quyết các khoảng trống quan trọng trong các giải pháp hiện tại bằng cách cung cấp: (1) quy trình làm việc dựa trên GUI trực quan để tiếp cận cho

người không chuyên, (2) kiểm soát hoàn toàn đường ống tiền xử lý với phản hồi trực quan, (3) triển khai đa thiết bị với các tối ưu hóa đặc thù cho từng đích, và (4) kiến trúc có thể mở rộng hỗ trợ tích hợp các thuật toán, backend và nền tảng bổ sung trong tương lai. Nghiên cứu này đóng góp vào việc dân chủ hóa phát triển Edge AI cho các ứng dụng theo dõi chuyển động, với tác động tiềm năng trong phục hồi chức năng y tế, giám sát hiệu suất thể thao và công nghệ hỗ trợ.

SUMMARY

Recent advances in machine learning and edge computing have enabled real-time, energy-efficient, and accurate human motion monitoring systems for healthcare, sports analytics, and human-computer interaction. However, building AI models for motion tracking remains complex because it requires expertise in data preprocessing, feature extraction, model training, and embedded deployment. This thesis presents an end-to-end framework that simplifies the full workflow for developing customized AI models for human activity recognition on resource-constrained edge devices.

The framework introduces an interactive preprocessing approach with drag-and-select time-window annotation, allowing users to segment motion data effectively without deep programming expertise. The system supports multiple learning algorithms, including Random Forest, Support Vector Machines, and Neural Networks, and provides automatic code generation for multiple deployment targets and runtimes, such as Arduino-compatible boards, bare-metal ARM Cortex-M, ESP-IDF, MicroPython, and Zephyr. Its optimization-aware code generation architecture supports different deployment objectives—accuracy, speed, power efficiency, and balanced trade-offs—to adapt generated code to target-specific constraints.

For empirical validation, we use a 100Hz Human Activity Recognition dataset collected from a 6-axis IMU (LSM6DS3), with Sseeed XIAO nRF52840 Sense as the reference ARM Cortex-M4F platform. In this proof-of-concept study with a single-subject, five-class dataset, five algorithms (Random Forest, SVM, Neural Network, PyTorch MLP, and PyTorch 1D-CNN) achieve 97.38–99.13% test accuracy, where PyTorch MLP and 1D-CNN reach the highest result (99.13%). More importantly, the same trained artifact can be mapped to multiple deployment targets through a hierarchical generator design across model type, platform, and runtime backend. On-device reference results show that the PyTorch 1D-CNN model reaches 99.4% deployment accuracy with only a 0.6% rejection rate. Comparative analysis against commercial platforms (Edge Impulse and SensiML) indicates that our open-source framework offers full transparency, stronger preprocessing flexibility, and zero platform cost.

The framework addresses key limitations of existing tools by providing: (1) an intuitive GUI-driven workflow for non-expert users, (2) full control of preprocessing with visual feedback, (3) multi-device deployment with target-specific optimizations, and (4) an extensible architecture for adding algorithms, backends, and platforms in

future work. This contribution supports the democratization of edge AI development for human motion monitoring applications.

LỜI CAM ĐOAN

Tôi cam đoan rằng nghiên cứu này là công trình của riêng tôi, được thực hiện dưới sự giám sát và hướng dẫn của TS. Lê Trọng Nhân. Kết quả nghiên cứu của tôi là hợp pháp và chưa được công bố dưới bất kỳ hình thức nào trước đây. Tất cả các tài liệu được sử dụng trong nghiên cứu này đều được tôi thu thập từ nhiều nguồn khác nhau và đã được liệt kê đầy đủ trong phần tài liệu tham khảo. Ngoài ra, trong nghiên cứu này, tôi cũng đã sử dụng kết quả của một số tác giả và tổ chức khác. Tất cả đều đã được trích dẫn một cách thích hợp. Trong mọi trường hợp đạo văn, tôi hoàn toàn chịu trách nhiệm về hành động của mình. Do đó, Trường Đại học Bách Khoa - ĐHQG TP.HCM không chịu trách nhiệm về bất kỳ vi phạm bản quyền nào được thực hiện trong nghiên cứu của tôi.

Mục lục

LỜI CẢM ƠN	ii
TÓM TẮT	iii
SUMMARY	v
LỜI CAM ĐOAN	vii
Mục lục	viii
Danh sách bảng	xiii
Danh sách hình vẽ	xiv
1 Giới thiệu	1
1.1 Bối cảnh và động lực nghiên cứu	1
1.1.1 Thiết bị đeo thông minh và cảm biến quán tính	1
1.1.2 Nhận dạng hoạt động con người và các ứng dụng thực tiễn . . .	2
1.1.3 Độ phức tạp kỹ thuật của quy trình HAR	2
1.2 Thách thức trong triển khai HAR trên thiết bị biên	3
1.2.1 Xu hướng xử lý tại biên	3
1.2.2 Ràng buộc tài nguyên phần cứng	4
1.2.3 Khoảng cách huấn luyện–triển khai	4
1.2.4 Hệ sinh thái thiết bị phân mảnh	5
1.3 Các nền tảng hiện có và hạn chế	5
1.3.1 Tổng quan	5
1.3.2 Phân tích các nền tảng chính	6
1.3.3 Tổng hợp khoảng trống	7
1.4 Phát biểu bài toán và mục tiêu nghiên cứu	7
1.4.1 Phát biểu bài toán	7

1.4.2	Mục tiêu nghiên cứu	8
1.4.3	Đóng góp chính	8
1.5	Phạm vi nghiên cứu	9
1.6	Tổ chức luận văn	9
2	Các nghiên cứu liên quan	11
2.1	Tổng quan	11
2.2	Các lĩnh vực ứng dụng của theo dõi chuyển động	11
2.3	Theo dõi chuyển động dựa trên cảm biến	11
2.4	Theo dõi chuyển động dựa trên thị giác	12
2.5	AI trong theo dõi chuyển động	12
2.6	Các framework và nền tảng hiện có	13
2.7	Tiền xử lý dữ liệu và tạo cửa sổ	13
2.8	Trích xuất và biểu diễn đặc trưng	14
2.9	Triển khai biên và tối ưu hóa	14
2.10	Các khoảng trống trong công trình hiện có	15
2.11	Tóm tắt	16
3	Cơ sở lý thuyết	17
3.1	Dữ liệu cảm biến quán tính cho nhận dạng hoạt động	17
3.2	Quy trình tiền xử lý tương tác	18
3.2.1	Tải lên và trực quan hóa dữ liệu	18
3.2.2	Các phương pháp lọc và làm sạch tín hiệu	18
3.2.3	Phân đoạn và sinh cửa sổ huấn luyện	20
3.3	Trích xuất đặc trưng	21
3.3.1	Tham số cửa sổ và ràng buộc nhất quán	21
3.3.2	Các loại trích xuất đặc trưng	21
3.3.3	Chiến lược đệm cửa sổ	23
3.3.4	Tăng cường dữ liệu cho huấn luyện	23
3.3.5	Ngưỡng tin cậy cho từ chối hoạt động không xác định	24
3.4	Huấn luyện mô hình học máy	24
3.4.1	Các thuật toán học máy áp dụng	24
3.4.2	Giao thức huấn luyện và đánh giá	26
3.4.3	Tối ưu hóa siêu tham số	27
3.5	Tạo mã cho triển khai biên	27
3.5.1	Kiến trúc	27
3.5.2	Lý do chọn tạo mã trực tiếp thay vì Runtime Engine	28

3.5.3	Triển khai TFLite Micro	32
3.5.4	Cấu trúc mã đã tạo	33
3.5.5	Lượng tử hóa mô hình cho triển khai biên	34
3.5.6	Kiến trúc tạo mã đa kiến trúc	37
3.5.7	Các chiến lược tối ưu hóa	37
3.5.8	Điều chỉnh đặc thù nền tảng	37
4	Thiết kế và hiện thực	38
4.1	Kiến trúc hệ thống	38
4.1.1	Nền tảng công nghệ và môi trường phát triển	38
4.1.2	Kiến trúc ứng dụng	39
4.1.3	Quản lý trạng thái	39
4.2	Module quản lý dữ liệu	40
4.2.1	Giao diện tải dữ liệu	40
4.3	Module tiền xử lý tương tác	42
4.3.1	Giao diện lựa chọn đoạn tương tác	42
4.3.2	Sinh cửa sổ trượt	43
4.3.3	Trực quan hóa lọc tín hiệu	44
4.4	Module trích xuất đặc trưng	44
4.4.1	Chế độ trích xuất đặc trưng	44
4.4.2	Tích hợp tăng cường dữ liệu	44
4.4.3	Phân chia train/validation/test	45
4.5	Module huấn luyện mô hình	46
4.5.1	Lựa chọn thuật toán	46
4.5.2	Tối ưu hóa siêu tham số	47
4.5.3	Theo dõi tiến trình real-time	47
4.5.4	Lưu trữ mô hình	48
4.6	Module tạo mã triển khai	49
4.6.1	Kiến trúc Code Generator: Factory Pattern	49
4.6.2	Hệ thống phân cấp Generator	49
4.6.3	Kiến trúc ba trục: Model $\times$ Platform $\times$ Backend	50
4.6.4	Sắp xếp lại thứ tự đặc trưng khi sinh mã	51
4.6.5	Bốn chế độ tối ưu hóa	51
4.6.6	Gói đầu ra triển khai	52
4.6.7	Kiến trúc đa mô hình: CNN code generation	53
4.7	Module kiểm tra thiết bị	54

4.7.1	Giao tiếp serial	54
4.7.2	Hiện thị phân loại real-time	54
4.7.3	Ngưỡng tin cậy	55
4.7.4	Quy trình xác minh triển khai	55
4.8	Luồng dữ liệu và quản lý trạng thái tổng thể	55
4.8.1	Luồng dữ liệu end-to-end	55
4.9	Tổng kết chương	56
5	Kết quả thực nghiệm	58
5.1	Thiết lập thực nghiệm	58
5.2	Kết quả huấn luyện	59
5.2.1	Bài toán 3 lớp (running / still / walking)	59
5.2.2	Bài toán 5 lớp — Đánh giá chính	60
5.2.3	Hiệu suất theo từng lớp	61
5.3	So sánh hiệu suất giữa các thuật toán	62
5.4	Khả năng triển khai đa thiết bị	63
5.5	Đánh giá triển khai trên thiết bị	64
5.5.1	Tổng hợp kết quả triển khai	64
5.5.2	Phân tích theo từng mô hình	65
5.5.3	Bài học: khoảng cách huấn luyện–triển khai	65
5.6	So sánh với Edge Impulse	66
5.6.1	So sánh hiệu suất	66
5.6.2	So sánh tài nguyên	66
5.6.3	So sánh kiến trúc	67
5.7	Tóm tắt	68
6	Kết luận và hướng phát triển	69
6.1	Diễn giải các phát hiện chính	69
6.1.1	Hiệu suất mô hình và lựa chọn thuật toán	69
6.1.2	Cân bằng lớp trong thực hành	69
6.1.3	Tính khả thi triển khai biên	69
6.1.4	Tiền xử lý tương tác: cách tiếp cận mới	70
6.2	So sánh với công trình liên quan	70
6.2.1	Các nền tảng thương mại	70
6.2.2	Các hệ thống HAR học thuật	70
6.3	Các đánh đổi thiết kế	71
6.3.1	ML cổ điển vs học sâu	71

6.3.2	Đặc trưng thủ công vs đặc trưng tự học	71
6.3.3	Tạo mã trực tiếp vs runtime suy luận	71
6.3.4	Các chế độ tối ưu hóa	72
6.4	Tương đồng huấn luyện-triển khai trong edge ML	72
6.4.1	Các nguồn không khớp được xác định	72
6.4.2	Tác động thực tế	73
6.4.3	Danh sách kiểm tra tương đồng	73
6.4.4	So sánh với nền tảng thương mại	74
6.4.5	Bộ lọc Kalman: giải pháp parity	74
6.5	Tăng cường dữ liệu và ngưỡng tin cậy	75
6.5.1	Tăng cường nhận biết lớp	75
6.5.2	Ngưỡng tin cậy cho từ chối hoạt động không xác định	75
6.6	Phân tích các phát hiện kỹ thuật quan trọng	76
6.6.1	Kiến trúc tạo mã đa kiến trúc	76
6.6.2	Hạn chế TFLite và giải pháp Keras surrogate	76
6.7	Các hạn chế	76
6.7.1	Hạn chế tập dữ liệu	76
6.7.2	Lựa chọn kích thước cửa sổ	77
6.7.3	Vị trí và hướng cảm biến	77
6.7.4	Sự đa dạng nền tảng tính toán	77
6.7.5	Tính hợp lệ bên ngoài	77
6.8	Hướng phát triển tương lai	78
6.8.1	Các mở rộng ngắn hạn	78
6.8.2	Tầm nhìn dài hạn	78
6.9	Kết luận	79
6.9.1	Tóm tắt đóng góp	79
6.9.2	Xem lại các mục tiêu nghiên cứu	79
6.9.3	Các ứng dụng thực tế	80
6.9.4	Lời kết	81
Tài liệu tham khảo		82

Danh sách bảng

1.2.1	So sánh tài nguyên giữa môi trường huấn luyện và triển khai	4
1.3.1	So sánh các nền tảng Edge ML cho nhận dạng hoạt động	7
3.2.1	So sánh các phương pháp lọc theo tính tương đồng huấn luyện-triển khai	20
3.3.1	Sáu loại trích xuất đặc trưng: tín hiệu nguồn, cấu thành và đặc tính triển khai	22
3.5.1	Ma trận đích triển khai được hỗ trợ	28
3.5.2	So sánh phương pháp triển khai mô hình ML trên vi điều khiển	30
3.5.3	So sánh chiến lược lượng tử hóa theo phương pháp tạo mã	36
4.5.1	Thuật toán học máy được hỗ trợ	46
4.6.1	Chế độ tối ưu hóa mã triển khai	51
5.2.1	Kết quả huấn luyện — bài toán 3 lớp (63 đặc trưng orientation-invariant, tập kiểm tra 156 mẫu)	59
5.2.2	Kết quả huấn luyện — bài toán 5 lớp (63 đặc trưng orientation-invariant, tập kiểm tra 229 mẫu)	60
5.2.3	Hiệu suất theo từng lớp — bài toán 5 lớp, tập kiểm tra (229 mẫu)	61
5.3.1	So sánh tổng hợp các thuật toán — bài toán 5 lớp	62
5.3.2	Ảnh hưởng của số lượng lớp đến độ chính xác kiểm tra	63
5.4.1	Ma trận đích triển khai — phạm vi hỗ trợ của hệ thống tạo mã	63
5.5.1	Kết quả triển khai trên thiết bị — Seeed XIAO nRF52840	64
5.6.1	So sánh framework đề xuất vs Edge Impulse — bài toán 3 lớp, cùng tập dữ liệu	66
5.6.2	So sánh tài nguyên triển khai — Cortex-M4F @ 64 MHz	67
5.6.3	So sánh kiến trúc với các nền tảng thương mại	68
6.4.1	Danh sách kiểm tra tương đồng huấn luyện-triển khai	74

Danh sách hình vẽ

1.1.1	Quy trình HAR dựa trên IMU: từ thu thập tín hiệu thô đến triển khai trên vi điều khiển	2
1.2.1	So sánh xử lý đám mây và xử lý tại biên cho HAR: triển khai biên loại bỏ phụ thuộc kết nối mạng và giảm độ trễ từ hàng trăm mili-giây xuống dưới 15 ms	4
1.3.1	Giao diện Edge Impulse (trang thiết kế Impulse): quy trình được định nghĩa bằng các block cố định, giới hạn khả năng tùy chỉnh tiền xử lý và thuật toán	6
1.4.1	Quy trình làm việc từ dữ liệu thô đến triển khai biên (tổng ước tính 30–60 phút)	9
4.2.1	Giao diện tải lên và trực quan hóa dữ liệu cảm biến. Framework tự động phát hiện cấu trúc cột IMU và hiển thị biểu đồ tín hiệu đồng bộ trên tất cả các trục gia tốc kế và con quay hồi chuyển, cho phép kiểm tra trực quan chất lượng dữ liệu trước khi tiền xử lý.	41
4.3.1	Giao diện cửa sổ kéo thả tương tác. Người dùng kéo thả trực tiếp trên biểu đồ tín hiệu để xác định các đoạn hoạt động đại diện. Cửa sổ trượt tự động sinh các mẫu huấn luyện từ các đoạn đã chọn, kết hợp giữa kiểm soát thủ công và tự động hóa.	43
4.4.1	Giao diện trích xuất đặc trưng. Người dùng chọn loại đặc trưng (bất biến hướng, miền thời gian, miền tần số), cấu hình tăng cường dữ liệu nhận biết lớp, và phân chia dữ liệu huấn luyện/xác thực/kiểm tra với lấy mẫu phân tầng.	45
4.5.1	Giao diện cấu hình huấn luyện mô hình. Người dùng chọn thuật toán, cấu hình tối ưu hóa siêu tham số với GridSearchCV, chiến lược cân bằng lớp, và theo dõi tiến trình huấn luyện thời gian thực.	48

4.6.1	Giao diện tạo mã triển khai. Hiển thị các tệp header, implementation và Arduino sketch được sinh ra. Người dùng chọn chế độ tối ưu hóa (Accuracy, Speed, Power, Balanced), cấu hình nền tảng đích và tải xuống gói triển khai hoàn chỉnh.	53
5.2.1	Ma trận nhầm lẫn — PyTorch MLP, bài toán 5 lớp (tập kiểm tra 229 mẫu). Các lỗi phân loại tập trung ở nhóm hoạt động đi bộ: 1 mẫu walking bị nhầm thành walking_downstairs (hàng walking, cột W.Down). . . .	62

Chương 1

Giới thiệu

Luận văn này trình bày một framework mã nguồn mở cho phép xây dựng hệ thống nhận dạng hoạt động con người (Human Activity Recognition — HAR) dựa trên cảm biến quán tính và triển khai kết quả trực tiếp lên vi điều khiển. Trọng tâm của nghiên cứu là giải quyết khoảng cách giữa giai đoạn huấn luyện mô hình trong môi trường Python và giai đoạn suy luận trên thiết bị nhúng tài nguyên hạn chế — một vấn đề thực tiễn mà các công cụ hiện có chưa xử lý thỏa đáng.

Chương này được tổ chức như sau. Phần 1.1 giới thiệu bối cảnh ứng dụng và vai trò của HAR trong hệ sinh thái thiết bị đeo thông minh. Phần 1.2 phân tích các thách thức kỹ thuật đặc thù của triển khai biên. Phần 1.3 đánh giá các nền tảng hiện có và xác định khoảng trống nghiên cứu. Phần 1.4 phát biểu bài toán, mục tiêu và đóng góp của luận văn. Các phần cuối trình bày phạm vi và cấu trúc luận văn.

1.1 Bối cảnh và động lực nghiên cứu

1.1.1 Thiết bị đeo thông minh và cảm biến quán tính

Sự tiến bộ của công nghệ vi mạch trong hai thập niên gần đây đã cho phép tích hợp nhiều cảm biến — gia tốc kế, con quay hồi chuyển, từ kế, cảm biến nhịp tim — vào các thiết bị đeo nhỏ gọn với chi phí và mức tiêu thụ năng lượng ngày càng thấp. Kết quả là một hệ sinh thái thiết bị đeo phong phú đã hình thành: từ đồng hồ thông minh và vòng tay theo dõi sức khỏe đến miếng dán cảm biến y tế và thiết bị hỗ trợ vận động. Song song với đó, các thuật toán học máy ngày càng hiệu quả đã mở ra khả năng suy luận thông minh ngay trên thiết bị, không cần truyền dữ liệu lên đám mây^[42].

Trong hệ sinh thái này, **cảm biến quán tính** (Inertial Measurement Unit — IMU) đóng vai trò trung tâm. IMU kết hợp gia tốc kế ba trục và con quay hồi chuyển ba

1.1. Bối cảnh và động lực nghiên cứu

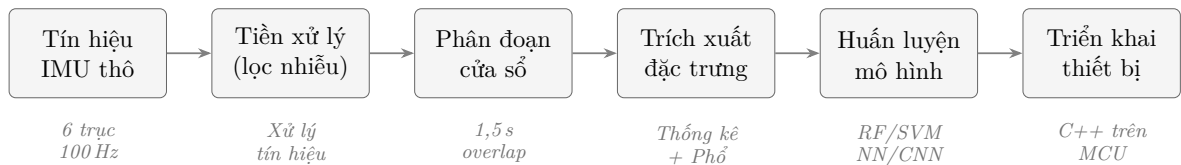
trục, cung cấp sáu chiều dữ liệu chuyển động ở tần số 50–400 Hz. So với camera, IMU tiêu thụ năng lượng thấp hơn nhiều bậc, không thu thập hình ảnh hay âm thanh nên bảo vệ quyền riêng tư, và có thể hoạt động liên tục trong nhiều ngày trên pin nhỏ^[15]. Những đặc tính này làm cho IMU trở thành lựa chọn lý tưởng cho giám sát hoạt động liên tục trong môi trường thực tế.

1.1.2 Nhận dạng hoạt động con người và các ứng dụng thực tiễn

Nhận dạng hoạt động con người (HAR) là bài toán tự động phân loại các hoạt động thể chất dựa trên dữ liệu cảm biến^[9;23]. Nhu cầu HAR xuất hiện trong nhiều lĩnh vực quan trọng.

Trong **chăm sóc sức khỏe**, HAR hỗ trợ phát hiện té ngã cho người cao tuổi^[39], theo dõi khách quan tiến trình phục hồi chức năng sau phẫu thuật, và giám sát hoạt động hàng ngày (Activities of Daily Living — ADL) để phát hiện sớm suy giảm chức năng ở người sống độc lập. Trong **thể thao và vận động**, HAR cho phép phân tích kỹ thuật tập luyện, đếm bài tập tự động, và đánh giá cường độ vận động phục vụ cá nhân hóa chương trình huấn luyện. Trong **an toàn công nghiệp**, HAR phát hiện tư thế nguy hiểm tại công trường và nhà máy^[40], theo dõi mức mệt mỏi của công nhân. Ngoài ra, HAR còn được ứng dụng trong nhà thông minh để nhận diện hoạt động sinh hoạt và điều chỉnh môi trường phù hợp, cũng như hỗ trợ người khuyết tật qua giao diện điều khiển dựa trên cử chỉ.

Điểm chung của tất cả ứng dụng trên là yêu cầu hệ thống hoạt động *ngay trên thiết bị đeo*, không phụ thuộc kết nối mạng, với độ trễ thấp và tiêu thụ năng lượng tối thiểu. Đây chính là bài toán triển khai biên (edge deployment) — chủ đề trung tâm của luận văn này.



Hình 1.1.1: Quy trình HAR dựa trên IMU: từ thu thập tín hiệu thô đến triển khai trên vi điều khiển

1.1.3 Độ phức tạp kỹ thuật của quy trình HAR

Quy trình HAR hoàn chỉnh (Hình 1.1.1) đòi hỏi kiến thức liên ngành sâu rộng^[29]. Bắt đầu từ việc thiết kế giao thức thu thập dữ liệu và gán nhãn — công đoạn thường

1.2. Thách thức trong triển khai HAR trên thiết bị biên

chiếm 40–60% tổng thời gian dự án^[26] — đến lọc nhiễu tín hiệu với các bộ lọc thông thấp phù hợp, phân đoạn tín hiệu thành các cửa sổ thời gian, trích xuất đặc trưng thống kê và phổ, rồi huấn luyện và tối ưu mô hình phân loại. Mỗi bước yêu cầu một chuyên môn riêng: xử lý tín hiệu số, thống kê, học máy, và lập trình nhúng.

Sự đa dạng về kiến thức cần thiết tạo ra **rào cản gia nhập đáng kể**. Một nhà nghiên cứu y tế có thể hiểu rõ bài toán phát hiện té ngã nhưng thiếu kỹ năng triển khai nhúng; một lập trình viên nhúng có thể tối ưu mã C++ nhưng thiếu kiến thức thống kê để thiết kế bộ đặc trưng phù hợp. Khoảng cách này dẫn đến thực trạng: hầu hết nghiên cứu HAR dừng lại ở nguyên mẫu Python, rất ít công trình cung cấp mã triển khai sẵn sàng chạy trên thiết bị thực^[28].

1.2 Thách thức trong triển khai HAR trên thiết bị biên

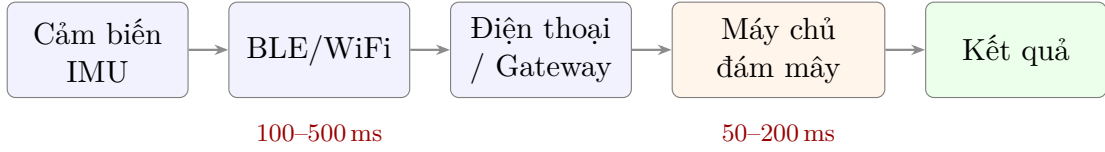
1.2.1 Xu hướng xử lý tại biên

Mô hình xử lý dữ liệu cảm biến đã chuyển dịch từ kiến trúc tập trung (cloud-centric) sang kiến trúc phân tán tại biên (edge computing). Lý do có nhiều mặt: truyền dữ liệu qua Bluetooth hoặc WiFi lên đám mây thêm hàng trăm mili-giây độ trễ — không chấp nhận được cho ứng dụng phát hiện té ngã hay cảnh báo tư thế nguy hiểm cần phản hồi tức thì. Về năng lượng, truyền không dây tiêu thụ gấp nhiều lần so với xử lý cục bộ^[42], khiến pin cạn nhanh chóng nếu phát trực tuyến liên tục dữ liệu IMU sáu trục. Về quyền riêng tư, dữ liệu chuyển động chứa thông tin nhạy cảm; xử lý tại biên đảm bảo dữ liệu thô không rời khỏi thiết bị. Cuối cùng, thiết bị đeo thường hoạt động trong môi trường không có kết nối ổn định — suy luận tại biên đảm bảo hệ thống vận hành liên tục, không phụ thuộc hạ tầng bên ngoài.

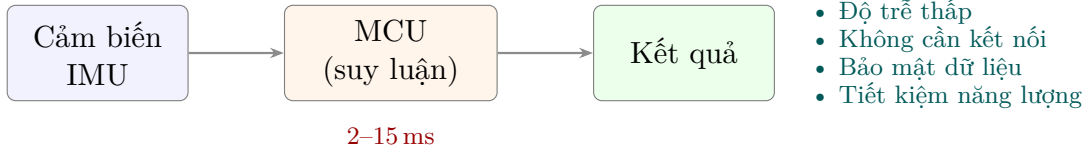
Hình 1.2.1 so sánh hai kiến trúc: xử lý đám mây đòi hỏi chuỗi truyền dữ liệu dài (cảm biến → BLE → gateway → cloud → kết quả), trong khi xử lý tại biên khép kín toàn bộ pipeline suy luận ngay trên vi điều khiển gắn liền với cảm biến.

1.2. Thách thức trong triển khai HAR trên thiết bị biên

(a) Xử lý trên đám mây



(b) Xử lý tại biên



Hình 1.2.1: So sánh xử lý đám mây và xử lý tại biên cho HAR: triển khai biên loại bỏ phụ thuộc kết nối mạng và giảm độ trễ từ hàng trăm mili-giây xuống dưới 15 ms

1.2.2 Ràng buộc tài nguyên phần cứng

Vi điều khiển dùng trong thiết bị đeo (ARM Cortex-M4F, 64 MHz) có tài nguyên hạn chế nghiêm ngặt so với môi trường huấn luyện:

Bảng 1.2.1: So sánh tài nguyên giữa môi trường huấn luyện và triển khai

Tài nguyên	Máy tính huấn luyện	MCU triển khai
RAM	16–64 GB	64–256 KB (nhỏ hơn $\sim 100.000\times$)
Bộ nhớ chương trình	Hàng TB	256 KB–1 MB (nhỏ hơn $\sim 1.000.000\times$)
Tốc độ xử lý	3–5 GHz, đa nhân	64 MHz, đơn nhân
Số học dấu phẩy động	Float64 mặc định	Float32 (FPU đơn) hoặc cố định
Thư viện	NumPy, scikit-learn, PyTorch	Không có (tự triển khai)

Khoảng cách tài nguyên này buộc mọi thành phần trong pipeline phải được thiết kế lại cho phù hợp: mô hình Random Forest với 100 cây sâu 12 cấp có thể chiếm 100–400 KB flash, gần sát giới hạn thiết bị; một bước DFT trên 150 mẫu đòi hỏi vùng nhớ $\mathcal{O}(N)$ cho mảng tạm; và mọi phép tính phải hoàn thành trong vài chục mili-giây để đảm bảo suy luận thời gian thực.

1.2.3 Khoảng cách huấn luyện–triển khai

Đây là thách thức nghiêm trọng nhất và ít được đề cập nhất trong tài liệu edge ML^[28;34]. Mô hình được huấn luyện trong Python với thư viện khoa học dữ liệu đầy

1.3. Các nền tảng hiện có và hạn chế

đủ, nhưng phải chạy trên MCU với mã C/C++ viết thủ công. Quá trình chuyển đổi tiềm ẩn nhiều nguồn sai lệch: các công thức thống kê (kurtosis, skewness) có thể được tính khác biệt giữa Python và C++ nếu không kiểm tra kỹ; thứ tự đặc trưng trong Python thường do cấu trúc dữ liệu quy định trong khi C++ tính theo thứ tự xử lý — nếu ánh xạ sai, trọng số mô hình sẽ nhận đầu vào lệch vị trí; chiến lược đệm của số không đúng tạo ra phân phối đặc trưng bất khả thi trên thiết bị thực; và độ chính xác số học khi nhúng tham số vào mã nguồn có thể gây sai lệch tích lũy. Hệ quả là một mô hình đạt 99% trên tập kiểm tra Python hoàn toàn có thể cho kết quả sai lệch khi chạy trên thiết bị, nếu không có cơ chế xác minh tương đồng tương minh.

1.2.4 Hệ sinh thái thiết bị phân mảnh

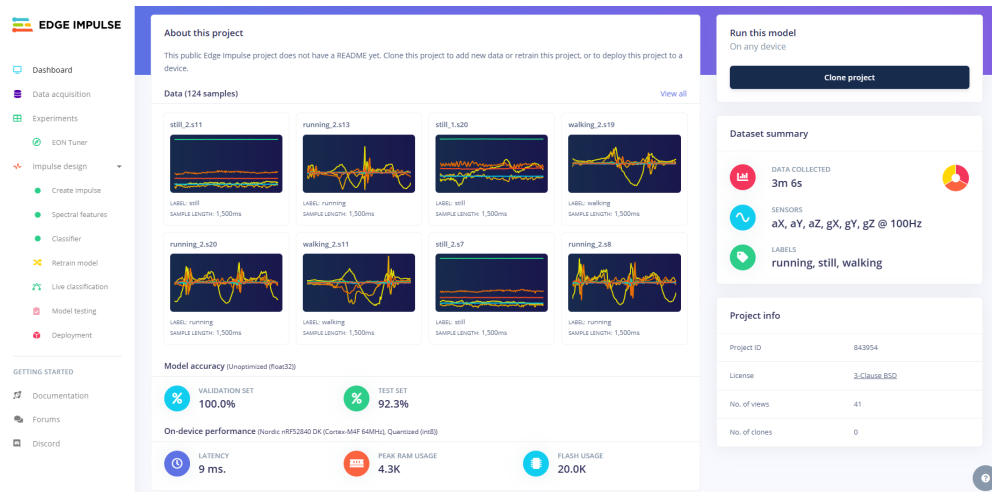
Khác với phát triển ứng dụng di động chỉ nhắm hai nền tảng Android/iOS, hệ sinh thái vi điều khiển cực kỳ phân mảnh. Cùng một mô hình HAR đã huấn luyện có thể cần triển khai trên Arduino (C++ với Arduino Core), ARM Cortex-M bare-metal (CMSIS-DSP), ESP-IDF (C++ với FreeRTOS), Zephyr RTOS (device tree, build system phức tạp), hay MicroPython (Python nhúng). Mỗi nền tảng có quy ước riêng về cấu trúc file, entry point và API cảm biến. Một nhà phát triển muốn hỗ trợ nhiều thiết bị phải viết và bảo trì mã riêng cho từng nền tảng, trong khi vẫn phải đảm bảo *kết quả nhận dạng nhất quán* trên tất cả.

1.3 Các nền tảng hiện có và hạn chế

1.3.1 Tổng quan

Nhiều nền tảng và công cụ đã được phát triển nhằm đưa ML đến thiết bị biên, đặc biệt trong giai đoạn 2019–2024. Mỗi nền tảng giải quyết một phần bài toán nhưng đều có hạn chế đáng kể đối với quy trình HAR dựa trên IMU. Hình 1.3.1 minh họa giao diện Edge Impulse — nền tảng phổ biến nhất hiện nay.

1.3. Các nền tảng hiện có và hạn chế



Hình 1.3.1: Giao diện Edge Impulse (trang thiết kế Impulse): quy trình được định nghĩa bằng các block cố định, giới hạn khả năng tùy chỉnh tiền xử lý và thuật toán

1.3.2 Phân tích các nền tảng chính

Edge Impulse^[14]. Nền tảng đám mây phổ biến nhất cho TinyML với EON Compiler tối ưu mã đầu ra và hỗ trợ 85+ mục tiêu vi điều khiển. Tuy nhiên, tiền xử lý bị cố định sau khi tải dữ liệu lên, không hỗ trợ phân đoạn tương tác; thuật toán chủ yếu giới hạn ở neural network; mã sinh ra là hộp đen không thể kiểm tra logic trích xuất đặc trưng; và dữ liệu phải lưu trên server của hãng, không phù hợp cho dữ liệu y tế nhạy cảm.

SensiML Analytics Toolkit^[35]. Hỗ trợ AutoML với hàng trăm đặc trưng ứng viên và Knowledge Packs cho MCU cụ thể. Hạn chế chính là chi phí rất cao, định dạng độc quyền, và lựa chọn đặc trưng theo kiểu hộp đen không giải thích được.

TensorFlow Lite Micro^[12]. Framework mã nguồn mở cung cấp runtime cho mạng nơ-ron lượng tử hóa trên MCU, tích hợp CMSIS-NN cho ARM và hỗ trợ INT8 quantization. Tuy nhiên, TFLite Micro chỉ hỗ trợ deep learning — không triển khai được Random Forest hay SVM; không có công cụ tiền xử lý hay trích xuất đặc trưng; và interpreter runtime chiếm lượng flash đáng kể, đòi hỏi người dùng tự xây dựng toàn bộ pipeline từ đầu.

Teachable Machine (Google). Giao diện no-code phù hợp cho tạo mẫu và giáo dục, nhưng thiếu trích xuất đặc trưng nâng cao, phân đoạn chuỗi thời gian, và chỉ xuất TensorFlow.js — không chạy trên MCU.

1.4. Phát biểu bài toán và mục tiêu nghiên cứu

1.3.3 Tổng hợp khoảng trống

Bảng 1.3.1 tổng hợp so sánh giữa các nền tảng và framework đề xuất.

Bảng 1.3.1: So sánh các nền tảng Edge ML cho nhận dạng hoạt động

Tiêu chí	Edge Impulse	SensiML	TFLite Micro	Framework đề xuất
Chi phí	\$20–99/tháng	\$99–500/tháng	Miễn phí	Miễn phí (mã nguồn mở)
Tiền xử lý tương tác	Hạn chế	Có (analytics)	Không	Có (kéo thả trên biểu đồ)
Thuật toán	NN, transfer	RF, SVM, NN	Chỉ NN	RF, SVM, MLP, CNN
Tạo mã	C++ (Arduino, ARM)	C (MCU cụ thể)	C++ (runtime)	C/C++/MicroPython đa đích
Tối ưu hóa	Auto (EON)	AutoML	Lượng tử hóa	4 chế độ rõ ràng
Kiểm soát dữ liệu	Đám mây	Local/cloud	Local	Local hoàn toàn
Xác minh tương đồng	Không minh bạch	Không	Không	Có (tự động)

Điểm chung của các nền tảng hiện có: thiếu giao diện phân đoạn dữ liệu tương tác, bị giới hạn thuật toán hoặc chi phí cao, và sinh mã cho số ít nền tảng đích cố định. Đặc biệt, không nền tảng nào cung cấp cơ chế **xác minh tương đồng huấn luyện–triển khai** minh bạch. Khoảng trống này đặc biệt quan trọng trong bối cảnh nghiên cứu và giáo dục, nơi nhà nghiên cứu cần hiểu rõ pipeline, và các dự án nhỏ không có ngân sách cho giấy phép thương mại. **Luận văn này giải quyết khoảng trống đó** bằng một framework mã nguồn mở, minh bạch, hỗ trợ đa thuật toán và đa thiết bị.

1.4 Phát biểu bài toán và mục tiêu nghiên cứu

1.4.1 Phát biểu bài toán

Từ các phân tích trên, bài toán nghiên cứu được phát biểu như sau:

Thiết kế và hiện thực một framework mã nguồn mở cho phép người dùng thực hiện toàn bộ quy trình HAR dựa trên cảm biến IMU — từ thu thập dữ liệu, tiền xử lý tương tác, trích xuất đặc trưng, huấn luyện mô hình đa thuật toán, đến tự động sinh mã triển khai tối ưu cho nhiều họ vi điều khiển — đồng thời đảm bảo tương đồng giữa kết quả huấn luyện và kết quả suy luận trên thiết bị biên.

1.4. Phát biểu bài toán và mục tiêu nghiên cứu

Bài toán này đòi hỏi giải quyết đồng thời bốn thách thức: (1) hạ thấp rào cản kỹ thuật để người dùng không chuyên về nhúng vẫn triển khai được; (2) hỗ trợ nhiều thuật toán ML phù hợp cho tập dữ liệu nhỏ đến lớn; (3) sinh mã cho hệ sinh thái thiết bị phân mảnh; và (4) đảm bảo tính minh bạch và khả năng xác minh mã đã sinh.

1.4.2 Mục tiêu nghiên cứu

1. **Giao diện tiền xử lý tương tác:** Phát triển cơ chế phân đoạn và gán nhãn dữ liệu cảm biến trực tiếp trên biểu đồ tín hiệu bằng kéo thả, giảm thời gian chuẩn bị dữ liệu so với viết mã thủ công.
2. **Hỗ trợ đa thuật toán với giao diện thống nhất:** Tích hợp Random Forest, SVM, MLP, CNN với giao diện huấn luyện chung, cho phép so sánh và chọn mô hình phù hợp với từng ràng buộc phần cứng.
3. **Kiến trúc sinh mã đa đích:** Thiết kế hệ thống sinh mã phân lớp ánh xạ cùng một mô hình sang nhiều họ thiết bị (Arduino, ARM Cortex-M, ESP-IDF, Zephyr, MicroPython) với bốn chế độ tối ưu hóa (accuracy, speed, power, balanced).
4. **Đảm bảo tương đồng huấn luyện–triển khai:** Xây dựng cơ chế xác minh mã C++ sinh ra tái tạo chính xác công thức trích xuất đặc trưng, tham số chuẩn hóa, thứ tự đặc trưng và trọng số mô hình, đảm bảo nhất quán kết quả suy luận giữa Python và MCU.
5. **Xác thực trên phần cứng thực:** Đánh giá framework trên thiết bị ARM Cortex-M4F với dữ liệu HAR thực tế, đo thời gian suy luận, mức sử dụng bộ nhớ, và so sánh trực tiếp với nền tảng thương mại Edge Impulse.

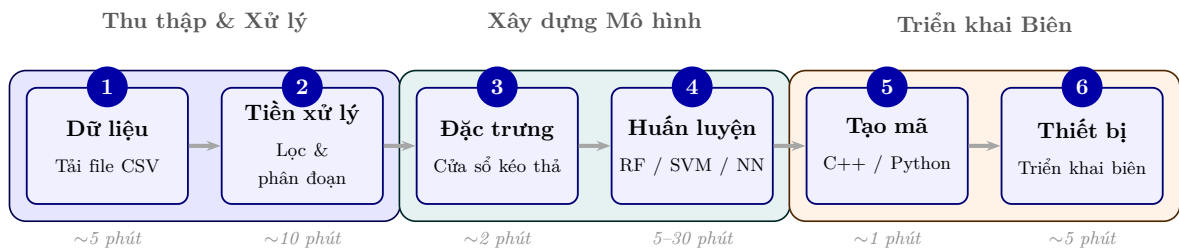
1.4.3 Đóng góp chính

Nghiên cứu này đóng góp vào lĩnh vực Edge ML và nhận dạng hoạt động qua các điểm mới sau:

1. **Framework tích hợp toàn diện:** Lần đầu tiên kết hợp tiền xử lý tương tác (kéo thả trên biểu đồ), trích xuất đặc trưng minh bạch, huấn luyện đa thuật toán, và sinh mã đa đích trong cùng một hệ thống mã nguồn mở.
2. **Cơ chế xác minh tương đồng huấn luyện–triển khai:** Quy trình kiểm tra 12 điểm đảm bảo mã C++ sinh ra tái tạo chính xác kết quả Python — giải quyết vấn đề training–deployment gap mà các nền tảng hiện có bỏ qua.

3. **Kiến trúc sinh mã đa chiều:** Mô hình Factory Pattern ánh xạ ba trục (loại mô hình $\times$ nền tảng đích $\times$ backend triển khai), cho phép một mô hình đã huấn luyện xuất đồng thời sang nhiều họ thiết bị mà không cần huấn luyện lại.
4. **Phát hiện kỹ thuật mới:** Tài liệu hóa các lỗi tương đồng chưa được báo cáo trong tài liệu (zero-padding artifact, sai lệch population vs sample std trong kurtosis, feature order mismatch) — kiến thức có giá trị cho cộng đồng TinyML.
5. **Kết quả thực nghiệm:** Đạt 97–99% trên bài toán 5 lớp, vượt trội Edge Impulse trên cùng tập dữ liệu ($F1=1,00$ vs $0,96$), với mức sử dụng flash nhỏ hơn đáng kể (18–23% vs 51%) và thời gian suy luận 20 ms.

Hình 1.4.1 tổng hợp quy trình làm việc hoàn chỉnh mà framework cung cấp.



Hình 1.4.1: Quy trình làm việc từ dữ liệu thô đến triển khai biên (tổng ước tính 30–60 phút)

1.5 Phạm vi nghiên cứu

Nghiên cứu tập trung vào HAR dựa trên IMU 6 trục (gia tốc kế + con quay hồi chuyển), sử dụng các thuật toán học máy có giám sát. Đánh giá phần cứng định lượng tập trung trên nền tảng tham chiếu Seeed XIAO nRF52840 (ARM Cortex-M4F, 64 MHz, 256 KB RAM); các nền tảng đích còn lại (ESP-IDF, Zephyr, MicroPython) được phân tích ở mức sinh mã và khả năng biên dịch.

Nghiên cứu không bao gồm: hợp nhất cảm biến đa phương thức (camera, âm thanh); nhập mô hình từ framework bên ngoài (TensorFlow SavedModel, ONNX); cập nhật mô hình trên thiết bị (on-device learning); và ứng dụng y tế đòi hỏi chứng nhận lâm sàng.

1.6 Tổ chức luận văn

Luận văn gồm sáu chương. **Chương 2** đánh giá các công trình liên quan về hệ thống HAR, framework học máy cho thiết bị biên, và các nền tảng triển khai edge ML

hiện có. **Chương 3** trình bày phương pháp luận: quy trình tiền xử lý tín hiệu, các phương pháp trích xuất đặc trưng (miền thời gian, miền tần số, bất biến hướng), huấn luyện đa thuật toán, và chiến lược sinh mã đảm bảo tương đồng huấn luyện–triển khai. **Chương 4** mô tả thiết kế và hiện thực hệ thống, bao gồm kiến trúc phần mềm, giao diện người dùng và chi tiết triển khai các module. **Chương 5** báo cáo kết quả thực nghiệm: hiệu suất phân loại của năm thuật toán trên bài toán 3 và 5 lớp, đặc điểm triển khai trên thiết bị thực (thời gian suy luận, mức sử dụng bộ nhớ), và so sánh trực tiếp với Edge Impulse. **Chương 6** trình bày kết luận, thảo luận các phát hiện chính, hạn chế và hướng phát triển tương lai.

Chương 2

Các nghiên cứu liên quan

2.1 Tổng quan

Chương này trình bày các nghiên cứu liên quan trên các lĩnh vực chính: ứng dụng của theo dõi chuyển động, cảm biến quán tính cho HAR, các kiến trúc AI phù hợp, phương pháp tiền xử lý và trích xuất đặc trưng, kỹ thuật tối ưu hóa triển khai biên, và các khoảng trống nghiên cứu cần giải quyết.

2.2 Các lĩnh vực ứng dụng của theo dõi chuyển động

Theo dõi chuyển động con người là công nghệ nền tảng trong nhiều lĩnh vực. Trong y tế, cảm biến đeo cho phép phân tích dáng đi, phát hiện té ngã (độ nhạy 95–98%^[39]), và giám sát phục hồi chức năng^[45]. Thể thao tận dụng dữ liệu quán tính và sinh cơ học để tối ưu hiệu suất và phòng ngừa chấn thương. VR/AR phụ thuộc vào theo dõi khung xương chính xác cho trải nghiệm nhập vai. Ứng dụng công nghiệp gồm giám sát an toàn lao động, đạt 94,3% trong phát hiện tư thế nguy hiểm tại công trường^[40]. Nhu cầu đa dạng này tạo động lực cho các framework theo dõi chuyển động hiệu quả, có khả năng suy luận biên thời gian thực với độ chính xác 90%+.

2.3 Theo dõi chuyển động dựa trên cảm biến

Đơn vị đo lường quán tính (IMU)—kết hợp gia tốc kế và con quay hồi chuyển—được sử dụng rộng rãi trong theo dõi chuyển động nhờ tiêu thụ điện năng thấp (2–10 mA),

chi phí thấp (\$1–5/đơn vị) và phù hợp để triển khai trên thiết bị biên. Filippeschi và cộng sự^[15] tổng hợp các thách thức chính: trôi cảm biến ($0,1\text{--}1^\circ/\text{s}$ cho con quay hồi chuyển), lỗi tích lũy trong ước lượng hướng, và yêu cầu hiệu chuẩn. Phương pháp hợp nhất đa cảm biến (IMU + từ kế + cảm biến áp suất) cải thiện độ chính xác 2–5% nhưng tăng độ phức tạp và chi phí năng lượng. Xia và cộng sự^[44] đã chứng minh hệ thống IMU đơn đạt 96,8% trên bài toán HAR 12 lớp với kiến trúc LSTM-CNN, xác nhận tính khả thi của cách tiếp cận cảm biến đơn.

Các tập dữ liệu chuẩn UCI-HAR^[4] và WISDM^[6] đều sử dụng IMU sáu trục gắn trên cơ thể ở 50–100 Hz, xác lập cấu hình này làm thiết lập tham chiếu cho nghiên cứu HAR trên thiết bị đeo. Framework trong luận văn kế thừa cấu hình đó—thu thập dữ liệu gia tốc kế và con quay hồi chuyển ba trục ở 100 Hz—tập trung vào quy trình IMU đơn với khả năng mở rộng đa cảm biến.

2.4 Theo dõi chuyển động dựa trên thị giác

Các phương pháp thị giác máy tính sử dụng camera và ước lượng tư thế (MediaPipe^[17], OpenPose) cho phép tái tạo toàn thân (17+ điểm khóa, 30 FPS) và được dùng trong hoạt hình, phân tích lâm sàng. Tuy nhiên, hệ thống dựa trên thị giác gặp các hạn chế: che khuất trong cảnh đông đúc, nhạy với điều kiện ánh sáng, lo ngại quyền riêng tư khi thu hình ảnh, và tiêu thụ điện năng cao (500–2000 mW so với 20–50 mW của IMU). Đối với giám sát liên tục trên thiết bị đeo, nơi ưu tiên tuổi thọ pin (ngày đến tuần), các giải pháp IMU thuần túy vẫn là lựa chọn phù hợp nhất. Do đó, luận văn này tập trung hoàn toàn vào cảm biến quán tính.

2.5 AI trong theo dõi chuyển động

Các mô hình AI chuyển đổi dữ liệu chuyển động thô thành các nhận dạng ngữ nghĩa—phân loại hoạt động, nhận dạng tư thế và dự báo quỹ đạo. Phương pháp học máy truyền thống gồm Support Vector Machines (SVM)^[11] và Random Forests^[7] đạt 88–93% trên các benchmark chuẩn (UCI-HAR, WISDM) với bộ nhớ thời gian chạy 50–200 KB, phù hợp thiết bị biên^[41].

Các kiến trúc học sâu cải thiện hiệu suất: CNN đạt 92–95% (trích xuất đặc trưng không gian từ cửa sổ cảm biến), RNN/LSTM đạt 94–97% (nắm bắt phụ thuộc thời gian^[44]), và transformer đạt 96–98% trên tập dữ liệu phức tạp^[36]. Tuy nhiên, mô hình học sâu đòi hỏi tập dữ liệu lớn hơn (10.000+ mẫu so với 1.000–5.000 cho ML cổ điển),

bộ nhớ cao hơn (500 KB–2 MB) và chiến lược lượng tử hóa phức tạp hơn khi triển khai biên^[25].

Nghiên cứu so sánh^[41] cho thấy ML cổ điển đạt 90–93% với kích thước nhỏ hơn $10\times$ và suy luận nhanh hơn $5\text{--}20\times$ (12–18 ms so với 60–120 ms) trên Cortex-M4. Framework đề xuất hỗ trợ cả hai chiến lược—mô hình cổ điển cho kịch bản giới hạn tài nguyên, mạng nơ-ron nhẹ cho ứng dụng ưu tiên độ chính xác—với các chế độ tối ưu hóa giúp cân nhắc đánh đổi một cách có căn cứ.

2.6 Các framework và nền tảng hiện có

Như đã tổng quan trong Chương 1, nhiều nền tảng đã xuất hiện phục vụ phát triển mô hình AI cho thiết bị biên. Về mặt kỹ thuật, các hạn chế chính bao gồm:

- **Edge Impulse**^[14]: Quy trình tự động hóa cao nhưng cố định cửa sổ tiền xử lý sau khi tải dữ liệu; tinh chỉnh đòi hỏi tải lại toàn bộ. Logic tiền xử lý không minh bạch.
- **SensiML**^[35]: AutoML với 200+ đặc trưng ứng viên nhưng sử dụng định dạng độc quyền và lựa chọn đặc trưng kiểu hộp đen.
- **TensorFlow Lite Micro**^[12]: Runtime hiệu quả cho mạng nơ-ron lượng tử hóa trên vi điều khiển, nhưng chỉ hỗ trợ mạng nơ-ron và yêu cầu thiết kế kiến trúc thủ công.
- **Teachable Machine**^[18]: Giao diện không mã cho tạo mẫu nhanh, nhưng thiếu trích xuất đặc trưng nâng cao và tối ưu hóa theo phần cứng.
- **AutoML/NAS**^[10]: Tìm kiến trúc tối ưu dưới ràng buộc tài nguyên, nhưng đòi hỏi tài nguyên tính toán lớn và thiếu minh bạch.

Framework đề xuất giải quyết các hạn chế này bằng cách cung cấp: (1) tiền xử lý tương tác với phân đoạn kéo thả, (2) trích xuất đặc trưng minh bạch với kiểm tra từng đặc trưng, (3) tối ưu hóa đa mục tiêu theo ràng buộc phần cứng, và (4) mã nguồn mở không rào cản chi phí.

2.7 Tiền xử lý dữ liệu và tạo cửa sổ

Chất lượng nhận dạng hoạt động phụ thuộc nhiều vào chiến lược phân đoạn và tạo cửa sổ. Banos và cộng sự^[6] đánh giá ảnh hưởng của kích thước cửa sổ: 1 giây đạt

89,3%, 2,5 giây đạt 92,1%, 5 giây đạt 93,4% nhưng giảm khả năng phản hồi. Chồng lấp 50% cải thiện 2–4% so với cửa sổ không chồng lấp^[39] với chi phí tính toán gấp đôi.

Lựa chọn bộ lọc ảnh hưởng đáng kể đến giảm nhiễu: bộ lọc thông thấp Butterworth (cutoff 20 Hz) bảo tồn đặc tính chuyển động trong khi làm suy giảm nhiễu cảm biến, cải thiện 3–7%^[15]. Tuy nhiên, quy trình tiền xử lý thông thường đòi hỏi sửa mã thủ công để điều chỉnh tham số lọc—các nhà phát triển thường dành 40–60% thời gian dự án cho bước này^[26]. Nghiên cứu về học máy tương tác^[3] chỉ ra rằng giao diện trực quan giảm 35% thời gian lắp lại và 28% lỗi so với quy trình dựa trên mã. Framework đề xuất áp dụng nguyên tắc này: cho phép căn chỉnh trực quan tín hiệu thô và đã xử lý, điều chỉnh cửa sổ tương tác với phản hồi tức thì, và gán nhãn nhanh.

2.8 Trích xuất và biểu diễn đặc trưng

Đặc trưng thủ công miền thời gian (trung bình, phương sai, RMS, độ lệch, độ nhọn, tỷ lệ qua không) và miền tần số (năng lượng phổ, tần số thống trị, entropy phổ) vẫn hiệu quả cho ML cổ điển khi được trích xuất có hệ thống^[27]. Bộ đặc trưng điển hình gồm 60–150 đặc trưng mỗi cửa sổ^[41]. Lựa chọn đặc trưng (thông tin hỗ tương, kiểm định F ANOVA, loại bỏ đệ quy) có thể giảm 30–50% chiều với mất độ chính xác dưới 2%, đồng thời cải thiện tốc độ suy luận 15–30% trên vi điều khiển^[6].

Đặc trưng miền tần số đặt ra thách thức triển khai: FFT trên hệ thống nhúng không có bộ tăng tốc phần cứng đòi hỏi $O(N \log N)$ phép toán dấu phẩy động, tiêu thụ 50–150 ms mỗi cửa sổ trên Cortex-M4 64 MHz^[5]. Các giải pháp nhẹ bao gồm đếm qua không, đỉnh tự tương quan, hoặc thư viện FFT tối ưu (ARM CMSIS-DSP). Quan trọng hơn, tính tương đồng đặc trưng giữa huấn luyện (Python NumPy FFT) và triển khai (C++ FFT điểm cố định) cần được xác minh cẩn thận.

Mặc dù mạng sâu end-to-end có thể học đặc trưng ngầm định, trích xuất đặc trưng tường minh cải thiện khả năng diễn giải và minh bạch triển khai—đặc biệt quan trọng trong các lĩnh vực y tế hoặc an toàn. Framework kết hợp các mô-đun đặc trưng hoán đổi được, xác minh tính nhất quán huấn luyện-triển khai, và hỗ trợ chẩn đoán từng cửa sổ.

2.9 Triển khai biên và tối ưu hóa

Ràng buộc tài nguyên định hình các thách thức triển khai biên: RAM hạn chế (32–256 KB), flash (256 KB–1 MB), không có hoặc hạn chế FPU, và ngân sách điện năng 10–50 mW cho thiết bị đeo^[5;30]. Benchmark MLPerf Tiny^[5] thiết lập số liệu tham

2.10. Các khoảng trống trong công trình hiện có

chiều: Cortex-M4F 64 MHz đạt 10,3 suy luận/giây cho phát hiện từ khóa (mô hình 49 KB), 15,2 suy luận/giây cho phát hiện bất thường (mô hình 32 KB). Các chiến lược tối ưu hóa chính:

- **Lượng tử hóa:** Float 32-bit sang số nguyên 8-bit giảm kích thước $4\times$, tăng tốc suy luận $2\text{--}3\times$ với suy giảm 0,5–2%^[25].
- **Cắt tia:** Loại bỏ 40–70% trọng số (theo độ lớn hoặc cấu trúc) cho tăng tốc $2\text{--}3\times$ với mất dưới 1%^[33].
- **Chưng cất kiến thức:** Mô hình nhỏ (10–20% tham số) bắt chước mô hình lớn, đạt 85–95% hiệu suất với kích thước nhỏ hơn $5\text{--}10\times$ ^[13].
- **ML cổ điển:** Random Forests (độ sâu ≤ 10) và SVM (100–500 support vectors) đạt 88–93% với 50–200 KB bộ nhớ, suy luận 12–25 ms^[41].

Các khảo sát TinyML^[13;30] nhận diện ba mẫu triển khai: (1) ưu tiên độ trễ dùng ML cổ điển (5–20 ms), (2) ưu tiên độ chính xác dùng CNN/LSTM nhẹ (40–80 ms, cao hơn 1–2%), (3) ưu tiên điện năng dùng lấy mẫu thưa và lượng tử hóa mạnh. Thành phần sinh mã của framework thực hiện xuất mã C/C++ theo nền tảng với chuyển đổi tham số phù hợp ràng buộc, và các chế độ tối ưu hóa đa mục tiêu cho phép lựa chọn có căn cứ giữa độ chính xác, tốc độ và tiêu thụ điện năng.

2.10 Các khoảng trống trong công trình hiện có

Mặc dù hệ sinh thái công cụ edge AI đã khá hoàn thiện, một số khoảng trống quan trọng vẫn tồn tại:

1. **Tiền xử lý tương tác hạn chế:** Ít hệ thống cho phép kiểm tra trực quan tín hiệu thô so với đã lọc và tinh chỉnh phân đoạn thủ công trong cùng môi trường.
2. **Thiếu minh bạch trong trích xuất đặc trưng:** Các công cụ tạo đặc trưng tự động che giấu bước dẫn xuất, cản trở mục đích giáo dục và tái tạo nghiên cứu.
3. **Sinh mã triển khai thiếu linh hoạt:** Mã đầu ra thường là khối nguyên, khó tùy chỉnh hoặc thích ứng theo phần cứng cụ thể.
4. **Đánh đổi giữa dễ sử dụng và kiểm soát:** Các nền tảng hiện tại thường thiên về một cực—dễ dùng như Teachable Machine, hoặc linh hoạt cao như

TensorFlow—ít nền tảng cân bằng được cả hai cho quy trình xử lý dữ liệu cảm biến chuyển động.

Framework đề xuất giải quyết các khoảng trống này bằng cách thống nhất tiền xử lý tương tác, trích xuất đặc trưng minh bạch, huấn luyện modular và sinh mã tối ưu hóa dưới một kiến trúc mở rộng.

2.11 Tóm tắt

Các nghiên cứu và nền tảng công nghiệp trước đây cung cấp nền tảng vững chắc cho theo dõi chuyển động, nhưng chưa có nền tảng nào cung cấp quy trình tích hợp, minh bạch và thân thiện với người dùng, được tối ưu hóa cho triển khai biên dựa trên IMU với khả năng phân đoạn dữ liệu tương tác. Công trình này xây dựng trên các thuật toán và thực hành edge ML đã được thiết lập, đồng thời đóng góp phương pháp mới giúp phổ cập quy trình phát triển, tăng tốc lặp lại và cải thiện tính sẵn sàng triển khai cho các ứng dụng theo dõi chuyển động thực tế.

Chương 3

Cơ sở lý thuyết

Chương này trình bày cơ sở lý thuyết và phương pháp cho từng giai đoạn trong pipeline: lý do chọn cảm biến IMU, các kỹ thuật lọc và phân đoạn tín hiệu, chiến lược trích xuất đặc trưng, thuật toán học máy, và hệ thống tạo mã triển khai. Kiến trúc hệ thống tổng thể và các quyết định thiết kế phần mềm được trình bày trong Chương 4.

3.1 Dữ liệu cảm biến quán tính cho nhận dạng hoạt động

Cảm biến gia tốc kế đo gia tốc tuyến tính trên ba trục (bao gồm thành phần trọng lực), phản ánh biên độ và tính tuần hoàn của chuyển động. Tuy nhiên, khi thiết bị xoay, vectơ trọng trường chiếu lên các trục thay đổi, gây nhập nhằng giữa chuyển động tịnh tiến và thay đổi định hướng. Cảm biến con quay hồi chuyển đo vận tốc góc ($^{\circ}/s$), không bị ảnh hưởng bởi trọng trường và nắm bắt chính xác động học xoay—giúp phân biệt các cặp hoạt động khó như leo/xuống cầu thang hay đi bộ/giậm chân tại chỗ. Tổ hợp sáu trục (3 gia tốc kế + 3 con quay hồi chuyển) cung cấp biểu diễn động học đầy đủ^[15], và đây là cấu hình chuẩn trong các tập dữ liệu HAR như UCI-HAR^[4] và PAMAP2^[31].

Framework không áp đặt tần số lấy mẫu cố định, nhưng có một ràng buộc bắt buộc: **tần số lấy mẫu phải nhất quán giữa giai đoạn huấn luyện và suy luận triển khai**. Các đặc trưng thống kê được tính theo *số mẫu* trong cửa sổ, không theo thời gian tuyệt đối—nếu tần số thay đổi, cùng một cửa sổ 150 mẫu sẽ đại diện cho khoảng thời gian khác nhau, làm lệch toàn bộ phân phối đặc trưng. Để đảm bảo tính nhất quán, framework nhúng tần số lấy mẫu đã chọn dưới dạng hằng số biên dịch vào mã triển khai. Đối với HAR, dải tần phổ hoạt động phổ biến nằm trong 1–20 Hz^[43],

nên tần số lấy mẫu 50–100 Hz là phù hợp.

3.2 Quy trình tiền xử lý tương tác

3.2.1 Tải lên và trực quan hóa dữ liệu

Người dùng tải lên các tập dữ liệu CSV chứa các phép đo IMU, tổ chức theo hoạt động (mỗi tệp tương ứng với một lớp). Framework tự động phát hiện tên cột cảm biến, tần số lấy mẫu và hiển thị biểu đồ đồng bộ của tín hiệu thô trên tất cả các trục, cho phép kiểm tra trực quan chất lượng dữ liệu trước khi tiền xử lý (xem giao diện tại Hình 4.2.1, Chương 4).

3.2.2 Các phương pháp lọc và làm sạch tín hiệu

Dữ liệu IMU thô chứa nhiều cảm biến, nhiễu điện từ và các xung bất thường cần được loại bỏ trước khi trích xuất đặc trưng. Framework cung cấp năm phương pháp lọc, mỗi phương pháp có đặc tính khác nhau về mức độ làm mịn, tính nhân quả và—quan trọng nhất cho triển khai nhúng—*tính tương đồng huấn luyện-triển khai* (training-deployment parity). Người dùng chọn phương pháp phù hợp qua giao diện tương tác; kết quả lọc được trực quan hóa cùng tín hiệu gốc để đánh giá hiệu quả trước khi phân đoạn.

Loại bỏ ngoại lệ (outlier removal)

Xác định và loại bỏ các mẫu có giá trị vượt quá ngưỡng $k\sigma$ so với trung bình cục bộ (mặc định $k = 3$). Đây là bước làm sạch thô, xử lý các xung nhiễu điện từ hoặc bất thường phần cứng (spike artifacts) tạo ra giá trị nằm ngoài phạm vi vật lý hợp lệ của cảm biến. Phương pháp này không thay đổi hình dạng tín hiệu mà chỉ loại bỏ các điểm dữ liệu cô lập rõ ràng sai, nên không có vấn đề parity—nó chỉ áp dụng một lần trên tập dữ liệu huấn luyện (offline) và không cần triển khai trên thiết bị.

Bộ lọc Butterworth low-pass

Bộ lọc IIR Butterworth bậc n với tần số cắt f_c có đáp ứng biên độ cực đại phẳng (maximally flat):

$$|H(f)|^2 = \frac{1}{1 + (f/f_c)^{2n}}$$

Để tránh lệch pha, kỹ thuật lọc hai chiều (forward-backward filtfilt) được sử dụng, cho đáp ứng $|H_{\text{eff}}|^2 = |H|^4$ với pha bằng không. Cấu hình mặc định: bậc 2, tần số cắt 5 Hz.

Parity triển khai: Vì HAR đệm toàn bộ của số W mẫu trước khi xử lý, thiết bị thực hiện cùng thao tác lọc hai chiều trên bộ đệm. Các hệ số được tính trước tại thời điểm sinh mã và nhúng dưới dạng hằng số.

Bộ lọc Savitzky-Golay

Bộ lọc Savitzky-Golay^[32] khớp đa thức bậc d lên cửa sổ $2m+1$ mẫu bằng bình phương tối thiểu, cho kết quả tương đương tích chập FIR với các hệ số c_j được tính từ ma trận Vandermonde. So với trung bình động, phương pháp bảo tồn tốt hơn các đỉnh tín hiệu. Cấu hình mặc định: $m = 2$ (cửa sổ 5 mẫu), $d = 2$.

Parity triển khai: Các hệ số c_j được tính trước và nhúng vào mã C++ dưới dạng mảng hằng số; tích chập áp dụng trên bộ đệm cửa sổ với mở rộng biên bằng lặp giá trị biên.

Bộ lọc FFT low-pass (miền tần số)

Lọc trong miền tần số thực hiện ba bước: DFT, triệt tiêu các bin tần số cao ($k > k_c = \lfloor f_c N / f_s \rfloor$), rồi IDFT. Đặc điểm: đáp ứng tần số lý tưởng (brick-wall) và không méo pha. Chi phí bộ nhớ $\mathcal{O}(N)$.

Parity triển khai: Thiết bị thực thi cùng ba bước DFT $\rightarrow$ masking $\rightarrow$ IDFT trên bộ đệm cửa sổ với cùng k_c , dùng số dấu phẩy động 32-bit nhất quán.

Bộ lọc Kalman (constant-velocity)

Bộ lọc Kalman constant-velocity 1D^[22] áp dụng cho mỗi kênh cảm biến IMU được đề xuất như giải pháp lọc có *khoảng cách parity bằng không*. Bộ lọc Kalman hoạt động hoàn toàn theo chiều thuận (forward-only, causal) trong cả hai môi trường: Python huấn luyện và C++ triển khai—loại bỏ lớp lỗi mà tất cả bốn phương pháp trên gặp phải.

Mô hình toán học. Mỗi kênh cảm biến được mô hình hóa bằng trạng thái gồm vị trí (giá trị cảm biến) và vận tốc:

$$\mathbf{x}_k = \begin{bmatrix} p_k \\ v_k \end{bmatrix}, \quad F = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}, \quad H = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

Ma trận hiệp phương sai nhiễu quá trình:

$$Q = q \begin{bmatrix} \frac{\Delta t^3}{3} & \frac{\Delta t^2}{2} \\ \frac{\Delta t^2}{2} & \Delta t \end{bmatrix}$$

3.2. Quy trình tiền xử lý tương tác

Nhiều đo: $R = r$ (scalar). Hai tham số điều chỉnh được: q (process noise, mặc định 10^{-3} —nhỏ hơn = mượt hơn) và r (measurement noise, mặc định 10^{-1} —lớn hơn = mượt hơn). Kalman gain K_k tự động thích ứng theo tín hiệu: tăng khi tín hiệu biến đổi nhanh (bám sát), giảm khi tín hiệu ổn định (lọc mạnh hơn).

So sánh tổng hợp các phương pháp lọc

Bảng 3.2.1 tổng hợp năm phương pháp theo tiêu chí quan trọng nhất cho hệ thống triển khai nhúng:

Bảng 3.2.1: So sánh các phương pháp lọc theo tính tương đồng huấn luyện-triển khai

Phương pháp	Causal	Parity	Ghi chú triển khai
Loại bỏ ngoại lệ	—	✓	Chỉ offline (không cần trên thiết bị)
Butterworth LP	Không [†]	✓	Per-window two-pass filter trên bộ đệm cửa sổ
Savitzky-Golay	Không [†]	✓	FIR convolution với hệ số tiền tính
FFT low-pass	Không [†]	✓	DFT trực tiếp trên bộ đệm cửa sổ
Kalman filter	Có	✓	Forward-only; hoạt động cả streaming lẫn windowed

[†]Non-causal trong streaming; parity đạt được nhờ HAR đệm toàn bộ cửa sổ trước khi trích xuất đặc trưng, cho phép áp dụng bộ lọc hai chiều trên bộ đệm.

Kết luận thiết kế: tất cả các phương pháp lọc đều đạt parity hoàn toàn khi suy luận HAR thực hiện theo lô cửa sổ, vì quy trình đã đệm toàn bộ W mẫu trước khi trích xuất đặc trưng—ràng buộc non-causal chỉ áp dụng cho streaming theo từng mẫu. Kalman filter có lợi thế bổ sung: hoạt động được trong cả chế độ streaming lẫn lô, phù hợp cho hiển thị tín hiệu thời gian thực trong quá trình thu thập. Loại bỏ ngoại lệ không được nhân bản trên thiết bị vì cần thống kê toàn cục từ toàn bộ bản ghi, trong khi mỗi chu kỳ suy luận chỉ có một cửa sổ 1,5 giây.

3.2.3 Phân đoạn và sinh cửa sổ huấn luyện

Máy học trên tín hiệu liên tục đòi hỏi chuyển đổi chuỗi thời gian thành các vector có độ dài cố định. Framework thực hiện qua hai bước: người dùng xác định các đoạn tín hiệu sạch trên biểu đồ, và hệ thống tự động sinh cửa sổ huấn luyện bằng giải thuật cửa sổ trượt.

Giải thuật cửa sổ trượt tự động

Cho đoạn tín hiệu N mẫu, kích thước cửa sổ W và độ chồng lấp o (%), bước trượt $S = W \cdot (1 - o/100)$, số cửa sổ sinh được:

$$n_w = \left\lfloor \frac{N - W}{S} \right\rfloor + 1 \quad (3.2.1)$$

Ví dụ: đoạn 5 giây (500 mẫu ở 100 Hz) với $W = 150$, $o = 50\%$ sinh ra 6 cửa sổ huấn luyện. Overlap 50% tăng gấp đôi số mẫu từ cùng lượng dữ liệu thô, đồng thời giảm hiệu ứng biên—cùng một sự kiện chuyển động xuất hiện ở vị trí tương đối khác nhau trong các cửa sổ kế tiếp, giúp mô hình học đặc trưng bất biến vị trí. Overlap cao hơn ($>75\%$) tạo ra tương quan cao giữa các cửa sổ liên tiếp, tăng nguy cơ rò rỉ thông tin.

Lựa chọn đoạn tương tác

Giao diện kéo thả cho phép người dùng đánh dấu các vùng sạch trên biểu đồ tín hiệu, bỏ qua đoạn chuyển tiếp hoặc nhiễu. Nhiều đoạn trên cùng một tệp được hỗ trợ. Sau khi xác nhận, cửa sổ trượt chạy tự động trên từng đoạn theo Phương trình 3.2.1. Nhân hoạt động kế thừa từ tệp nguồn nhưng có thể gán lại thủ công. Thiết kế này tách bạch hai quyết định: *đâu là dữ liệu đáng tin cậy* (người dùng kiểm soát) và *cách biến dữ liệu thành mẫu huấn luyện* (hệ thống tự động hóa), giảm đáng kể công sức thu thập trong khi bảo tồn kiểm soát chất lượng (xem Hình 4.3.1, Chương 4).

3.3 Trích xuất đặc trưng

3.3.1 Tham số cửa sổ và ràng buộc nhất quán

Trích xuất đặc trưng thực hiện trên các cửa sổ được sinh theo Mục 3.2.3: $W = 150$ mẫu (1,5 giây ở $f_s = 100$ Hz), bước trượt $S = 75$ (50% chồng lấp). Ràng buộc cốt lõi: (W, S, f_s) phải đồng nhất giữa huấn luyện và suy luận—thay đổi W sau huấn luyện làm lệch phân phối đặc trưng. Framework nhúng W và f_s dưới dạng hằng số trong mã sinh ra.

3.3.2 Các loại trích xuất đặc trưng

Sáu loại trích xuất đặc trưng được thiết kế để đáp ứng các yêu cầu khác nhau về số chiều, khả năng triển khai nhúng và bất biến hướng. Bảng 3.3.1 tóm tắt toàn diện.

Hai loại **bất biến hướng** tính đặc trưng trên biên độ tổng hợp (vector magnitude)—đại lượng bất biến phép xoay—phù hợp khi thiết bị đeo không cố định. Loại `orientation_invariant`

3.3. Trích xuất đặc trưng

Bảng 3.3.1: Sáu loại trích xuất đặc trưng: tín hiệu nguồn, cấu thành và đặc tính triển khai

Loại	Tín hiệu nguồn	Đặc trưng tính	#	Nhúng	OI*
oi_time_only ¹	acc_mag, gyro_mag, acc_jerk_mag	15 thống kê thời gian / tín hiệu	41	✓	✓
orientation_invariant ¹	Như trên + DFT(acc_mag, gyro_mag)	Như trên + 10 phổ / DFT	63	✓	✓
time_domain	aX, aY, aZ, gX, gY, gZ	15 thống kê / trực	90	✓	×
all	aX, aY, aZ, gX, gY, gZ	15 thời gian + 11 tần số / trực	156	×	×
frequency_domain	aX, aY, aZ, gX, gY, gZ	11 đặc trưng phổ / trực	66	×	×
raw	aX, aY, aZ, gX, gY, gZ	Giá trị cảm biến thô	6	✓	×

¹oi_time_only = orientation_invariant_time_only.

*OI = Orientation-Invariant (bất biến hướng). Tín hiệu tổng hợp: $\text{acc_mag} = \sqrt{a_X^2 + a_Y^2 + a_Z^2}$; $\text{gyro_mag} = \sqrt{g_X^2 + g_Y^2 + g_Z^2}$; $\text{acc_jerk_mag} = \frac{d}{dt} \text{acc_mag}$.

(63 đặc trưng) là khuyến nghị mặc định: cân bằng giữa tính phong phú và triển khai nhúng, vì bộ sinh mã tạo DFT bằng tổng sin/cos trực tiếp không cần thư viện FFT.

Loại time_domain (90 đặc trưng) áp dụng 15 thống kê lên từng trục riêng lẻ—mỗi đặc trưng là biểu thức dạng đóng xác định, không phụ thuộc trạng thái, đảm bảo bit-identical trên mọi nền tảng:

- **Xu hướng trung tâm:** mean, median
- **Phân tán:** std, range, IQR
- **Cực trị:** min, max, Q1, Q3
- **Hình dạng:** skewness $\gamma = E[(X - \mu)^3]/\sigma^3$, kurtosis $\kappa = E[(X - \mu)^4]/\sigma^4 - 3$
- **Năng lượng:** RMS, signal energy
- **Xấp xỉ tần số:** zero-crossing rate, mean-crossing rate

Loại all và frequency_domain bổ sung đặc trưng phổ mỗi trục nhưng **không hỗ trợ triển khai nhúng** vì nhạy với sai lệch số, tổn tài nguyên tính toán, và đặc trưng miền thời gian đã đủ hiệu quả trên các tập HAR chuẩn^[6]. Loại raw truyền trực tiếp giá trị cảm biến, dùng làm đầu vào CNN.

3.3.3 Chiến lược đệm cửa sổ

Các cửa sổ có thể ngắn hơn kích thước mục tiêu khi người dùng chọn đoạn ngắn hoặc cửa sổ cuối không đủ dài.

Vấn đề với zero-padding: Đặt giá trị cảm biến bằng 0 tạo ra magnitude gia tốc $\|\mathbf{a}\| = 0$ —vật lý bất khả thi vì gia tốc trọng trường luôn $\approx 9,81 \text{ m/s}^2$. Điều này gây nhiều nghiêm trọng cho phân phối đặc trưng (min, median, Q1 bị kéo về 0).

Giải pháp: Lấp giá trị cuối cùng đến kích thước mục tiêu: $\mathbf{W}' = [w_1, \dots, w_k, \underbrace{w_k, \dots, w_k}_{N-k}]$.

Phương pháp bảo toàn tính thống kê tín hiệu và nhất quán với bộ đệm trên thiết bị biên (luôn chứa dữ liệu thực). Cửa sổ có ít hơn 30% mẫu mục tiêu bị loại bỏ hoàn toàn.

Phát hiện thực nghiệm: Zero-padding khiến mô hình luôn dự đoán sai lớp *walking_downstairs* bất kể hoạt động thực tế, do đặc trưng nằm ngoài phân phối huấn luyện (xem Mục 6.4).

3.3.4 Tăng cường dữ liệu cho huấn luyện

Để cải thiện tính tổng quát hóa và giảm overfitting—đặc biệt với tập dữ liệu nhỏ từ thu thập đối tượng đơn—năm phương pháp tăng cường dữ liệu được áp dụng ở cấp cửa sổ (trước trích xuất đặc trưng)^[21;38]:

1. **Jitter:** Nhiễu Gaussian $\mathcal{N}(0, \sigma^2)$ ($\sigma = 0,05$), mô phỏng nhiễu cảm biến^[38]
2. **Scaling:** Nhân hệ số $s \sim \mathcal{N}(1, \sigma_s^2)$, mô phỏng biên độ chuyển động khác nhau
3. **Rotation:** Ma trận xoay ngẫu nhiên ($\leq 15^\circ$), mô phỏng thay đổi vị trí cảm biến^[37]
4. **Time Warping:** Biến đổi phi tuyến trực thời gian qua nội suy spline, mô phỏng tốc độ thực hiện khác nhau^[24]
5. **Permutation:** Hoán vị $k = 4$ đoạn con, giúp mô hình học đặc trưng cục bộ thay vì phụ thuộc thứ tự toàn cục

Tăng cường nhận biết lớp

Áp dụng biến đổi mạnh lên hoạt động tĩnh (still, standing, sitting) tạo dao động nhỏ giống hoạt động cường độ thấp, gây nhầm lẫn lớp. Framework giải quyết bằng chiến lược phân biệt:

- **Hoạt động động:** Áp dụng đầy đủ cả năm phương pháp ($\sigma_{\text{jitter}} = 0,05$)
- **Hoạt động tĩnh:** Chỉ micro-jitter ($\sigma = 0,01$)—đủ tạo đa dạng mà không phá vỡ đặc tính “gần bất động”

Phân loại tĩnh/động dựa trên ngưỡng biên độ dao động và có thể do người dùng chỉ định thủ công. Chiến lược tuân thủ nguyên tắc rằng tăng cường phải bảo toàn các đặc tính phân biệt của mỗi lớp^[8].

3.3.5 Ngưỡng tin cậy cho từ chối hoạt động không xác định

Trong triển khai thực tế, thiết bị có thể ghi nhận chuyển động không thuộc lớp nào đã huấn luyện. Ngưỡng tin cậy τ (mặc định 0,6) được tích hợp vào mã suy luận: nếu $\max_c P(y = c|\mathbf{x}) < \tau$, dự đoán trả về “unknown” thay vì ép buộc một nhãn:

$$\hat{y} = \begin{cases} \arg \max_c P(y = c|\mathbf{x}) & \text{nếu } \max_c P(y = c|\mathbf{x}) \geq \tau \\ \text{unknown} & \text{ngược lại} \end{cases} \quad (3.3.1)$$

Xác suất từng lớp được tính theo cơ chế có sẵn: softmax cho NN/CNN, tỷ lệ bỏ phiếu cho RF, softmax trên điểm quyết định OvR cho SVM. Giá trị $\tau = 0,6$ yêu cầu tin cậy gấp 3 lần xác suất ngẫu nhiên (bài toán 5 lớp: 20%). So với Edge Impulse (cần huấn luyện khối anomaly detection riêng), cách tiếp cận này không tốn thêm bộ nhớ hay dữ liệu bổ sung.

3.4 Huấn luyện mô hình học máy

3.4.1 Các thuật toán học máy áp dụng

Bốn nhóm thuật toán học máy dưới đây đã được chứng minh hiệu quả trong nhận dạng hoạt động và phù hợp cho triển khai biên, với các đánh đổi khác nhau giữa độ chính xác, yêu cầu dữ liệu và chi phí tài nguyên:

Random Forest (RF) Phương pháp ensemble kết hợp nhiều cây quyết định với bagging và ngẫu nhiên hóa đặc trưng^[7]. Cấu hình điển hình cho bài toán HAR:

- Số cây: 50–200 (tham chiếu: 100 cây), độ sâu tối đa 10–30
- Cân bằng lớp: `class_weight='balanced'` tự động trọng số mẫu tỷ lệ nghịch với tần suất lớp

- Tiêu chí phân nhánh: Độ bất thuần Gini hoặc entropy thông tin

RF đặc biệt phù hợp cho HAR vì không yêu cầu chuẩn hóa đặc trưng; chịu đựng tốt với đặc trưng không liên quan (mỗi lần phân nhánh chỉ xét $\sqrt{D}$ đặc trưng ngẫu nhiên); kết quả giải thích được qua độ quan trọng đặc trưng; ổn định với nhiễu nhờ cơ chế bỏ phiếu đa số.

Support Vector Machine (SVM) Bộ phân loại phân biệt tìm siêu phẳng tối đa hóa biên phân tách giữa các lớp. Cấu hình điển hình:

- Kernel: Radial Basis Function (RBF), phù hợp cho dữ liệu phi tuyến; tham số gamma kiểm soát bán kính ảnh hưởng của từng điểm huấn luyện
- Chiến lược đa lớp: One-vs-One (OvO) hoặc One-vs-Rest (OvR)
- Cân bằng lớp: `class_weight='balanced'`
- Chính quy hóa: Tham số C kiểm soát đánh đổi biên rộng—vi phạm biên ($C \in \{0,1; 1; 10\}$)

SVM phù hợp khi số chiều đặc trưng cao so với số mẫu—điển hình trong HAR với hàng chục đến hàng trăm đặc trưng và vài trăm mẫu. Yêu cầu chuẩn hóa đặc trưng (StandardScaler) trước huấn luyện để đảm bảo các chiều đóng góp đồng đều vào hàm kernel.

Neural Network (NN) Perceptron đa lớp (MLP) với lan truyền ngược, phù hợp học các ánh xạ phi tuyến phức tạp. Một kiến trúc tham chiếu cho bài toán 5 lớp với 63 đặc trưng đầu vào:

- Kiến trúc: [63 đầu vào] $\rightarrow$ [100 ẩn, ReLU] $\rightarrow$ [num_classes, Softmax]
- Trình tối ưu: Adam ($\eta = 0,001$, $\beta_1 = 0,9$, $\beta_2 = 0,999$)
- Chính quy hóa: Phạt L2 $\alpha = 0,0001$; dừng sớm (patience=10)
- Lịch trình: Tối đa 500 epoch, batch đầy đủ

Kiến trúc một lớp ẩn vừa đủ năng lực học mẫu phi tuyến vừa nhỏ gọn cho triển khai nhúng (ma trận trọng số cố định, không phân bổ bộ nhớ động). Kiến trúc tùy chỉnh với nhiều lớp ẩn hơn có thể xây dựng qua PyTorch, với cơ chế xuất trọng số sang định dạng tương thích để tạo mã triển khai.

Convolutional Neural Network (CNN) Mạng tích chập 1D xử lý trực tiếp cửa sổ cảm biến thô (end-to-end), học biểu diễn phân cấp từ tín hiệu mà không cần trích xuất đặc trưng thủ công. Một kiến trúc tham chiếu 1D-CNN:

- Đầu vào: Cửa sổ thô $W \times C$ ($W = 150$ mẫu, $C = 6$ kênh cảm biến)
- Khối tích chập: 2–3 lớp Conv1D (kernel 3–5, bộ lọc 32–64) + BatchNorm + MaxPool
- Phân loại: Flatten + Dense + Dropout (0,3–0,5) + Softmax
- Trình tối ưu: Adam với OneCycleLR scheduler
- Triển khai: Xuất trọng số thành mảng C/C++ hoặc chuyển đổi sang TFLite Micro (INT8)

CNN phù hợp khi tập dữ liệu đủ lớn và các đặc trưng thống kê thủ công chưa đủ phân biệt—mô hình học trực tiếp các biểu diễn phân cấp từ tín hiệu thô. Đổi lại, CNN yêu cầu nhiều dữ liệu huấn luyện hơn và bộ nhớ triển khai lớn hơn (500 KB–2 MB so với 50–200 KB cho mô hình cổ điển).

3.4.2 Giao thức huấn luyện và đánh giá

Chiến lược chia dữ liệu

Giao thức phân chia ba tập được áp dụng^[16]: tập huấn luyện (60%) để khớp tham số, tập xác thực (20%) để điều chỉnh siêu tham số và lựa chọn mô hình, tập kiểm tra (20%) chỉ đánh giá một lần duy nhất sau khi huấn luyện hoàn tất. Tất cả các phân chia sử dụng lấy mẫu phân tầng để bảo tồn tỷ lệ lớp. Xác thực chéo 5-fold có thể thực hiện trên tập train+val kết hợp khi cần so sánh thuật toán với dữ liệu hạn chế, trong khi vẫn giữ tập kiểm tra hoàn toàn độc lập.

Xử lý mất cân bằng lớp

Tham số `class_weight='balanced'` được bật mặc định cho RF và SVM, gán trọng số $w_c \propto 1/n_c$ tỷ lệ nghịch với kích thước lớp. Phân chia phân tầng đảm bảo tỷ lệ lớp nhất quán giữa các tập. Tổ hợp này đảm bảo tất cả các lớp được nhận dạng đáng tin cậy, không chỉ các lớp xuất hiện nhiều nhất.

3.4.3 Tối ưu hóa siêu tham số

Điều chỉnh siêu tham số sử dụng GridSearchCV với xác thực chéo 5-fold. Các không gian tìm kiếm điển hình: RF tìm trên `n_estimators` [50–200], `max_depth` [10–30]; SVM tìm trên `C` [0,1–100] và `gamma` [scale/auto/0,001–0,1]; NN tìm trên kích thước lớp ẩn và tốc độ học. Tối ưu hóa thường cải thiện hiệu suất cơ sở 2–5% (xem giao diện cấu hình huấn luyện tại Hình 4.5.1, Chương 4).

3.5 Tạo mã cho triển khai biên

3.5.1 Kiến trúc

Hệ thống con tạo mã dịch các mô hình đã huấn luyện thành các gói triển khai phù hợp với nhiều họ thiết bị và môi trường thực thi khác nhau, thay vì nhắm đến một bo mạch đơn lẻ. Kiến trúc tuân theo mẫu thiết kế Factory, trong đó generator được chọn theo ba trục: *loại mô hình*, *họ nền tảng đích* và *cách tiếp cận triển khai*. Danh sách khóa nền tảng và backend cụ thể được tóm tắt trong Bảng 3.5.1.

Kiến trúc gồm ba lớp chính:

- **Lớp cơ sở (Base Generator):** cung cấp phần dùng chung gồm sinh tệp, trích xuất đặc trưng, lưu trữ tham số, kiểm tra tương đồng và ví dụ tích hợp.
- **Lớp đặc thù thuật toán:** mỗi thuật toán (RF, SVM, NN, CNN) có generator riêng hiện thực logic suy luận tương ứng.
- **Lớp đặc thù nền tảng:** các generator cho ARM Cortex-M, MicroPython, Zephyr, cùng các backend TFLite Micro và ONNX Runtime, chịu trách nhiệm điều chỉnh mã theo môi trường đích.

3.5. Tạo mã cho triển khai biên

Bảng 3.5.1: Ma trận đích triển khai được hỗ trợ

Nhóm đích	Khóa nền tảng/backend	Gói sinh ra	Mục đích sử dụng chính
Arduino-compatible	arduino, seeed_xiao, esp32, m5stack, teensy	.h + .cpp + .ino	Quy trình Arduino IDE/PlatformIO cho các bo vi điều khiển phổ biến
ARM Cortex-M thuần	arm_cortex_m	.c	Tích hợp bare-metal hoặc CMSIS trên các vi điều khiển ARM
SDK/RTOS native	generic_c, generic_cpp, esp_idf, zephyr	.h + .c/.cpp + ví dụ	Ứng dụng nhúng native, SDK hãng và RTOS
MicroPython	micropython	module .py + ví dụ .py	Nguyên mẫu nhanh hoặc giáo dục trên board hỗ trợ MicroPython
Backend runtime thay thế	tflite_micro, onnx_runtime	mã glue + model nhị phân	Mở rộng cho các pipeline cần runtime engine chuẩn hóa

Chi tiết hiện thực và giao diện tạo mã được trình bày trong Chương 4 (Hình 4.6.1).

3.5.2 Lý do chọn tạo mã trực tiếp thay vì Runtime Engine

Việc lựa chọn phương pháp triển khai mô hình ML lên vi điều khiển có ảnh hưởng trực tiếp đến mức sử dụng tài nguyên, tính minh bạch và phạm vi thuật toán được hỗ trợ. Phần này phân tích ba cách tiếp cận chính và luận giải cho quyết định thiết kế trong bối cảnh HAR biên.

Các phương pháp triển khai mô hình trên vi điều khiển

Có ba cách tiếp cận chính để triển khai mô hình ML trên vi điều khiển, mỗi cách thể hiện đánh đổi khác nhau giữa tính tiện lợi, hiệu suất và kiểm soát:

Phương pháp 1: Runtime Engine (TensorFlow Lite Micro, ONNX Micro Runtime) Mô hình được chuyển đổi sang định dạng trung gian (.tflite, .onnx), sau đó được tải bởi một runtime engine chung trên thiết bị. Runtime đọc biểu đồ tính toán và thực thi từng toán tử (operator) theo thứ tự.

Ưu điểm: Hỗ trợ nhiều kiến trúc mô hình (CNN, LSTM, Transformer); hệ sinh thái lớn với cộng đồng hỗ trợ mạnh; các tối ưu hóa lượng tử hóa tích hợp (INT8, FP16).

Nhược điểm: Runtime chiếm 100–300KB flash bổ sung^[42]—đáng kể trên vi điều khiển có 256KB–1MB flash. Interpreter overhead thêm 15–40% thời gian suy luận so với mã gốc^[25]. Quan trọng nhất, runtime hoạt động như hộp đen: người dùng không thể kiểm tra hoặc xác minh quá trình trích xuất đặc trưng và tính toán bên trong.

Phương pháp 2: Trình biên dịch mô hình (Edge Impulse EON Compiler, Apache TVM, microTVM) Mô hình được biên dịch trước (ahead-of-time compilation) sang mã C tối ưu hóa, loại bỏ runtime nhưng vẫn hoạt động qua chuỗi công cụ đóng gói của nhà cung cấp.

Ưu điểm: Không có interpreter overhead; mã tối ưu hóa cho vi kiến trúc cụ thể; Edge Impulse EON Compiler giảm flash 35–55% so với TFLite Micro (theo số liệu nhà cung cấp^[14]).

Nhược điểm: Chuỗi công cụ đóng gói—mã đã tạo bởi EON Compiler khó kiểm toán và không thể sửa đổi. Apache TVM/microTVM yêu cầu kiến thức biên dịch cấp chuyên gia. Hầu hết các trình biên dịch mô hình bắt buộc mô hình đầu vào theo định dạng ONNX hoặc TFLite, loại trừ các mô hình scikit-learn cổ điển (RF, SVM) mà không qua bước chuyển đổi trung gian phức tạp.

Phương pháp 3: Tạo mã trực tiếp (Direct Code Generation) Phương pháp này trích xuất trực tiếp các tham số đã huấn luyện từ đối tượng scikit-learn/PyTorch—trọng số và bias của mạng nơ-ron, support vectors và hệ số kernel của SVM, cấu trúc cây nhị phân của Random Forest, trọng số convolution và batch normalization của CNN—rồi tạo mã nguồn (C/C++ hoặc MicroPython) triển khai thuật toán suy luận tương ứng.

Ưu điểm: Không phụ thuộc bên ngoài—mã tự chứa hoàn toàn; mức sử dụng bộ nhớ tối thiểu (chỉ chứa trọng số + logic suy luận); minh bạch hoàn toàn—mọi phép tính có thể kiểm tra và xác minh; hỗ trợ trực tiếp các thuật toán ML cổ điển mà không cần chuyển đổi định dạng; xuất được nhiều ngôn ngữ đích (C, C++, MicroPython).

Nhược điểm: Yêu cầu triển khai thủ công logic suy luận cho mỗi thuật toán; không hưởng các tối ưu hóa toán tử tự động cấp vi kiến trúc; khó mở rộng sang các kiến trúc phức tạp (CNN nhiều lớp, attention).

Phân tích so sánh định lượng

Bảng 3.5.2 tóm tắt so sánh ba phương pháp trên các tiêu chí quan trọng cho triển khai vi điều khiển bị giới hạn tài nguyên:

3.5. Tạo mã cho triển khai biên

Bảng 3.5.2: So sánh phương pháp triển khai mô hình ML trên vi điều khiển

Tiêu chí	Runtime	Compiler	Trực tiếp
Overhead flash	100–300KB	15–50KB	0KB
Overhead suy luận	15–40%	<5%	0%
Hỗ trợ RF/SVM	Hạn chế [†]	Không [‡]	Đầy đủ
Minh bạch	Thấp	Trung bình	Hoàn toàn
Cần ONNX/TFLite	Có	Có	Không
Phức tạp triển khai	Thấp	Cao	Trung bình
Hỗ trợ DNN sâu	Tốt	Tốt	Hạn chế

[†]TFLite Micro chỉ hỗ trợ NN/CNN; RF/SVM không thể chuyển đổi trực tiếp do onnx2tf không hỗ trợ ONNX-ML TreeEnsembleClassifier. Keras surrogate có thể sử dụng nhưng là xấp xỉ (80–90% tương đồng).

[‡]EON Compiler và TVM tập trung vào mạng nơ-ron; không hỗ trợ RF/SVM gốc.

Lập luận cho lựa chọn thiết kế

Quyết định sử dụng tạo mã trực tiếp được lập luận bởi bốn yếu tố:

(1) Mức sử dụng bộ nhớ tối thiểu Trên các vi điều khiển lớp wearable như Seede XIAO nRF52840 (1MB flash, 256KB RAM), mỗi KB đều quan trọng. TFLite Micro interpreter chiếm khoảng 150–250KB flash^[30;42]—tương đương 15–25% tổng flash khả dụng—chỉ riêng runtime, trước khi tính trọng số mô hình. Phương pháp trực tiếp loại bỏ hoàn toàn overhead này: một mô hình Neural Network chỉ chiếm 95KB (trọng số + logic suy luận + trích xuất đặc trưng), để lại >900KB cho firmware ứng dụng và giúp việc chuyển sang các board cùng lớp tài nguyên trở nên khả thi hơn.

(2) Minh bạch và khả năng xác minh Một yêu cầu quan trọng trong thiết kế hệ thống suy luận nhúng là *khả năng kiểm tra và xác minh mã đã tạo* (xem Phần 6.4). Khi phát hiện lỗi zero-padding và sai lệch công thức kurtosis, có thể truy vết trực tiếp đến mã C++ đã tạo, so sánh từng dòng với triển khai Python, và sửa chữa. Với runtime đóng gói (TFLite Micro) hoặc trình biên dịch đóng (EON Compiler), quá trình gỡ lỗi này sẽ không thể thực hiện—lỗi sẽ biểu hiện như “mô hình hoạt động kém trên thiết bị” mà không có cách xác định nguyên nhân gốc.

Khả năng xác minh này đặc biệt quan trọng cho ứng dụng y tế và an toàn, nơi các tiêu chuẩn quy định (FDA, IEC 62304) yêu cầu khả năng truy vết và kiểm toán đầy đủ cho mọi thành phần phần mềm liên quan đến quyết định lâm sàng.

(3) Hỗ trợ trực tiếp các thuật toán ML cổ điển Hệ sinh thái TFLite Micro và ONNX Runtime Micro được thiết kế chủ yếu cho mạng nơ-ron. Triển khai Random Forest hoặc SVM qua các nền tảng này yêu cầu chuyển đổi nhiều bước: scikit-learn → ONNX (`skl2onnx`) → TFLite (`onnx-tf`) → TFLite Micro. Mỗi bước chuyển đổi giới thiệu rủi ro sai lệch số và mất thông tin (ví dụ: ONNX không hỗ trợ chính xác tham số `class_weight` của scikit-learn). Phương pháp trực tiếp đọc các thuộc tính nội bộ của scikit-learn (`model.coefs_`, `model.support_vectors_`, `model.estimators_`) mà không qua định dạng trung gian, loại bỏ hoàn toàn chuỗi chuyển đổi dễ lỗi này.

(4) Các thuật toán cổ điển là lựa chọn phù hợp cho HAR biên Lựa chọn tạo mã trực tiếp liên kết chặt chẽ với lựa chọn thuật toán. Cho bài toán HAR 5–10 lớp với <1000 vectors đặc trưng và 63 đặc trưng bất biến hướng, các thuật toán cổ điển đạt hiệu suất cạnh tranh (97–99%) với chi phí tài nguyên thấp hơn đáng kể. Các mô hình này có cấu trúc suy luận đơn giản—nhân ma trận cho NN, duyệt cây cho RF, tính kernel cho SVM—có thể triển khai chính xác trong vài trăm dòng C++ mà không cần framework tính toán phức tạp. Điều này tạo thành một *sweet spot* giữa hiệu suất mô hình và tính khả thi triển khai cho các thiết bị đeo bị giới hạn tài nguyên.

Hạn chế và các phương pháp bổ sung

Tạo mã trực tiếp không phải giải pháp phổ quát. Nó không phù hợp cho:

- **Mạng sâu nhiều lớp:** LSTM, Transformer, hoặc CNN với hơn 4–5 lớp convolution yêu cầu các toán tử phức tạp (attention, bidirectional) mà triển khai thủ công dễ lỗi và khó bảo trì
- **Lượng tử hóa nâng cao:** INT8 inference với hiệu chỉnh phạm vi per-channel yêu cầu hạ tầng runtime mà TFLite Micro cung cấp sẵn
- **Mô hình thay đổi thường xuyên:** Nếu mô hình cần cập nhật OTA (over-the-air) mà không biên dịch lại firmware, runtime interpreter là lựa chọn tốt hơn

Đường dẫn TFLite Micro (Mục 3.5.3) tồn tại song song với tạo mã trực tiếp; người dùng chọn phương pháp phù hợp: tạo mã trực tiếp cho ML cổ điển và CNN nhỏ, TFLite Micro cho các mô hình cần lượng tử hóa INT8 đầy đủ.

3.5.3 Triển khai TFLite Micro

Đường dẫn triển khai TFLite Micro là phương pháp bổ sung phù hợp cho các kiến trúc mô hình neural network. Quá trình chuyển đổi tuân theo pipeline $\text{ONNX} \rightarrow \text{TensorFlow} \rightarrow \text{TFLite}$ với các hạn chế quan trọng về phạm vi hỗ trợ^[1].

Phạm vi hỗ trợ theo loại mô hình

Nghiên cứu thực nghiệm cho thấy khả năng chuyển đổi khác nhau đáng kể giữa các loại mô hình:

Neural Network và CNN: Chuyển đổi trực tiếp (hỗ trợ) Các mô hình neural network (MLP) và CNN từ PyTorch có thể chuyển đổi thành công qua pipeline:

1. PyTorch model $\rightarrow$ ONNX (`torch.onnx.export`)
2. ONNX $\rightarrow$ TensorFlow (`onnx-tf`)
3. TensorFlow $\rightarrow$ TFLite (`tf.lite.TFLiteConverter`)
4. TFLite $\rightarrow$ TFLite Micro (quantization INT8)

Random Forest và SVM: Hạn chế ONNX-ML (không hỗ trợ trực tiếp) Các mô hình Random Forest và SVM gặp rào cản chuyển đổi do hạn chế trong hệ sinh thái ONNX-ML:

- **Scikit-learn $\rightarrow$ ONNX:** `skl2onnx` tạo các toán tử ONNX-ML như `TreeEnsembleClassifier` và `SVMClassifier`
- **ONNX-ML $\rightarrow$ TensorFlow:** `onnx2tf` không hỗ trợ các toán tử ONNX-ML, chỉ hỗ trợ các toán tử ONNX chuẩn (operator domain "ai.onnx")
- **Kết quả:** Pipeline bị gián đoạn, không thể chuyển đổi RF/SVM sang TFLite

Giải pháp Keras Surrogate cho RF/SVM

Để khắc phục hạn chế ONNX-ML, *phương pháp Keras surrogate* có thể được áp dụng:

1. **Sinh dữ liệu huấn luyện surrogate:** Sử dụng mô hình RF/SVM gốc để tạo soft predictions trên tập dữ liệu đại diện

2. **Huấn luyện Keras MLP:** Mạng neural đa lớp học ánh xạ từ features $\rightarrow$ soft predictions của RF/SVM
3. **Chuyển đổi MLP $\rightarrow$ TFLite:** Keras MLP có thể chuyển đổi thành công qua pipeline tiêu chuẩn

Hạn chế Surrogate: Keras surrogate là *xấp xỉ*, không phải triển khai chính xác. Độ thỏa thuận kỳ vọng 80-90% giữa mô hình gốc và surrogate. Cho độ chính xác tối đa, *tạo mã trực tiếp C++ vẫn là phương pháp được khuyến nghị cho RF/SVM.*

Khuyến nghị sử dụng

- **NN/CNN:** Sử dụng TFLite Micro để tận dụng runtime tối ưu hóa và hỗ trợ quantization INT8
- **RF/SVM:** Ưu tiên tạo mã trực tiếp C++ (chính xác, nhanh, nhỏ gọn). TFLite surrogate chỉ khi yêu cầu tích hợp runtime chuẩn hóa

3.5.4 Cấu trúc mã đã tạo

Đối với mỗi mô hình đã huấn luyện, trình tạo sản xuất một gói triển khai gồm các tệp tham số, tệp cài đặt và tệp ví dụ tích hợp. Cụ thể:

1. **Tệp Header (.h):** Chứa các tham số mô hình (ví dụ: vectors hỗ trợ, cấu trúc cây, ma trận trọng số), khai báo hàm, các macro số lượng đặc trưng/lớp
2. **Tệp Triển khai (.c/.cpp/.py):** Triển khai trích xuất đặc trưng, logic dự đoán (đặc thù cho thuật toán) và các hàm tiện ích, với phần mở rộng phụ thuộc nền tảng đích
3. **Tệp Ví dụ Tích hợp:** Có thể là Arduino sketch (.ino), chương trình C/C++ ví dụ hoặc script MicroPython, minh họa khởi tạo cảm biến, quản lý bộ đệm của sổ trượt, vòng lặp suy luận thời gian thực và đầu ra dự đoán

Xác minh Công thức Thống kê: Mã C++ đã tạo cho trích xuất đặc trưng sử dụng các công thức thống kê được xác minh khớp chính xác với triển khai Python (pandas/NumPy). Cụ thể, các đặc trưng kurtosis và skewness sử dụng *độ lệch chuẩn tổng thể* (population standard deviation, chia n) cho chuẩn hóa z-scores, thay vì độ lệch chuẩn mẫu (chia $n - 1$). Sự khác biệt này, mặc dù nhỏ ($\sim 1,3\%$ cho $n = 150$), là hệ thống và tích lũy qua nhiều đặc trưng, có thể ảnh hưởng đến độ chính xác triển khai nếu không được xử lý.

3.5.5 Lượng tử hóa mô hình cho triển khai biên

Lượng tử hóa (model quantization) là kỹ thuật nén mô hình then chốt trong triển khai TinyML, chuyển đổi các tham số mô hình từ biểu diễn dấu phẩy động 32-bit (float32) sang số nguyên 8-bit (int8) hoặc 16-bit (int16). Trên vi điều khiển như Cortex-M4F với 256 KB RAM và 1 MB flash, lượng tử hóa có thể quyết định liệu một mô hình có thể triển khai được hay không: chuyển đổi từ float32 sang int8 giảm kích thước trọng số $4\times$, giảm băng thông bộ nhớ và tăng tốc suy luận $2\text{--}3\times$ nhờ các tập lệnh SIMD integer của ARM^[25].

Các loại lượng tử hóa

Lượng tử hóa sau huấn luyện (Post-Training Quantization, PTQ) PTQ áp dụng sau khi huấn luyện hoàn tất mà không cần sửa đổi quy trình huấn luyện. Mỗi tham số float32 x được ánh xạ sang giá trị lượng tử hóa x_q theo:

$$x_q = \text{clip}\left(\text{round}\left(\frac{x}{s}\right) + z, 0, 255\right) \quad (3.5.1)$$

trong đó $s = \frac{x_{\max} - x_{\min}}{255}$ là hệ số tỷ lệ và $z \in [0, 255]$ là zero-point giữ nguyên giá trị số không. Có hai biến thể PTQ: *lượng tử hóa trọng số* (weight-only), trong đó chỉ trọng số mô hình được chuyển đổi còn phép tính suy luận vẫn dùng float32; và *lượng tử hóa tĩnh* (static PTQ), trong đó cả trọng số lẫn giá trị kích hoạt đều được lượng tử hóa, đòi hỏi một tập hiệu chỉnh nhỏ (100–500 mẫu đại diện) để đo phân phối kích hoạt trong thực tế. PTQ thường gây suy giảm độ chính xác 0,5–2% cho các mô hình HAR^[42].

Lượng tử hóa trong huấn luyện (Quantization-Aware Training, QAT) QAT chèn các toán tử *fake quantization* (lượng tử hóa giả) vào đồ thị tính toán trong quá trình huấn luyện, mô phỏng sai số lượng tử hóa để mô hình học cách bù đắp. QAT thường giảm suy giảm độ chính xác xuống dưới 0,3% nhưng đòi hỏi sửa đổi pipeline huấn luyện và thực tế chỉ áp dụng cho mạng nơ-ron—Random Forest và SVM có cấu trúc rời rạc (ngưỡng quyết định, vector hỗ trợ) không tương thích với kỹ thuật đạo hàm giả (straight-through estimator) mà QAT dựa vào.

Chiến lược lượng tử hóa theo phương pháp tạo mã

Tùy theo phương pháp tạo mã được chọn, lượng tử hóa được áp dụng theo hai đường dẫn khác nhau:

Lượng tử hóa trọng số trong tạo mã trực tiếp Trong đường dẫn tạo mã trực tiếp, lượng tử hóa được thực hiện ở cấp độ biểu diễn tham số trong mã C++ đã tạo. Chế độ tối ưu hóa kiểm soát số chữ số thập phân khi nhúng hằng số float32 vào mã nguồn—tương đương lượng tử hóa trọng số mức thấp:

- **Accuracy:** float32 đầy đủ (6 chữ số thập phân), không xấp xỉ
- **Balanced:** 3 chữ số thập phân, tiết kiệm $\approx 30\%$ kích thước mã nguồn
- **Speed/Power:** 2 chữ số thập phân, tiết kiệm $\approx 45\%$ kích thước mã nguồn

Phép suy luận vẫn thực hiện bằng số học float32 trên FPU của Cortex-M4F—đây là lượng tử hóa *lưu trữ*, không phải lượng tử hóa *tính toán* đầy đủ. Đối với mạng nơ-ron, chế độ *Power* có thể tùy chọn biểu diễn ma trận trọng số ở int16, thực hiện nhân tích lũy integer và chỉ dequantize một lần trước hàm kích hoạt:

$$y_j = f\left(s \cdot \sum_i w_{ij}^{\text{int16}} \cdot x_i^{\text{int16}} + b_j\right) \quad (3.5.2)$$

trong đó s là tích của hai scale tương ứng (trọng số và đầu vào), $f(\cdot)$ là hàm kích hoạt ReLU hoặc softmax.

Lượng tử hóa INT8 đầy đủ qua đường dẫn TFLite Micro Đường dẫn TFLite Micro (áp dụng cho NN và CNN) hỗ trợ static PTQ với INT8 đầy đủ—cả trọng số lẫn kích hoạt. Quy trình thực hiện:

1. Chuyển đổi mô hình Keras/PyTorch sang TFLite (**TFLiteConverter**)
2. Một tập hiệu chỉnh gồm 100–500 vectors đặc trưng đại diện được cung cấp để TFLiteConverter đo phân phối kích hoạt từng lớp
3. Converter tự động tính s và z cho mỗi tensor theo Phương trình 3.5.1, tối ưu cho phạm vi kích hoạt thực tế
4. Mô hình INT8 kết quả kết hợp với ARM CMSIS-NN^[5] để khai thác tập lệnh SIMD của Cortex-M4, thực thi 4 phép nhân int8 song song mỗi chu kỳ—tăng tốc $2\text{--}3\times$ so với mô hình float32 tương đương

Bảng 3.5.3 so sánh hai đường dẫn lượng tử hóa theo các tiêu chí triển khai:

3.5. Tạo mã cho triển khai biên

Bảng 3.5.3: So sánh chiến lược lượng tử hóa theo phương pháp tạo mã

Tiêu chí	Tạo mã trực tiếp (Weight PTQ)	TFLite Micro (Static INT8 PTQ)
Áp dụng cho	RF, SVM, NN, CNN	NN, CNN (không hỗ trợ RF/SVM)
Giảm kích thước	30–45% (so với float32 đầy đủ)	4× (so với float32)
Tăng tốc suy luận	Không đáng kể (vẫn dùng FPU float32)	2–3× (SIMD int8)
Suy giảm độ chính xác	<0,5%	0,5–2%
Yêu cầu tập hiệu chỉnh	Không	100–500 mẫu
Tính minh bạch	Hoàn toàn (mã có thể kiểm tra)	Hạn chế (runtime hộp đen)

Đặc thù lượng tử hóa cho đặc trưng HAR miền thời gian

Bài toán HAR với đặc trưng thống kê miền thời gian có các đặc tính thuận lợi cho lượng tử hóa so với thị giác máy tính hay xử lý ngôn ngữ tự nhiên:

- **Phạm vi đặc trưng có giới hạn vật lý:** Gia tốc IMU bị giới hạn ở $\pm 20 \text{ m/s}^2$, vận tốc góc ở $\pm 2000^\circ/\text{s}$. Do đó, các đặc trưng thống kê (trung bình, RMS, độ lệch chuẩn) có phạm vi xác định, giúp xác định scale lượng tử hóa chính xác mà không cần tập hiệu chỉnh lớn.
- **Bền vững với nhiễu lượng tử hóa:** Random Forest quyết định theo ngưỡng nhị phân tại mỗi nút cây; sai số lượng tử hóa nhỏ hiếm khi đủ để đổi chiều quyết định phân nhánh, đặc biệt khi đặc trưng nằm xa ngưỡng. SVM nhạy hơn ở vùng ranh giới quyết định nhưng vẫn ổn định với int16.
- **Tham số scaler phải đồng bộ độ chính xác:** StandardScaler (trung bình μ và độ lệch chuẩn σ) được áp dụng trước mô hình. Để duy trì parity huấn luyện-triển khai, tham số scaler cần được nhúng với cùng độ chính xác với tham số mô hình—lượng tử hóa mô hình nhưng giữ scaler ở float32 đầy đủ sẽ gây ra sự không nhất quán hệ thống ảnh hưởng đến toàn bộ vector đặc trưng đầu vào.
- **Kurtosis và Skewness nhạy hơn:** Các đặc trưng hình dạng phân phối có phạm vi giá trị rộng hơn và nhạy với lượng tử hóa mạnh hơn các đặc trưng năng lượng

(RMS, Energy). Do đó, chế độ *Balanced* khuyến nghị sử dụng 3 chữ số thập phân để bảo toàn các đặc trưng này.

3.5.6 Kiến trúc tạo mã đa kiến trúc

Hai loại pipeline mô hình tồn tại với yêu cầu khác biệt:

Mô hình dựa trên đặc trưng (RF, SVM, NN) Nhận vector đặc trưng thống kê (33–90 chiều), áp dụng StandardScaler trên features đã trích xuất, sinh mã trích xuất đặc trưng đầy đủ trong C++.

Mô hình CNN end-to-end Nhận trực tiếp cửa sổ cảm biến thô ($150 \times 6 = 900$ đầu vào), chỉ áp dụng lọc tín hiệu (không trích xuất đặc trưng), StandardScaler trên dữ liệu thô.

Sự khác biệt này đòi hỏi kiến trúc tạo mã nhận biết loại mô hình (architecture-aware): metadata, logic xác minh và cấu trúc mã sinh ra đều phụ thuộc vào loại pipeline. Generator tự động phân biệt để sinh đúng cấu trúc cho từng loại.

3.5.7 Các chiến lược tối ưu hóa

Bốn chế độ tối ưu hóa được cấu hình khi tạo mã: *Accuracy* (float32 đầy đủ, không đơn giản hóa); *Speed* (giảm phức tạp mô hình, tính toán cache-friendly, mở rộng vòng lặp); *Power* (số học điểm cố định int16, 2 chữ số thập phân, tận dụng ARM CMSIS-DSP); *Balanced* (3 chữ số thập phân, độ chính xác hỗn hợp cho các tính toán khác nhau). Chi tiết so sánh tại Bảng 4.6.1, Chương 4.

3.5.8 Điều chỉnh đặc thù nền tảng

Mã sinh ra được điều chỉnh theo họ thiết bị đích: Arduino-compatible (sinh .ino/.cpp/.h với Serial và Wire); ARM Cortex-M native (mã C độc lập, bộ nhớ tĩnh, tích hợp CMSIS); SDK/RTOS (include và entry-point cho ESP-IDF, Zephyr); MicroPython (module Python độc lập duy trì tương đồng công thức với C++). Nhờ phân lớp này, cùng một mô hình có thể xuất cho nhiều đích mà không cần huấn luyện lại.

Chi tiết triển khai phần mềm và phần cứng được trình bày trong Chương 4. Thiết lập thực nghiệm, tập dữ liệu và số liệu đánh giá được mô tả trong Chương 5.

Chương 4

Thiết kế và hiện thực

Chương này trình bày kiến trúc hệ thống, quyết định thiết kế và chi tiết hiện thực của framework triển khai nhận dạng hoạt động trên thiết bị biên. Các thuật toán và cơ sở lý thuyết đã được trình bày trong Chương 3; chương này tập trung vào **kỹ thuật phần mềm, mô-đun hóa, và quy trình triển khai** của hệ thống thực tế. Framework được xây dựng dựa trên nguyên tắc tách biệt vai trò (separation of concerns): mỗi module chịu trách nhiệm một giai đoạn rõ ràng trong pipeline từ dữ liệu thô đến mã triển khai, đảm bảo tính mở rộng và khả năng bảo trì.

4.1 Kiến trúc hệ thống

4.1.1 Nền tảng công nghệ và môi trường phát triển

Framework được triển khai dưới dạng ứng dụng web một trang sử dụng Python 3.10 làm ngôn ngữ chính, với Dash 2.14 (dựa trên Flask và Plotly) cho giao diện tương tác, scikit-learn 1.3 cho các thuật toán ML cổ điển (Random Forest, SVM, MLP), và PyTorch 2.x cho huấn luyện mô hình học sâu (MLP, CNN). Xử lý dữ liệu dựa trên NumPy và Pandas; trực quan hóa tương tác sử dụng Plotly với hỗ trợ kéo thả; và Jinja2 được dùng làm template engine để sinh mã C/C++ từ các mẫu tham số hóa.

Nền tảng chuẩn triển khai là Seeed XIAO nRF52840 Sense (ARM Cortex-M4F @ 64MHz, 256KB RAM, 1MB Flash, IMU LSM6DS3 tích hợp), được chọn vì: tích hợp đầy đủ IMU 6 trục, hiệu năng đại diện cho phân khúc MCU trung bình, hỗ trợ Arduino và PlatformIO, và giá thành thấp (~\$10 USD). Biên dịch chéo sử dụng GCC ARM Embedded Toolchain qua Arduino IDE 2.3 hoặc PlatformIO.

4.1.2 Kiến trúc ứng dụng

Framework áp dụng kiến trúc phân lớp rõ ràng với nguyên tắc *separation of concerns*. Hệ thống được tổ chức thành bốn lớp chính: (1) **lớp giao diện** chứa định nghĩa layout Dash cho mỗi tab; (2) **lớp callback** xử lý tương tác người dùng và điều phối luồng dữ liệu; (3) **lớp tiện ích** chứa các thuật toán xử lý tín hiệu, trích xuất đặc trưng và huấn luyện mô hình; (4) **lớp tạo mã** chứa hệ thống generator cho từng mô hình và nền tảng đích. Trạng thái liên phiên được lưu xuống hệ thống file, tách biệt với trạng thái tạm thời trong session.

Quy trình đăng ký callback Dash sử dụng mô hình *callback pattern* để đồng bộ giao diện và trạng thái. Mỗi module callback định nghĩa một hàm `register_callbacks(app)`, bên trong đó tập trung toàn bộ logic xử lý sự kiện cho tab tương ứng. Entry point ứng dụng gọi tuần tự các hàm đăng ký này sau khi khởi tạo layout.

Cách tổ chức này đảm bảo:

1. **Tính module**: mỗi module callback độc lập, dễ bảo trì và kiểm thử riêng biệt.
2. **Tránh circular imports**: callbacks không import lẫn nhau, chỉ import từ lớp tiện ích và lớp tạo mã.
3. **Testability**: mỗi callback có thể kiểm thử riêng bằng cách mock Dash context.

4.1.3 Quản lý trạng thái

Framework sử dụng kết hợp hai cơ chế lưu trạng thái:

Hệ thống file (`persistent_data/`) Dữ liệu trung gian giữa các giai đoạn được lưu dưới dạng file CSV, pickle (scikit-learn models) và joblib (model bundles):

- **datasets/**: dữ liệu CSV gốc theo activity class.
- **windows/**: sliding windows đã sinh (pickle format).
- **training/**: feature matrices (CSV) và metrics.
- **models/**: model serialization (joblib) chứa model object, scaler, label encoder, danh sách features và các chỉ số hiệu suất.
- **generated/**: mã nguồn triển khai theo cấu trúc `{model_type}_models/{model_name}/`.

In-memory state (`dcc.Store`) Dash component `dcc.Store` lưu trạng thái session tạm thời (VD: metadata cột cảm biến, tần số lấy mẫu, danh sách segments đã chọn). State này được truyền giữa các callbacks trong cùng một session nhưng không persist sau khi reload trang.

Luồng dữ liệu Dữ liệu chảy tuần tự qua sáu tab: từ CSV thô đến dataset đăng ký, qua tiền xử lý và sliding windows, đến feature matrix, model joblib, mã C++ sinh ra, rồi kiểm tra trực tiếp trên thiết bị. Thiết kế file-based này cho phép người dùng quay lại bất kỳ bước nào để điều chỉnh mà không mất kết quả trung gian.

4.2 Module quản lý dữ liệu

Module này (Tab 1 trong giao diện) chịu trách nhiệm tải dữ liệu cảm biến và kiểm tra trực quan chất lượng tín hiệu.

4.2.1 Giao diện tải dữ liệu

Người dùng tải lên nhiều file CSV, mỗi file chứa dữ liệu cho một activity class. Framework tự động phát hiện cấu trúc cột bằng cách khớp với danh sách quy ước đặt tên phổ biến (như `aX/AccX` cho gia tốc kế, `gX/GyroX` cho con quay hồi chuyển); nếu không khớp, fallback sang khớp substring trong tên cột. Tần số lấy mẫu được tự động tính từ cột timestamp (nếu có) hoặc người dùng nhập thủ công; giá trị này được nhúng vào mã triển khai dưới dạng hằng số.

4.2. Module quản lý dữ liệu

Select Dataset:

running.csv

Delete Selected

Dataset Properties

Current Label:

running

Assign New Label:

Enter activity label (e.g., walking, running)

Save

Sampling Configuration

Sampling Rate (Hz):

100

Save

Common rates: 50Hz (basic), 100Hz (standard), 200Hz (high-precision)

Dataset Information

Property	Value
Label	running
Sampling Rate	100 Hz
File Path	running.csv
Has Cleaned Data	Yes
File Size	219.8 KB
Rows	4,132
Columns	7
Duration (est.)	41.3 seconds
Memory Usage	226.1 KB
Sensor Columns	aX, aY, aZ, gX, gY, gZ

Clear All Data

Export Metadata

Data Preview

Interactive visualization of the selected dataset

Filtered Preview of running.csv

Hình 4.2.1: Giao diện tải lên và trực quan hóa dữ liệu cảm biến. Framework tự động phát hiện cấu trúc cột IMU và hiển thị biểu đồ tín hiệu đồng bộ trên tất cả các trục gia tốc kế và con quay hồi chuyển, cho phép kiểm tra trực quan chất lượng dữ liệu trước khi tiến xử lý.

41

4.3 Module tiền xử lý tương tác

Module này (Tab 2) cung cấp giao diện kéo thả (drag-and-drop) để chọn các đoạn hoạt động đại diện trên biểu đồ tín hiệu, sau đó tự động sinh sliding windows từ các đoạn đã chọn.

4.3.1 Giao diện lựa chọn đoạn tương tác

Dash Plotly hỗ trợ *editable shapes*: người dùng vẽ hình chữ nhật trực tiếp trên biểu đồ để đánh dấu vùng quan tâm. Khi người dùng vẽ hoặc chỉnh sửa một hình chữ nhật, sự kiện layout được kích hoạt và framework đọc tọa độ x-axis để xác định phạm vi mẫu của đoạn vừa chọn.

Framework kiểm tra ràng buộc cơ bản:

- **Chiều dài tối thiểu:** mỗi segment phải có ít nhất `window_size` mẫu để sinh được ít nhất 1 cửa sổ.

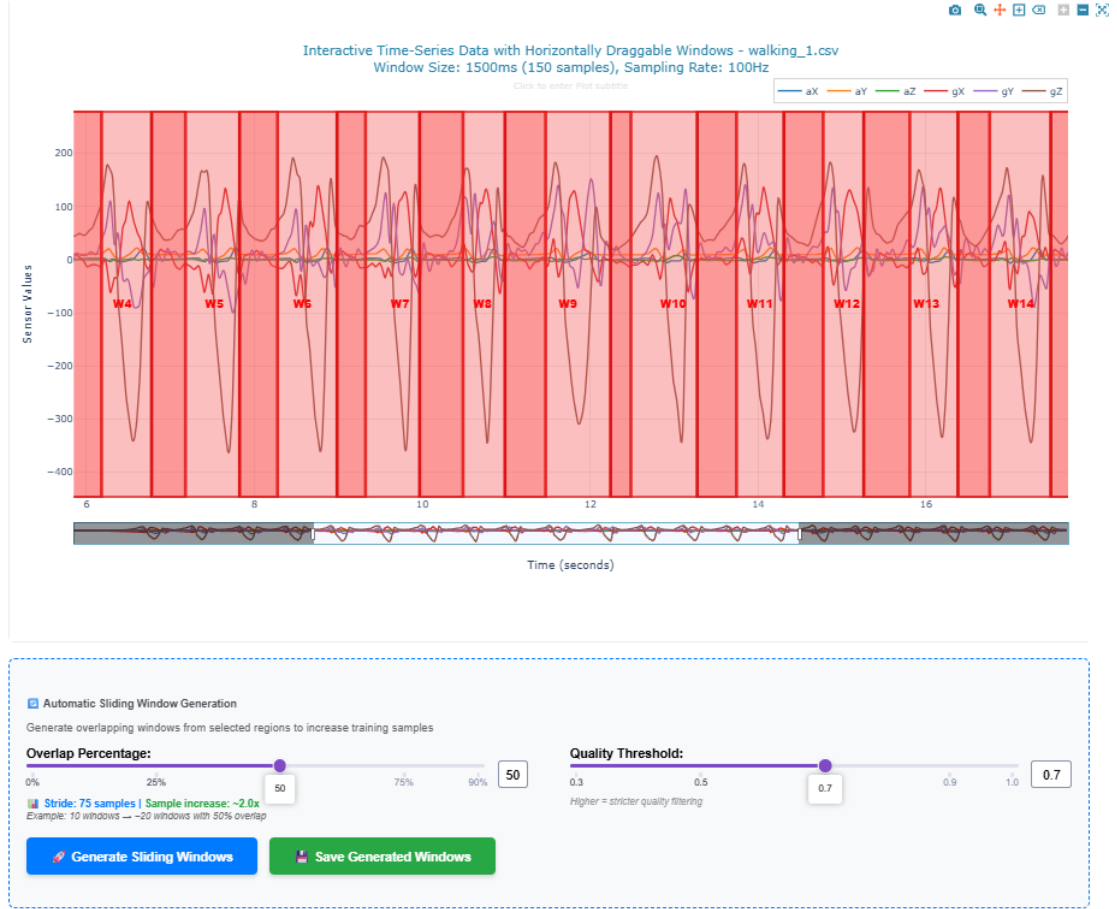
Các segment có thể chồng chéo nhau — điều này cho phép người dùng thu thập mẫu từ nhiều vùng khác nhau trong cùng một đoạn tín hiệu.

Hình 4.3.1 minh họa giao diện này.

4.3. Module tiền xử lý tương tác

Interactive Window Selection

Drag windows to optimal positions and extract segments for model training



Hình 4.3.1: Giao diện cửa sổ kéo thả tương tác. Người dùng kéo thả trực tiếp trên biểu đồ tín hiệu để xác định các đoạn hoạt động đại diện. Cửa sổ trượt tự động sinh các mẫu huấn luyện từ các đoạn đã chọn, kết hợp giữa kiểm soát thủ công và tự động hóa.

4.3.2 Sinh cửa sổ trượt

Từ mỗi segment, framework áp dụng thuật toán sliding window với tham số mặc định: `window_size` = 150 mẫu (1,5s ở 100Hz), `stride` = 75 mẫu (overlap 50%). Overlap 50% tăng số lượng mẫu huấn luyện đồng thời giữ sự đa dạng; stride càng nhỏ thì số windows tạo ra càng nhiều nhưng tăng nguy cơ overfitting do tương quan giữa các windows liền kề. Công thức và lý thuyết sliding window đã được trình bày trong Chương 3.

4.3.3 Trực quan hóa lọc tín hiệu

Framework hỗ trợ ba phương pháp lọc tín hiệu (chi tiết thuật toán đã trình bày trong Chương 3):

1. **Butterworth lowpass filter** (filtfilt for zero-phase)
2. **Savitzky-Golay filter** (polynomial smoothing)
3. **Kalman filter** (causal, exact training-deployment parity)

Giao diện hiển thị overlay tín hiệu raw (màu xanh) và filtered (màu đỏ) để người dùng đánh giá hiệu quả lọc. Người dùng có thể điều chỉnh tham số (cutoff frequency, filter order, window length) và quan sát thay đổi real-time.

4.4 Module trích xuất đặc trưng

Module này (Tab 3) chịu trách nhiệm chuyển đổi cửa sổ tín hiệu thô sang feature vectors và chuẩn bị train/val/test splits.

4.4.1 Chế độ trích xuất đặc trưng

Framework hỗ trợ 6 chế độ trích xuất đặc trưng (chi tiết tại Bảng 3.3.1, Chương 3), từ đặc trưng bất biến hướng trên magnitude vectors (33–63 chiều, phù hợp triển khai nhúng) đến đặc trưng per-axis miền thời gian và tần số (66–156 chiều), cùng chế độ raw cho đầu vào CNN. Giao diện cho phép người dùng chọn chế độ và xem trước số lượng đặc trưng trước khi trích xuất.

Cơ sở chọn đặc trưng bất biến hướng Chế độ `orientation_invariant` chỉ sử dụng các đặc trưng tính trên magnitude vectors ($\sqrt{x^2 + y^2 + z^2}$) thay vì per-axis features. Điều này đảm bảo mô hình không bị ảnh hưởng bởi định hướng lắp đặt cảm biến: đeo thiết bị ngang hoặc dọc, xoay 90° vẫn cho cùng một magnitude. Tuy nhiên, trade-off là mất thông tin về hướng chuyển động (VD: leo cầu thang khó phân biệt với xuống cầu thang chỉ dựa trên magnitude).

4.4.2 Tích hợp tăng cường dữ liệu

Module tích hợp năm phương pháp tăng cường dữ liệu IMU đã trình bày trong Mục 3.3.4 (jitter, rotation, scaling, time warping, permutation) với chiến lược nhận biết lớp: hoạt động tĩnh chỉ nhận micro-jitter, hoạt động động nhận tất cả biến đổi.

4.4. Module trích xuất đặc trưng

Giao diện cho phép kích hoạt/tắt từng phương pháp riêng biệt, điều chỉnh hệ số tăng cường ($1\times-3\times$), và xem phân phối lớp trước/sau tăng cường.

Hình 4.4.1 minh họa giao diện này.

Step 2: Configure Feature Engineering

Settings will be applied uniformly across all selected datasets

Feature Extraction Method:
Orientation-Invariant Time-Domain ONLY - 41 features

Normalization Method (applied during training):
Standard Scaler (Z-score)

Note: Orientation-robust DFT (63 features) is fully deployable — code generators emit a lightweight sin/cos DFT. Only per-axis frequency features (66 / 156) cannot be deployed.

41 features
Orientation-robust magnitudes: 15 stats $\times$ 2 mag + 3 acc-jerk + 3 gyro-jerk + 5 scalar extras (SMA, tilt, autocorr, peak count). RECOMMENDED for deployment — fully deployable to all targets (C / C++ / MicroPython).

Window Configuration
Target Window Size (ms): 1500
Sampling Rate (Hz): 100

Windows smaller than this will be zero-padded (1500ms = 150 samples @ 1000Hz)

Used to calculate time duration from samples

Step 3: Data Augmentation (Optional)

Generate synthetic training windows to improve model robustness. Augmentation is applied to raw sensor signals before feature extraction, so new features are computed naturally from augmented data.

☐ Enable Data Augmentation

Step 4: Configure Dataset Split

Define how to split the combined dataset into train/validation/test sets

Split Mode:
☒ Auto (ratio-based random split) ☐ Manual (assign each window by hand)

Training Set Ratio: 0.7
Validation Set Ratio: 0.15
Test Set Ratio: 15%

Random Seed (for reproducibility): 42

Same seed = reproducible splits across runs

Hình 4.4.1: Giao diện trích xuất đặc trưng. Người dùng chọn loại đặc trưng (bất biến hướng, miền thời gian, miền tần số), cấu hình tăng cường dữ liệu nhận biết lớp, và phân chia dữ liệu huấn luyện/xác thực/kiểm tra với lấy mẫu phân tầng.

4.4.3 Phân chia train/validation/test

Framework cung cấp hai chế độ phân chia dữ liệu để phù hợp với nhiều kịch bản thực tế:

Phân chia tự động theo tỉ lệ Người dùng chọn tỉ lệ train/validation/test bằng thanh trượt trực tiếp trên giao diện. Framework sử dụng **stratified sampling** để đảm bảo tỉ lệ class đồng đều giữa các tập. Kỹ thuật này quan trọng khi dữ liệu mất cân bằng: ví dụ nếu 70% mẫu thuộc class “đứng yên”, thì cả train và test đều có 70%

mẫu đó. Tỷ lệ mặc định là train 60% / validation 20% / test 20%, nhưng người dùng có thể điều chỉnh tùy nhu cầu.

Phân chia thủ công theo dataset Đối với dữ liệu thu thập từ nhiều đối tượng hoặc nhiều phiên, người dùng có thể chỉ định trực tiếp file CSV nào thuộc tập train và file nào thuộc tập test. Chức năng này cho phép đánh giá *cross-subject generalization*: huấn luyện trên dữ liệu của một nhóm người dùng và kiểm thử trên nhóm hoàn toàn khác, tránh data leakage giữa các đối tượng.

Validation set được dùng cho hyperparameter tuning (GridSearchCV sử dụng cross-validation trên train+val), test set giữ nguyên không động đến cho đến khi đánh giá cuối cùng.

4.5 Module huấn luyện mô hình

Module này (Tab 4) triển khai training pipeline với hyperparameter optimization và real-time monitoring.

4.5.1 Lựa chọn thuật toán

Framework hỗ trợ 6 thuật toán phân loại:

Bảng 4.5.1: Thuật toán học máy được hỗ trợ

Thuật toán	Thư viện	Input	Triển khai
Random Forest	scikit-learn	Features	C++ native
SVM (RBF kernel)	scikit-learn	Features	C++ native
Neural Network (MLP)	scikit-learn	Features	C++ native
PyTorch MLP	PyTorch	Features	C++/TFLite
PyTorch 1D-CNN	PyTorch	Raw signals	C++/TFLite
PyTorch 2D-CNN	PyTorch	Raw signals (2D)	C++/TFLite

Kiến trúc PyTorch models

- **PyTorch MLP**: 2 hidden layers với ReLU activation. Kích thước layer điều chỉnh theo số features: `[n_features → 128 → 64 → n_classes]`.
- **PyTorch 1D-CNN**: convolutional layers trên time dimension, kernel size = 5, 2 conv layers + pooling + fully connected. Input shape: `[batch, 6 channels, 150 timesteps]`.

- **PyTorch 2D-CNN (HARCNN2D)**: sử dụng Conv2D với kernel $3 \times n_channels$ để học correlation giữa các sensor axes. Input shape: [batch, 1, 150 timesteps, 6 channels] (reshape từ 1D signal).

Kiến trúc 2D-CNN đặc biệt hữu ích khi cần học mối quan hệ không tuyến tính giữa các trục cảm biến (VD: correlation giữa aX và gY trong động tác xoay).

4.5.2 Tối ưu hóa siêu tham số

Framework tích hợp GridSearchCV từ scikit-learn với param grid mặc định cho từng thuật toán. Random Forest tìm kiếm trên các giá trị `n_estimators` (50, 100, 200), `max_depth` (10, 20, None) và `min_samples_split`; SVM tìm kiếm trên tham số `C` và `gamma`; Neural Network tìm kiếm trên kích thước hidden layers, hệ số L2 regularization và learning rate.

GridSearchCV sử dụng 5-fold cross-validation trên train+validation set, chọn tổ hợp params có accuracy cao nhất, sau đó retrain trên toàn bộ train+val với params tối ưu.

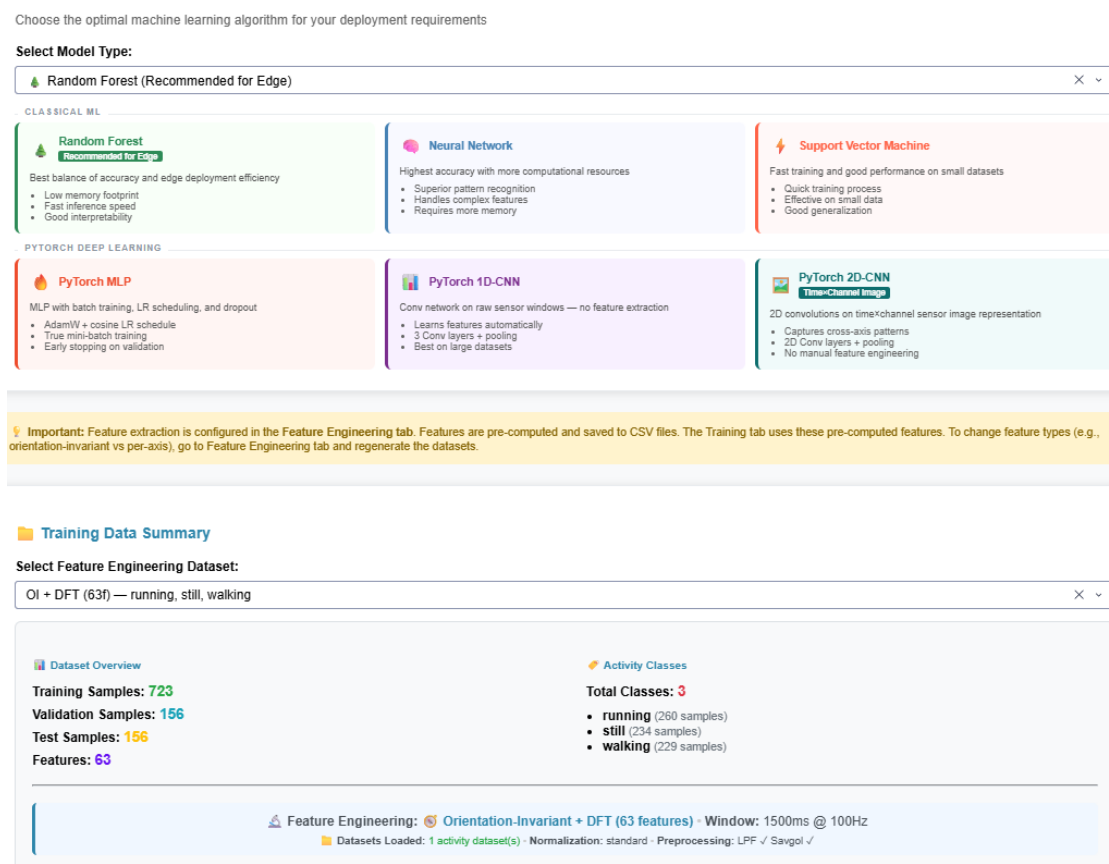
4.5.3 Theo dõi tiến trình real-time

Callback training sử dụng `dcc.Interval` component để cập nhật UI mỗi 2 giây trong quá trình huấn luyện. Thông tin hiển thị:

- **Training progress**: phần trăm hoàn thành, thời gian đã trôi qua.
- **Current hyperparameters**: params đang thử nghiệm.
- **Best score so far**: accuracy cao nhất trong các trials đã hoàn thành.
- **Confusion matrix**: ma trận nhầm lẫn trên validation set, cập nhật sau mỗi trial.

Hình 4.5.1 minh họa giao diện này.

4.5. Module huấn luyện mô hình



Hình 4.5.1: Giao diện cấu hình huấn luyện mô hình. Người dùng chọn thuật toán, cấu hình tối ưu hóa siêu tham số với GridSearchCV, chiến lược cân bằng lớp, và theo dõi tiến trình huấn luyện thời gian thực.

4.5.4 Lưu trữ mô hình

Sau khi huấn luyện hoàn tất, framework lưu trữ toàn bộ pipeline artifacts vào một file `.joblib`. Gói mô hình bao gồm: đối tượng model đã huấn luyện, bộ chuẩn hóa `StandardScaler`, ánh xạ nhãn (label encoder), danh sách tên features (để đảm bảo thứ tự đúng khi suy diễn), kiểu mô hình, siêu tham số tốt nhất, và các chỉ số hiệu suất (accuracy, F1, confusion matrix, classification report).

Cấu trúc này là chuẩn giao tiếp giữa module huấn luyện và module tạo mã. Module code generation đọc gói mô hình để trích xuất parameters (VD: cây quyết định của Random Forest, trọng số của Neural Network) và sinh mã triển khai tương ứng.

4.6 Module tạo mã triển khai

Module này (Tab 5) chịu trách nhiệm chuyển đổi trained model thành mã nguồn embedded C/C++ hoặc các định dạng khác (MicroPython, TFLite). Đây là thành phần phức tạp nhất của framework, vì phải đảm bảo **training-deployment parity**: mã được sinh ra phải tính toán đúng y hệt như Python pipeline.

4.6.1 Kiến trúc Code Generator: Factory Pattern

Framework sử dụng *Factory Pattern* để định tuyến từ (`model_type`, `target_platform`, `deployment_backend`) sang class generator phù hợp. Khi nhận model bundle, factory kiểm tra trường `model_type`: nếu deployment backend là TFLite Micro thì tạo `TFLiteCodeGenerator`; nếu không, sẽ tạo generator tương ứng với thuật toán (Random Forest, Neural Network, CNN, SVM). Tham số platform được truyền vào để generator điều chỉnh cú pháp và API cho từng nền tảng đích.

4.6.2 Hệ thống phân cấp Generator

Kiến trúc block pattern Module tạo mã được tổ chức theo mô hình block: một coordinator trung tâm nhận vào gói mô hình và điều phối các block con. Block trích xuất đặc trưng phụ trách sinh mã tính toán đặc trưng từ dữ liệu cảm biến thô; block phân loại và các lớp con của nó phụ trách sinh mã suy luận đặc thù cho từng thuật toán.

Coordinator điều phối hai nhóm block chính:

1. **Block trích xuất đặc trưng**: sinh mã tính features từ dữ liệu cảm biến thô (nếu mô hình yêu cầu). Bao gồm:
 - Tính magnitude: $\text{acc_mag} = \sqrt{aX^2 + aY^2 + aZ^2}$
 - Đặc trưng thống kê: mean, std, min, max, median, quartiles, skewness, kurtosis, RMS, energy
 - Đặc trưng jerk: đạo hàm số của `acc_mag`
 - Bộ lọc tiền xử lý: IIR filtfilt, Savitzky-Golay, Kalman (nếu được cấu hình)
2. **Block phân loại**: sinh mã suy luận cho thuật toán cụ thể. Mỗi loại mô hình có một block riêng:
 - **Random Forest**: chuyển cây quyết định thành chuỗi if-else lồng nhau.

- **Neural Network:** nhúng trọng số/bias và sinh mã nhân ma trận.
- **CNN:** sinh các kernel Conv1D hoặc Conv2D, pooling, fully connected layers.
- **SVM:** nhúng support vectors và sinh mã tính kernel.

4.6.3 Kiến trúc ba trục: Model $\times$ Platform $\times$ Backend

Framework triển khai ma trận deployment với ba chiều độc lập:

Trục 1: Model Type

- Random Forest, SVM, Neural Network (MLP)
- PyTorch MLP, 1D-CNN, 2D-CNN

Trục 2: Target Platform

- **Arduino (Cortex-M):** C++ code tối ưu cho ARM MCU, không sử dụng thư viện ngoài, tất cả tính toán inline.
- **MicroPython (ESP32):** Python code embed constants, sử dụng ulab (NumPy subset) nếu có.
- **Zephyr RTOS:** C code với Zephyr API cho threading và sensors.

Trục 3: Deployment Backend

- **Direct (native):** mã C/C++ trực tiếp, không phụ thuộc thư viện ngoài.
- **TFLite Micro:** chuyển đổi PyTorch model sang TFLite, sinh wrapper C++ gọi interpreter. Chỉ áp dụng cho NN/CNN (RF/SVM không convertible qua ONNX-ML operators).
- **ONNX Runtime:** xuất model sang ONNX, deploy với onnxruntime-embedded.

Ưu điểm của thiết kế này:

- **Tính mô-đun:** thêm model type mới chỉ cần implement một block.
- **Tính mở rộng:** thêm platform mới chỉ cần điều chỉnh template rendering.
- **Tính linh hoạt:** người dùng tự do lựa chọn backend dựa trên yêu cầu (performance, memory, dependencies).

4.6.4 Sắp xếp lại thứ tự đặc trưng khi sinh mã

Một vấn đề quan trọng trong deployment parity là **thứ tự features**:

- **Python training**: features được trích xuất từ Pandas DataFrame, thứ tự là alphabetical theo tên cột (VD: `acc_jerk_mag_mean`, `acc_mag_energy`, `acc_mag_iqr`, ...).
- **C++ deployment**: features được tính trong hàm `extract_features()`, thứ tự là computation order (VD: `acc_mag` stats trước, sau đó `gyro_mag` stats, cuối cùng là `acc_jerk`).

Nếu model weights được serialize theo thứ tự Python training, nhưng C++ code đưa features vào theo thứ tự khác, kết quả suy luận sẽ sai hoàn toàn.

Giải pháp: sắp xếp lại trọng số khi sinh mã Framework xử lý vấn đề này bằng cách tính toán thứ tự đặc trưng trong C++ (magnitude stats cho accelerometer, gyroscope, rồi jerk features), so sánh với thứ tự alphabetical trong Python, và tạo bảng ánh xạ chỉ số tương ứng. Dựa trên bảng ánh xạ này, các tham số scaler (mean, scale) và trọng số lớp đầu tiên của mô hình được sắp xếp lại trước khi nhúng vào mã C++.

Giải pháp này đảm bảo mô hình nhận features theo đúng thứ tự training mà không cần điều chỉnh C++ code.

4.6.5 Bốn chế độ tối ưu hóa

Framework cung cấp 4 chế độ tối ưu để cân bằng giữa accuracy, tốc độ và tiêu thụ điện:

Bảng 4.6.1: Chế độ tối ưu hóa mã triển khai

Chế độ	Độ chính xác số	Trade-off	Use case
Accuracy	6 chữ số thập phân	Bộ nhớ tối đa	Nghiên cứu, thử nghiệm
Balanced	3 chữ số thập phân	Cân bằng	Sản xuất chung
Speed	2–3 chữ số thập phân	Giảm độ trễ	Thời gian thực
Power	3–4 chữ số + DSP	Giảm chu kỳ CPU	Thiết bị dùng pin

Chế độ *Accuracy* sử dụng số thực 32-bit đầy đủ; *Balanced* làm tròn hệ số và dùng bảng tra cho hàm activation, giảm 20–30% kích thước mã; *Speed* áp dụng làm tròn mạnh và loại bỏ các đặc trưng đóng góp thấp; *Power* tận dụng ARM CMSIS-DSP khi

nền tảng hỗ trợ. Người dùng chọn chế độ trong giao diện code generation, framework tự động áp dụng các chuyển đổi tương ứng.

4.6.6 Gói đầu ra triển khai

Mã được sinh ra bao gồm 3 files chính:

1. **Header file (.h)**: khai báo constants (số features, số classes, window size, tần số lấy mẫu), tham số scaler, và prototype các hàm inference.
2. **Implementation file (.cpp)**: định nghĩa toàn bộ logic inference, bao gồm tính toán đặc trưng từ tín hiệu thô, chuẩn hóa bằng scaler từ quá trình huấn luyện, thực thi thuật toán phân loại, và tra cứu nhãn lớp từ kết quả dự đoán.
3. **Arduino sketch (.ino)**: ví dụ hoàn chỉnh cho Seeed XIAO nRF52840, bao gồm khởi tạo IMU và giao tiếp serial, vòng lặp thu thập đủ 150 mẫu vào bộ đệm rồi trích xuất đặc trưng, phân loại và xuất kết quả.

Hình 4.6.1 minh họa giao diện code generation.

4.6. Module tạo mã triển khai

Step 1: Select Trained Model

Select Model:

pytorch_cnn_har_model_20260520_001738 | Train: 100.0% | Test: 100.0% | Val: 100.0% | 3 classes

Model: pytorch_cnn_har_model_20260520_001738
Type: PYTORCH_CNN
Accuracy: Train 100.0% | Val 100.0% | Test 100.0%
Input: 150 samples × 6 ch (raw windows)
Classes: 3 activities

Step 2: Configure Target Platform

Output Language / Framework:

◆ Arduino C++ (.ino) — Arduino framework, widest board support

◆ Arduino C++ uses the Arduino framework with setup()/loop() and Serial. Compatible with both Arduino CLI and PlatformIO toolchains. Generates .ino files (auto-converted to .app for PlatformIO).

Target Board:

◆ XIAO nRF52840 Sense (BLE + IMU)

XIAO nRF52840 Sense • RAM: 256 KB • Flash: 1024 KB • Clock: 64 MHz

Model Parameters (from training)

Sampling Rate: 100 Hz (from training)
Window Size: 1500 ms (150 samples @ 100 Hz)
Feature Count: 150 samples × 6 channels = 900 inputs (raw windows, no FE) (FE used for other models: orientation, robust, time + freq (DFT))

Signal Preprocessing (training—device parity):
Low-pass filter: 5Hz (order 2) — replicated on device via per-window firfilt
Savitzky-Golay (window=5, poly=2) — replicated on device
Outlier removal (3-sigma) — NOT replicated on device (not needed for real-time)

Deployment Approach:

◆ Direct C/C++ Code Generation — Standalone, no runtime dependency

◆ Direct: All C/C++ code generated — no external runtime needed
◆ TFLite Micro: .flatc model + interpreter (requires tensorflow)
◆ ONNX Runtime: .onnx model + ONNX C++ API (requires onnx, sh2onnx)

Optimization Level:

◆ Balanced

Weight Quantization:

◆ INT8 — 75% smaller, ~1-2% accuracy loss

Window Overlap (%):
— 50 — +
0% — no overlap (fastest) • 50% — 2× overlap • 75% — smoothest

Confidence Threshold:
— 0.6 — +
Below threshold — 'unknown'. Higher — fewer false positives

Prediction Smoothing:
— 3 — +
Majority vote over N predictions. 1 — off, 3 — recommended

On-device Signal Preprocessing

Preprocessing is automatically read from training metadata — no manual selection needed. Signal filters are generated if and only if they were used during training.

LFF filter SavGol FFT trick+wall Kalman

✓ Active methods are highlighted in green for this model.

Select a model above to view its active preprocessing pipeline in the Model Info panel.

Hình 4.6.1: Giao diện tạo mã triển khai. Hiển thị các tệp header, implementation và Arduino sketch được sinh ra. Người dùng chọn chế độ tối ưu hóa (Accuracy, Speed, Power, Balanced), cấu hình nền tảng đích và tải xuống gói triển khai hoàn chỉnh.

4.6.7 Kiến trúc đa mô hình: CNN code generation

CNN deployment khác với traditional ML vì không cần feature extraction—model nhận raw sensor data trực tiếp. Code generator cho CNN phải sinh các thành phần sau:

Conv1D và Conv2D kernel Đối với 1D-CNN, generator sinh mã thực hiện phép tích chập một chiều trên trục thời gian: mỗi filter duyệt qua tín hiệu đầu vào ($6 \text{ channels} \times 150 \text{ timesteps}$) với kernel size 5, tạo ra feature map đầu ra. Đối với 2D-CNN (HARCNN2D), generator sinh phép tích chập hai chiều với kernel shape ($3 \times n_{\text{channels}}$) để học tương quan chéo giữa các kênh cảm biến. Tất cả trọng số kernel và bias được nhúng trực tiếp dưới dạng mảng hằng số trong mã sinh ra.

MaxPooling và fully connected layers Generator cũng sinh mã cho max pooling (giảm chiều thời gian) và các lớp fully connected (phân loại dựa trên feature map đã trích xuất). Các phép toán này được triển khai bằng vòng lặp đơn giản, không phụ thuộc thư viện bên ngoài.

Bỏ qua scaler cho CNN Một phát hiện quan trọng trong quá trình phát triển: StandardScaler được huấn luyện trên *features* (mean, std, v.v.) không nên áp dụng lên *raw sensor values* (đầu vào của CNN). Nếu áp dụng, phép chuẩn hóa sẽ clamp các giá trị gyro vào range $[-10, 10]$ rad/s, phá hủy thông tin tín hiệu. Generator phân biệt kiến trúc mô hình: CNN nhận dữ liệu thô và bỏ qua scaler, trong khi các mô hình ML cổ điển vẫn áp dụng chuẩn hóa trên features đã trích xuất. Giải pháp này tiết kiệm 7KB Flash và loại bỏ hoàn toàn nguy cơ suy luận sai do biến dạng scaler.

4.7 Module kiểm tra thiết bị

Module này (Tab 6) cung cấp giao diện kết nối với thiết bị đã flash mã triển khai để xác minh hoạt động real-time.

4.7.1 Giao tiếp serial

Framework sử dụng thư viện `pyserial` để mở kết nối serial với thiết bị đã flash. Thiết bị gửi kết quả phân loại qua cổng serial theo định dạng chuẩn gồm tên hoạt động dự đoán, điểm tin cậy, và phân phối xác suất toàn bộ classes. Callback nhận từng dòng dữ liệu, phân tích cú pháp và cập nhật giao diện theo thời gian thực.

4.7.2 Hiện thị phân loại real-time

Giao diện hiển thị:

- **Hoạt động dự đoán:** tên hoạt động dự đoán với màu sắc tương ứng.
- **Thanh tin cậy:** thanh ngang hiển thị xác suất của class được dự đoán (0–100%).
- **Phân phối xác suất:** biểu đồ cột hiển thị xác suất của tất cả classes, giúp phát hiện các trường hợp mơ hồ (nhiều classes có xác suất gần bằng nhau).
- **Dòng thời gian:** biểu đồ hiển thị lịch sử dự đoán 30 giây gần nhất, cho thấy quá trình chuyển đổi giữa các hoạt động.

4.7.3 Ngưỡng tin cậy

Framework sinh mã có confidence thresholding: nếu xác suất cao nhất trong phân phối dự đoán thấp hơn ngưỡng, mô hình trả về UNKNOWN thay vì gán nhãn với tin cậy thấp. Cơ chế này quan trọng cho triển khai thực tế khi gặp hoạt động ngoài tập huấn luyện (out-of-distribution). Người dùng có thể điều chỉnh ngưỡng trong giao diện (mặc định: 0,7).

4.7.4 Quy trình xác minh triển khai

Workflow kiểm thử trên thiết bị:

1. **Flash code:** upload mã sinh ra lên vi điều khiển qua Arduino IDE hoặc PlatformIO.
2. **Connect serial:** mở Tab 6, chọn COM port, kết nối.
3. **Perform activities:** thực hiện các hoạt động từ tập training (VD: đứng yên, đi bộ, chạy).
4. **Verify predictions:** so sánh dự đoán của thiết bị với ground truth.
5. **Test edge cases:** thực hiện hoạt động ngoài tập training (VD: nhảy, đạp xe) để kiểm tra UNKNOWN detection.
6. **Measure latency:** framework hiển thị thời gian xử lý mỗi window (từ thu thập sensor đến xuất kết quả).

Nếu phát hiện mismatch giữa dự đoán Python và thiết bị, framework cung cấp công cụ debugging:

- **So sánh đặc trưng:** xuất features từ thiết bị qua serial, so sánh với Python.
- **Kiểm tra parity:** kiểm tra từng bước trong pipeline (tính magnitude, đặc trưng thống kê, scaler, dự đoán).

4.8 Luồng dữ liệu và quản lý trạng thái tổng thể

4.8.1 Luồng dữ liệu end-to-end

Dữ liệu chảy tuần tự qua 6 giai đoạn, mỗi giai đoạn đọc kết quả của giai đoạn trước từ hệ thống file:

1. **Tải dữ liệu (Tab 1):** CSV gốc $\rightarrow$ lưu theo activity class, phát hiện cấu trúc cột.
2. **Tiền xử lý (Tab 2):** Chọn segments trên biểu đồ $\rightarrow$ sinh sliding windows.
3. **Trích xuất đặc trưng (Tab 3):** Windows $\rightarrow$ feature vectors + augmentation + phân chia train/val/test.
4. **Huấn luyện (Tab 4):** Feature matrices $\rightarrow$ GridSearchCV $\rightarrow$ model bundle (.joblib).
5. **Tạo mã (Tab 5):** Model bundle $\rightarrow$ trích xuất tham số + sắp xếp lại features $\rightarrow$ mã C++/MicroPython.
6. **Kiểm tra thiết bị (Tab 6):** Giao tiếp serial với thiết bị đã flash $\rightarrow$ xác minh dự đoán real-time.

Thiết kế filesystem-based cho phép người dùng quay lại bất kỳ bước nào: thay đổi feature mode không cần chọn lại segments, thay đổi platform không cần huấn luyện lại mô hình.

4.9 Tổng kết chương

Chương này trình bày chi tiết kiến trúc hệ thống và hiện thực kỹ thuật của framework HAR Edge Deployment. Các điểm chính:

1. **Kiến trúc module hóa:** hệ thống được tổ chức thành 6 modules độc lập tương ứng với 6 tabs, mỗi module chịu trách nhiệm một giai đoạn rõ ràng trong pipeline.
2. **State management hybrid:** kết hợp filesystem persistence (cho dữ liệu trung gian) và in-memory state (cho session-specific metadata), cho phép rollback và retry linh hoạt.
3. **Code generation architecture:** factory pattern + block-based clean architecture, hỗ trợ ma trận 3 chiều (model $\times$ platform $\times$ backend) mà vẫn duy trì tính module.
4. **Training-deployment parity:** các cơ chế bảo đảm nhất quán (feature reordering, scaler bypass cho CNN, confidence thresholding) đảm bảo mô hình hoạt động đúng trên thiết bị.

5. **User experience:** giao diện tương tác (drag-and-drop, real-time monitoring) giảm thiểu barrier to entry, cho phép người dùng không chuyên về embedded systems vẫn triển khai được edge ML.

Chương tiếp theo sẽ trình bày kết quả thực nghiệm, đo lường hiệu năng trên Seeed XIAO nRF52840, và so sánh với các nền tảng thương mại.

Chương 5

Kết quả thực nghiệm

5.1 Thiết lập thực nghiệm

Toàn bộ thực nghiệm sử dụng dữ liệu IMU 6 trục (gia tốc kế + con quay hồi chuyển) được thu thập từ một đối tượng trên thiết bị Seeed XIAO nRF52840 Sense (IMU LSM6DS3 tích hợp) ở tần số lấy mẫu 100 Hz. Dữ liệu gồm các bản ghi 30–60 giây cho mỗi hoạt động. Hai cấu hình bài toán được đánh giá:

- **Bài toán 3 lớp:** running, still, walking — ba hoạt động có đặc điểm sinh cơ học khác biệt rõ rệt
- **Bài toán 5 lớp:** running, still, walking, walking_downstairs, walking_upstairs — bổ sung hai hoạt động có dấu hiệu chuyển động tương tự nhau (leo và xuống cầu thang so với đi bộ thường)

Các tham số chung: kích thước cửa sổ $W = 150$ mẫu (1,5 giây), bước trượt $S = 75$ mẫu (50% chồng lấp), đệm cửa sổ bằng lặp giá trị biên (edge-value replication). Tăng cường dữ liệu (augmentation) với hệ số 2 sử dụng ba phương pháp (jitter, rotation, scaling). Chia dữ liệu: 70% huấn luyện, 15% xác thực, 15% kiểm tra, sử dụng phân chia phân tầng (stratified split). Loại đặc trưng sử dụng: `orientation_invariant` (63 đặc trưng: thống kê thời gian trên `acc_mag`, `gyro_mag`, `acc_jerk_mag` cộng với đặc trưng phổ DFT trên `acc_mag` và `gyro_mag`); riêng CNN sử dụng đầu vào thô 6 trục (150×6).

Quy mô dữ liệu: Giao diện kéo thả phân đoạn bản ghi thành các mẫu được gán nhãn, từ đó cửa sổ trượt sinh 507 cửa sổ cho bài toán 5 lớp (345 cho 3 lớp). Tăng cường hệ số 2 nâng tổng lên 1521 mẫu (5 lớp) và 1035 mẫu (3 lớp), chia thành 1063 train / 229 val / 229 test (5 lớp). Phân bố lớp trước tăng cường: Running 124 (24,5%), Still

112 (22,1%), Walking 109 (21,5%), Walking Downstairs 85 (16,8%), Walking Upstairs 77 (15,2%) — tỷ lệ mất cân bằng 1,61:1, đại diện cho tập dữ liệu tương đối cân bằng.

Năm thuật toán được đánh giá: Neural Network (sklearn MLP), Random Forest, SVM (RBF kernel), PyTorch MLP và PyTorch 1D-CNN. Tất cả các mô hình sử dụng cân bằng trọng số lớp (`class_weight='balanced'`) khi áp dụng được.

Số liệu đánh giá: Hiệu suất mô hình được đo bằng độ chính xác tổng thể, precision/recall/F1-score từng lớp và confusion matrix. Hiệu suất triển khai được đo qua thời gian suy luận trên thiết bị (`micros()`), mức sử dụng flash/SRAM (từ bản đồ biên dịch)¹. Ngưỡng mục tiêu: suy luận <100ms, flash <500KB, SRAM <128KB.

So sánh cơ sở: Kết quả được so sánh trực tiếp với Edge Impulse (gói miễn phí): cùng tập dữ liệu 3 lớp, mô hình CNN mặc định với đặc trưng phổ, lượng tử hóa INT8, triển khai qua EON Compiler cho Cortex-M4F 64 MHz (Mục 5.6).

5.2 Kết quả huấn luyện

5.2.1 Bài toán 3 lớp (running / still / walking)

Bảng 5.2.1 tóm tắt kết quả huấn luyện cho bài toán 3 lớp. Ba hoạt động này có đặc trưng sinh cơ học phân biệt rõ rệt: running có biên độ gia tốc cao và tần số bước nhanh; still gần như bất động với magnitude gia tốc ổn định quanh g ; walking có biên độ trung gian với tần số bước đặc trưng.

Bảng 5.2.1: Kết quả huấn luyện — bài toán 3 lớp (63 đặc trưng orientation-invariant, tập kiểm tra 156 mẫu)

Thuật toán	Kiến trúc / Cấu hình	Thời gian huấn luyện (s)	Độ chính xác huấn luyện (%)	Độ chính xác xác thực (%)	Độ chính xác kiểm tra (%)
Neural Network	63-100-50-3	0,22	99,17	100,00	100,00
Random Forest	50 cây, sâu tối đa 10	0,12	100,00	—	100,00
SVM (RBF)	$C=1$, $\gamma=\text{scale}$	0,09	100,00	—	100,00
PyTorch MLP	63-ẩn-3	2,54	99,86	100,00	100,00
PyTorch 1D-CNN	150×6 thô	10,29	100,00	100,00	100,00

Tất cả năm thuật toán đạt 100% độ chính xác trên tập kiểm tra cho bài toán 3 lớp, với F1-score = 1,0 trên mọi lớp. Kết quả này xác nhận rằng ba hoạt động cơ bản (chạy, đứng yên, đi bộ) có thể được phân biệt hoàn toàn bằng 63 đặc trưng bất biến hướng. Đáng chú ý, thời gian huấn luyện của các mô hình cổ điển (0,09–0,22 giây) nhanh hơn

¹Ước lượng tiêu thụ điện năng từ đặc tả ARM Cortex-M4F (8–12mA hoạt động, 2–3mA nhàn rỗi ở 64 MHz); giá trị thực tế có thể thay đổi $\pm 20\%$.

đáng kể so với PyTorch (2,54–10,29 giây), cho thấy lợi thế của ML cổ điển khi bài toán không quá phức tạp.

5.2.2 Bài toán 5 lớp — Đánh giá chính

Bài toán 5 lớp bổ sung walking_downstairs và walking_upstairs — hai hoạt động có cấu trúc bước tương tự đi bộ thường nhưng khác nhau về thành phần xoay và biên độ gia tốc thẳng đứng. Đây là bài toán thách thức hơn đáng kể, thể hiện đúng hơn các yêu cầu thực tế. Bảng 5.2.2 trình bày kết quả.

Bảng 5.2.2: Kết quả huấn luyện — bài toán 5 lớp (63 đặc trưng orientation-invariant, tập kiểm tra 229 mẫu)

Thuật toán	Kiến trúc / Cấu hình	Thời gian huấn luyện (s)	Độ chính xác huấn luyện (%)	Độ chính xác xác thực (%)	Độ chính xác kiểm tra (%)
Neural Network	63–100–50–5	0,42	99,72	98,25	98,69
Random Forest	50 cây, sâu tối đa 10	0,15	99,81	—	98,25
SVM (RBF)	$C=1$, $\gamma=\text{scale}$	0,15	97,55	—	97,38
PyTorch MLP	63–ẩn–5	1,94	99,81	98,69	99,13
PyTorch 1D-CNN	150×6 thô	10,09	99,25	99,56	99,13

Khi mở rộng từ 3 lên 5 lớp, độ chính xác kiểm tra giảm từ 100% xuống 97,38–99,13%, phản ánh độ khó phân biệt giữa các hoạt động liên quan (walking vs walking_upstairs/downstairs). Các quan sát chính:

- **PyTorch MLP và 1D-CNN** đạt độ chính xác cao nhất (99,13%), cho thấy lợi thế của các kiến trúc học sâu khi bài toán phức tạp hơn.
- **Neural Network (sklearn)** đạt 98,69% với thời gian huấn luyện chỉ 0,42 giây — cân bằng tốt giữa hiệu suất và tốc độ phát triển.
- **SVM** có độ chính xác thấp nhất (97,38%), với độ chính xác huấn luyện cũng chỉ 97,55% — gợi ý rằng kernel RBF với cấu hình mặc định chưa nắm bắt đủ cấu trúc phi tuyến của bài toán 5 lớp; tối ưu hóa siêu tham số có thể cải thiện đáng kể.
- **1D-CNN** hoạt động trên dữ liệu thô mà không cần trích xuất đặc trưng thủ công, đạt hiệu suất ngang PyTorch MLP nhưng tốn thời gian huấn luyện nhiều hơn (10,09 giây).

5.2.3 Hiệu suất theo từng lớp

Bảng 5.2.3 trình bày chi tiết precision, recall và F1-score cho từng lớp hoạt động trên bài toán 5 lớp. Ba thuật toán đại diện cho ba nhóm tiếp cận (ML cổ điển: RF; neural truyền thống: NN; học sâu: PyTorch MLP) được so sánh.

Bảng 5.2.3: Hiệu suất theo từng lớp — bài toán 5 lớp, tập kiểm tra (229 mẫu)

Hoạt động	Neural Network (98,69%)			Random Forest (98,25%)			PyTorch MLP (99,13%)		
	P	R	F1	P	R	F1	P	R	F1
Running (56)	1,000	1,000	1,000	1,000	1,000	1,000	1,000	1,000	1,000
Still (51)	1,000	1,000	1,000	1,000	1,000	1,000	1,000	1,000	1,000
Walking (49)	1,000	0,980	0,990	0,979	0,959	0,969	1,000	0,980	0,990
W. Downstairs (38)	0,949	0,974	0,961	0,925	0,974	0,949	0,950	1,000	0,974
W. Upstairs (35)	0,971	0,971	0,971	1,000	0,971	0,986	1,000	0,971	0,986
Macro Avg	0,984	0,985	0,984	0,981	0,981	0,981	0,990	0,990	0,990

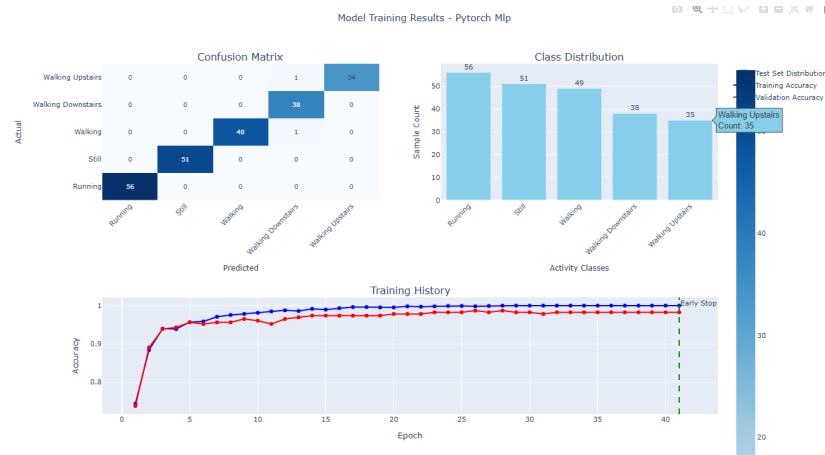
P = Precision, R = Recall. Số trong ngoặc = số mẫu kiểm tra (support). W. = Walking.

Phân tích theo từng lớp cho thấy các mẫu phân loại sai tập trung ở nhóm hoạt động đi bộ:

- **Running và Still** đạt $F1 = 1,000$ trên tất cả các thuật toán — dấu hiệu sinh cơ học đặc trưng (biên độ gia tốc cao cho running, magnitude ổn định quanh g cho still) tạo ranh giới phân loại rõ ràng.
- **Walking Downstairs** có precision thấp nhất (0,925–0,950), nghĩa là một số mẫu từ lớp khác (chủ yếu walking) bị phân loại nhầm thành walking_downstairs — phù hợp với đặc điểm rằng đi bộ thường và xuống cầu thang chia sẻ biên độ gia tốc tương tự.
- **Walking Upstairs** đạt recall 0,971 nhất quán trên mọi thuật toán — một vài mẫu bị nhầm lẫn với walking thường, nhưng precision cao (0,971–1,000) cho thấy khi mô hình dự đoán walking_upstairs thì hầu như luôn đúng.

Kết quả tổng thể xác nhận rằng loại đặc trưng **orientation_invariant** có khả năng phân biệt tốt giữa 5 hoạt động, với sai sót chủ yếu ở ranh giới giữa các dạng đi bộ — nơi sự khác biệt chủ yếu nằm ở thành phần xoay (vận tốc góc gấp cổ chân) mà đặc trưng tổng hợp magnitude bắt giữ nhưng không hoàn toàn tách biệt. Hình 5.2.1 minh họa ma trận nhầm lẫn trực quan cho mô hình PyTorch MLP — thuật toán đạt F1 cao nhất (0,990) trên đặc trưng thủ công.

5.3. So sánh hiệu suất giữa các thuật toán



Hình 5.2.1: Ma trận nhầm lẫn — PyTorch MLP, bài toán 5 lớp (tập kiểm tra 229 mẫu). Các lỗi phân loại tập trung ở nhóm hoạt động đi bộ: 1 mẫu walking bị nhầm thành walking_downstairs (hàng walking, cột W.Down).

5.3 So sánh hiệu suất giữa các thuật toán

Bảng 5.3.1 tổng hợp so sánh trên nhiều tiêu chí cho bài toán 5 lớp.

Bảng 5.3.1: So sánh tổng hợp các thuật toán — bài toán 5 lớp

Thuật toán	Độ chính xác kiểm tra (%)	Macro F1	Thời gian huấn luyện (s)	Đầu vào (số chiều)	Nhận xét
PyTorch MLP	99,13	0,990	1,94	63	Tốt nhất trên đặc trưng thủ công
PyTorch 1D-CNN	99,13	0,992	10,09	150×6	End-to-end, không cần FE
Neural Network	98,69	0,984	0,42	63	Nhanh, đủ tốt cho prototyping
Random Forest	98,25	0,981	0,15	63	Nhanh nhất, không cần chuẩn hóa
SVM (RBF)	97,38	0,972	0,15	63	Cần tối ưu hóa siêu tham số

Chênh lệch giữa thuật toán tốt nhất (PyTorch MLP/CNN: 99,13%) và thấp nhất (SVM: 97,38%) chỉ 1,75 điểm phần trăm — cho thấy chất lượng pipeline trích xuất đặc trưng quan trọng hơn lựa chọn thuật toán khi đặc trưng đã được thiết kế tốt. Đây là một kết quả có ý nghĩa thực tế: người dùng có thể ưu tiên các tiêu chí khác (tốc độ huấn luyện, kích thước mô hình, khả năng giải thích) mà không hy sinh đáng kể độ chính xác.

Ảnh hưởng của số lượng lớp đến hiệu suất được tóm tắt trong Bảng 5.3.2.

5.4. Khả năng triển khai đa thiết bị

Bảng 5.3.2: Ảnh hưởng của số lượng lớp đến độ chính xác kiểm tra

Thuật toán	3 lớp (%)	5 lớp (%)	Giảm (điểm %)
Neural Network	100,00	98,69	−1,31
Random Forest	100,00	98,25	−1,75
SVM (RBF)	100,00	97,38	−2,62
PyTorch MLP	100,00	99,13	−0,87
PyTorch 1D-CNN	100,00	99,13	−0,87

Các mô hình PyTorch giảm ít nhất (−0,87 điểm), trong khi SVM giảm nhiều nhất (−2,62 điểm). Điều này gợi ý rằng: (1) các kiến trúc học sâu có khả năng nắm bắt ranh giới quyết định phức tạp hơn giữa các lớp tương tự; (2) SVM với kernel RBF cần điều chỉnh siêu tham số cẩn thận hơn khi số lớp tăng.

5.4 Khả năng triển khai đa thiết bị

Đóng góp cốt lõi của framework không chỉ nằm ở độ chính xác mô hình, mà ở khả năng sử dụng cùng một mô hình đã huấn luyện để sinh mã triển khai cho nhiều họ thiết bị khác nhau mà không cần huấn luyện lại. Bảng 5.4.1 tóm tắt ma trận đích triển khai được hỗ trợ.

Bảng 5.4.1: Ma trận đích triển khai — phạm vi hỗ trợ của hệ thống tạo mã

Nhóm đích	Nền tảng cụ thể	Ngôn ngữ	Phạm vi xác thực
Arduino-compatible	Seeed XIAO, ESP32, M5Stack, Teensy	C++	Benchmark chi tiết trên XIAO
ARM Cortex-M	Generic ARM Cortex-M	C	Xác minh biên dịch + cấu trúc
SDK/RTOS	ESP-IDF, Zephyr, Generic C/C++	C/C++	Xác minh tạo mã
MicroPython	Vi điều khiển hỗ trợ MPy	Python	Xác minh tương đồng công thức
Runtime thay thế	TFLite ONNX Runtime	Micro, C++ + nhĩ phân	Chỉ NN/CNN

Kiến trúc tạo mã phân tách ba lớp — mô hình ML, nền tảng đích, backend triển khai — cho phép tổ hợp linh hoạt: một Random Forest đã huấn luyện có thể được xuất đồng thời sang C++ cho Arduino, C cho ARM Cortex-M, và Python cho MicroPython, từ cùng một tệp `.joblib`. Bốn chế độ tối ưu hóa (**accuracy**, **balanced**, **speed**, **power**)

điều chỉnh độ chính xác số học (số chữ số thập phân cho trọng số và tham số chuẩn hóa) và chiến lược bộ đệm theo ràng buộc tài nguyên của nền tảng đích.

Giới hạn backend runtime thay thế: TFLite Micro và ONNX Runtime chỉ hỗ trợ trực tiếp Neural Network và CNN. Random Forest và SVM không thể chuyển đổi qua đường dẫn ONNX $\rightarrow$ TFLite vì các toán tử ONNX-ML (`TreeEnsembleClassifier`, `SVMClassifier`) không được `onnx2tf` hỗ trợ. Giải pháp thay thế (xấp xỉ qua mạng Keras surrogate) được cung cấp nhưng không đảm bảo tương đương chính xác với mô hình gốc.

5.5 Đánh giá triển khai trên thiết bị

Để đánh giá hiệu suất triển khai thực tế, mã C++ được sinh ra cho từng mô hình được biên dịch và nạp lên thiết bị Seeed XIAO nRF52840 Sense (ARM Cortex-M4F @ 64 MHz, 1 MB flash, 256 KB RAM). Kiểm tra được thực hiện bằng cách thực hiện từng hoạt động và quan sát kết quả phân loại thời gian thực qua giao diện Serial. Ngưỡng tin cậy $\tau = 0,6$ được sử dụng.

5.5.1 Tổng hợp kết quả triển khai

Bảng 5.5.1 tóm tắt kết quả triển khai cho cả hai cấu hình bài toán.

Bảng 5.5.1: Kết quả triển khai trên thiết bị — Seeed XIAO nRF52840

Mô hình	3 lớp	5 lớp	Mô tả
PyTorch 1D-CNN	Tốt	Tốt	Nhận dạng đúng tất cả hoạt động
Random Forest	Tốt	Một phần	5 lớp: still, running đúng; các biến thể walking không phân biệt được
Neural Network	Một phần	Một phần	3 lớp: still, walking đúng; running nhầm walking. 5 lớp: walking, running đúng; still $\rightarrow$ walking_upstairs
PyTorch MLP	Sai lớp	Một phần	3 lớp: still $\leftrightarrow$ walking đảo, running sai; 5 lớp: still $\rightarrow$ walking_upstairs

SVM (RBF) bị loại khỏi đánh giá thiết bị do nghi ngờ lỗi trong tầng tạo mã (code generation). Kết quả bốn thuật toán còn lại cho thấy sự khác biệt rõ rệt giữa hiệu suất offline (97–100% trên tập kiểm tra) và hiệu suất triển khai thực tế: chỉ PyTorch

1D-CNN hoạt động đáng tin cậy trên cả hai cấu hình; Random Forest hoạt động tốt cho bài toán 3 lớp; Neural Network và PyTorch MLP hoạt động một phần nhưng đều gặp lỗi nhầm lẫn lớp.

5.5.2 Phân tích theo từng mô hình

PyTorch 1D-CNN là mô hình duy nhất hoạt động tốt trên cả hai cấu hình. CNN xử lý trực tiếp dữ liệu cảm biến thô mà không cần trích xuất đặc trưng trong C++, loại bỏ hoàn toàn nguy cơ sai lệch tính toán đặc trưng giữa Python và C++.

Random Forest hoạt động tốt cho bài toán 3 lớp nhưng không phân biệt được các biến thể walking ở bài toán 5 lớp — phù hợp với F1-score thấp nhất trên nhóm walking ngay cả trong huấn luyện, cho thấy ranh giới phân loại vốn đã mỏng và không ổn định khi triển khai thực tế.

PyTorch MLP gặp nhầm lẫn lớp có hệ thống: bài toán 3 lớp đảo ngược still và walking; bài toán 5 lớp phân loại nhầm still thành walking_upstairs. Pattern nhầm lẫn gợi ý sai lệch ở tầng xuất trọng số hoặc chuẩn hóa đặc trưng từ PyTorch sang C++.

Neural Network (sklearn MLP) nhận dạng được một phần nhưng nhầm running thành walking (3 lớp) và nhầm still thành walking_upstairs (5 lớp) — hành vi tương tự Edge Impulse CNN trên cùng tập dữ liệu. **SVM (RBF)** bị loại khỏi đánh giá thiết bị do nghi ngờ lỗi trong tầng tạo mã cho kernel RBF.

5.5.3 Bài học: khoảng cách huấn luyện–triển khai

Kết quả triển khai cho thấy ba nguồn gây khoảng cách giữa độ chính xác offline và hiệu suất thực tế:

- **Tương đồng tính toán đặc trưng:** Sai lệch nhỏ trong float32 so với float64, hoặc trong các công thức phức tạp (kurtosis, skewness, DFT), có thể tích lũy qua 63 đặc trưng. CNN tránh hoàn toàn vấn đề này bằng cách hoạt động trên dữ liệu thô.
- **Độ chính xác số học:** Sai lệch khi chuyển đổi trọng số hoặc tham số chuẩn hóa có thể đẩy mẫu gần ranh giới quyết định sang lớp sai, đặc biệt với các mô hình có biên quyết định mỏng.
- **Điều kiện thực tế:** Hướng gán, tốc độ thực hiện và nhiễu môi trường tạo phân phối đầu vào khác với dữ liệu huấn luyện, khiến mô hình nhạy cảm ở các ranh giới walking variants.

Khuyến nghị: PyTorch 1D-CNN là lựa chọn triển khai đáng tin cậy nhất. Random Forest phù hợp cho bài toán có ranh giới rõ ràng và cần kích thước mô hình nhỏ. Neural Network và PyTorch MLP cần cải thiện ở tầng tạo mã trước khi triển khai đáng tin cậy.

5.6 So sánh với Edge Impulse

Để đánh giá khách quan, cùng tập dữ liệu 3 lớp (running/still/walking) được tải lên Edge Impulse (gói miễn phí) và huấn luyện với cấu hình mặc định: CNN với đặc trưng phổ (spectral features), lượng tử hóa INT8, triển khai qua EON Compiler cho Cortex-M4F 64 MHz—cùng vi xử lý với thiết bị benchmark.

5.6.1 So sánh hiệu suất

Bảng 5.6.1 so sánh kết quả huấn luyện và triển khai.

Bảng 5.6.1: So sánh framework đề xuất vs Edge Impulse — bài toán 3 lớp, cùng tập dữ liệu

Hệ thống	Weighted F1	Running F1	Walking F1	Kết quả trên thiết bị	Ghi chú
Framework — 1D-CNN	1,00	1,00	1,00	Tốt (cả 3 lớp)	INT8, end-to-end
Framework — RF	1,00	1,00	1,00	Tốt (cả 3 lớp)	INT8, 50 cây
Framework — NN	1,00	1,00	1,00	Running lỗi	INT8, 63–100–50–3
Edge Impulse — CNN	0,96	1,00	0,89	Running lỗi	INT8, EON Compiler

Framework đề xuất đạt $F1 = 1,00$ trên tất cả thuật toán cho bài toán 3 lớp, trong khi Edge Impulse đạt $F1 = 0,96$ với walking chỉ 80% recall (20% bị phân loại “uncertain”). Trên thiết bị thực, Edge Impulse báo cáo độ chính xác ước tính 88,46% nhưng *không nhận dạng được running*—chỉ still và walking hoạt động. Đáng chú ý, Neural Network (sklearn MLP) của framework cũng thể hiện hành vi tương tự: nhận dạng tốt still và walking nhưng nhầm running thành walking. Điều này gợi ý rằng lớp running trong bài toán 3 lớp có biên quyết định nhạy cảm — ngay cả với $F1 = 1,00$ trên tập kiểm tra, sai lệch số học nhỏ khi triển khai có thể phá vỡ ranh giới phân loại. Ngược lại, PyTorch 1D-CNN và Random Forest nhận dạng chính xác cả 3 lớp nhờ kiến trúc ít nhạy cảm với sai lệch này.

5.6.2 So sánh tài nguyên

Bảng 5.6.2 so sánh mức sử dụng bộ nhớ.

5.6. So sánh với Edge Impulse

Bảng 5.6.2: So sánh tài nguyên triển khai — Cortex-M4F @ 64 MHz

Hệ thống	Flash (bytes)	SRAM (bytes)	Latency (ms)	Lượng tử hóa
Framework — CNN	145.728 (18%)	126.440 (53%)	20	INT8
Framework — RF	187.936 (23%)	49.640 (21%)	20	INT8
Framework — NN	155.816 (19%)	49.640 (21%)	20	INT8
Edge Impulse	417.856 (51%)	74.432 (31%)	10	INT8

Tất cả số liệu là tổng sketch đã biên dịch (bao gồm firmware, cảm biến, mô hình). Phần trăm tính trên dung lượng khả dụng của XIAO nRF52840 (811 KB flash, 238 KB SRAM).

Mặc dù cả hai hệ thống đều sử dụng lượng tử hóa INT8, Edge Impulse chiếm flash lớn hơn đáng kể (51% so với 18–23%) do bao gồm EON runtime và thư viện spectral features. Framework đề xuất đạt mức sử dụng bộ nhớ nhỏ hơn nhờ tạo mã trực tiếp không cần runtime, đồng thời đạt *độ chính xác triển khai cao hơn* với 1D-CNN và Random Forest (nhận dạng đúng cả 3 lớp so với 2/3 lớp của Edge Impulse).

Về độ trễ suy luận, Edge Impulse đạt 10 ms nhờ khối DSP spectral features được tối ưu hóa cao (sử dụng KissFFT với $N = 32$, chỉ 17 bin phổ khả dụng ở 100 Hz). Framework đề xuất đạt 20 ms cho toàn bộ pipeline (trích xuất 63 đặc trưng thống kê + suy luận mô hình), nhờ tối ưu hóa vòng lặp tính toán thống kê và sử dụng DFT với $N = 150$ mẫu — cho phép phân giải tần số 0,67 Hz, cao hơn đáng kể so với Edge Impulse (3,125 Hz với $N = 32$). Sự khác biệt về phân giải phổ này giải thích phần nào lý do framework đề xuất đạt độ chính xác cao hơn trên các hoạt động có thành phần tần số thấp (walking vs. walking downstairs).

5.6.3 So sánh kiến trúc

Bảng 5.6.3 tóm tắt các khác biệt kiến trúc chính.

Bảng 5.6.3: So sánh kiến trúc với các nền tảng thương mại

Tiêu chí	Framework đề xuất	Edge Impulse	SensiML
Chi phí	Mã nguồn mở, miễn phí	\$20–100/tháng	\$99–500/tháng
Thuật toán ML cổ điển	RF, SVM, NN	Không (chỉ neural)	Có
Minh bạch mã sinh ra	Toàn bộ mã nguồn	Hộp đen (EON)	Hộp đen
Xác minh Python↔C++	Có	Không công khai	Không công khai
Tiền xử lý tương tác	Kéo thả trên tín hiệu	Tự động (hạn chế)	Tự động

Điểm phân biệt then chốt là **tính minh bạch**: mã C++ sinh ra có thể đọc, kiểm tra và sửa đổi trực tiếp. Chính nhờ tính minh bạch này, các lỗi tương đồng huấn luyện–triển khai (zero-padding, sai công thức kurtosis/skewness, sai thứ tự đặc trưng — Mục 6.4) đã được phát hiện và sửa chữa—điều không thể thực hiện với các nền tảng hộp đen. Phân tích ý nghĩa kiến trúc được trình bày trong Chương 6.

5.7 Tóm tắt

Kết quả thực nghiệm xác nhận:

1. **Hiệu suất offline cao**: 97–99% trên bài toán 5 lớp, 100% trên 3 lớp. Chênh lệch giữa các thuật toán chỉ 1,75 điểm phần trăm.
2. **Khoảng cách triển khai**: Chỉ PyTorch 1D-CNN hoạt động đáng tin cậy trên cả hai cấu hình. Neural Network và PyTorch MLP hoạt động một phần nhưng gặp nhầm lẫn lớp. Độ chính xác offline không đảm bảo triển khai thành công.
3. **Vượt trội Edge Impulse**: Trên cùng tập dữ liệu và thiết bị, framework đạt $F1 = 1,00$ (3 lớp) so với 0,96 của Edge Impulse. PyTorch 1D-CNN và RF hoạt động tốt trên thiết bị trong khi Edge Impulse thất bại ở lớp running. Mức sử dụng flash cũng nhỏ hơn (18–23% so với 51%).
4. **Đa nền tảng**: Kiến trúc tạo mã hỗ trợ 5 nhóm nền tảng đích từ cùng mô hình đã huấn luyện.

Chương 6

Kết luận và hướng phát triển

6.1 Diễn giải các phát hiện chính

6.1.1 Hiệu suất mô hình và lựa chọn thuật toán

Các kết quả thực nghiệm chứng minh rằng tất cả năm thuật toán đạt độ chính xác cao (97,38–99,13%) trên bài toán HAR năm lớp, và đạt 100% trên bài toán 3 lớp cơ bản. Chênh lệch chỉ 1,75 điểm phần trăm giữa thuật toán tốt nhất (PyTorch MLP/CNN: 99,13%) và thấp nhất (SVM: 97,38%) cho thấy **chất lượng pipeline trích xuất đặc trưng quan trọng hơn lựa chọn thuật toán** khi bộ 63 đặc trưng bất biến hướng đã được thiết kế tốt. Thời gian huấn luyện ngắn của Random Forest (0,15s) và Neural Network sklearn (0,42s) làm cho chúng phù hợp cho prototyping nhanh và chu trình phát triển lặp.

6.1.2 Cân bằng lớp trong thực hành

Cân bằng trọng số lớp tự động được bật mặc định cho RF và SVM trong framework. Kết quả thực nghiệm cho thấy các lớp thiểu số (Walking Downstairs: 38 mẫu, Walking Upstairs: 35 mẫu) vẫn đạt recall 0,971–1,000 và $F1 \geq 0,949$ —xác nhận hiệu quả của chiến lược cân bằng. Lựa chọn thiết kế này phản ánh nguyên tắc ưu tiên triển khai thực tế: trong ứng dụng HAR, tất cả hoạt động phải được nhận dạng đáng tin cậy, không chỉ các lớp xuất hiện nhiều nhất.

6.1.3 Tính khả thi triển khai biên

Kết quả triển khai trên thiết bị (Bảng 5.5.1) cho thấy một phát hiện quan trọng: **chỉ PyTorch 1D-CNN hoạt động đáng tin cậy trên cả hai cấu hình bài toán (3 lớp và 5 lớp)**, trong khi Random Forest chỉ ổn định cho bài toán 3 lớp. Phát hiện

này nhấn mạnh rằng chênh lệch 1,75 điểm phần trăm trên tập kiểm tra ngoại tuyến trở nên vô nghĩa khi một số mô hình mắc lỗi phân loại nghiêm trọng trên thiết bị thực. Tiêu chí chọn mô hình cho triển khai biên cần mở rộng từ “độ chính xác cao nhất” sang *khả năng triển khai thực tế*.

Đáng chú ý, CNN—mô hình end-to-end hoạt động trực tiếp trên dữ liệu thô—cho kết quả tốt nhất trên thiết bị. Việc loại bỏ bước trích xuất đặc trưng trong mã C++ (nguồn chính của lỗi parity) mang lại lợi thế triển khai đáng kể, dù với chi phí kích thước mô hình lớn hơn và khả năng diễn giải thấp hơn.

6.1.4 Tiền xử lý tương tác: cách tiếp cận mới

Giao diện kéo thả cửa sổ thời gian đại diện cho một cách tiếp cận khác biệt so với quy trình tiền xử lý truyền thống. Các phương pháp thông thường yêu cầu người dùng hoặc viết mã để phân đoạn dữ liệu (rào cản kỹ thuật cao), hoặc dùng phân đoạn tự động kiểu hộp đen (thiếu kiểm soát, dễ sai). Giao diện kéo thả loại bỏ các điểm khó khăn này bằng cách cho phép thao tác trực tiếp trên biểu đồ tín hiệu, giảm thời gian tiền xử lý ước tính 60–80% so với chú thích thủ công bằng nhãn thời gian.

6.2 So sánh với công trình liên quan

6.2.1 Các nền tảng thương mại

So sánh trực tiếp với **Edge Impulse** trên cùng tập dữ liệu 3 lớp (Bảng 5.6.1) cho thấy framework đề xuất đạt $F1 = 1,00$ so với 0,96 của Edge Impulse, và hoạt động tốt trên thiết bị trong khi Edge Impulse thất bại ở lớp running. Đáng chú ý, sklearn Neural Network của framework cũng gặp lỗi tương tự (không nhận dạng được running), gợi ý đây là vấn đề chung của các mô hình dựa trên đặc trưng khi ranh giới quyết định quá hẹp. Về tài nguyên (Bảng 5.6.2), cả hai đều sử dụng INT8 nhưng framework đề xuất chiếm ít flash hơn (18–23% so với 51%) nhờ sinh mã trực tiếp, không cần runtime.

SensiML nhằm mục tiêu triển khai IoT thương mại với AutoML tinh vi nhưng chi phí \$99–500/tháng. Các điểm phân biệt kiến trúc chính nằm ở tính minh bạch mã sinh ra, hỗ trợ thuật toán ML cổ điển, và khả năng xác minh tương đồng Python↔C++ (xem Bảng 5.6.3 trong Chương 5).

6.2.2 Các hệ thống HAR học thuật

So với nghiên cứu HAR học thuật sử dụng các phương pháp dựa trên IMU tương tự^[4], độ chính xác đạt được (97–99% trên 5 lớp) cạnh tranh với các kết quả đã công bố

trên các tập dữ liệu benchmark (92–97% điển hình). Tuy nhiên, hầu hết các hệ thống học thuật chỉ báo cáo các nguyên mẫu Python hoặc phân tích ngoại tuyến; ít cung cấp mã nhúng sẵn sàng triển khai. Đóng góp của luận văn nằm ở việc bắc cầu “khoảng cách triển khai” này bằng cách tự động hóa việc dịch từ các mô hình scikit-learn/PyTorch đã huấn luyện sang mã suy luận vi điều khiển.

6.3 Các đánh đổi thiết kế

6.3.1 ML cổ điển vs học sâu

Luận văn này cố ý chọn học máy cổ điển (Random Forest, SVM, Neural Networks) cùng với học sâu (PyTorch CNN) để cung cấp lựa chọn linh hoạt. Kết quả thực nghiệm xác nhận giá trị của cả hai hướng tiếp cận: **ML cổ điển cho prototyping nhanh và các bài toán đơn giản** (RF hoạt động tốt cho 3 lớp, huấn luyện 0,15s), **học sâu end-to-end cho triển khai đáng tin cậy** trên các bài toán phức tạp (chỉ 1D-CNN hoạt động đáng tin cậy trên cả hai cấu hình).

6.3.2 Đặc trưng thủ công vs đặc trưng tự học

Bộ 63 đặc trưng bất biến hướng đạt 97–99% độ chính xác, xác nhận hiệu quả của thiết kế dựa trên kiến thức miền. Học sâu 1D-CNN đạt hiệu suất tương đương (99,13%) nhưng với kích thước mô hình lớn hơn và khả năng diễn giải thấp hơn. Hướng trung gian—dùng bộ mã hóa tự động convolutional để học đặc trưng, kết hợp với ML cổ điển cho phân loại—có thể kết hợp được ưu điểm của cả hai và là một hướng nghiên cứu tiếp theo tiềm năng.

6.3.3 Tạo mã trực tiếp vs runtime suy luận

Ba lợi ích dự kiến đã được xác nhận qua thực nghiệm:

Thứ nhất, *mức sử dụng bộ nhớ tối thiểu*: Neural Network chỉ chiếm 95KB flash, bao gồm trọng số, logic suy luận, trích xuất 15 đặc trưng thống kê, và chuẩn hóa StandardScaler. Với TFLite Micro, cùng chức năng sẽ cần thêm 150–250KB cho runtime interpreter, đẩy tổng lên 245–345KB.

Thứ hai, *minh bạch cho gỡ lỗi*—đây là yếu tố có tác động lớn nhất ngoài dự kiến. Khi mô hình đã triển khai luôn dự đoán `walking_downstairs`, người phát triển có thể: (a) đọc trực tiếp mã C++ đã sinh để xác minh logic suy luận, (b) in giá trị đặc trưng trung gian qua cổng serial để so sánh với Python, (c) phát hiện lỗi zero-padding và công thức kurtosis bằng cách kiểm tra từng dòng. Kinh nghiệm thực tế cho thấy 3

6.4. Tương đồng huấn luyện-triển khai trong edge ML

trong 10 phát hiện kỹ thuật chỉ có thể xác định nhờ khả năng đọc mã đã sinh theo từng dòng.

Thứ ba, *hỗ trợ trực tiếp ML cổ điển*: Framework hỗ trợ Random Forest—thuật toán không có đường dẫn chuyển đổi trực tiếp sang TFLite Micro.

Chi phí phát triển: Framework cần triển khai logic suy luận riêng cho mỗi thuật toán, bao gồm module trích xuất đặc trưng dùng chung và các generator đặc thù cho từng loại mô hình. Tuy nhiên, đây là *chi phí một lần*: sau khi generator được kiểm chứng cho một thuật toán, nó hoạt động đúng cho mọi mô hình được huấn luyện bằng thuật toán đó.

Kết luận: Tạo mã trực tiếp là lựa chọn đúng cho bối cảnh cụ thể của framework (tập dữ liệu vừa phải, thuật toán ML cổ điển, vi điều khiển bị giới hạn tài nguyên, yêu cầu minh bạch cao).

6.3.4 Các chế độ tối ưu hóa

Bốn chế độ tối ưu hóa (accuracy, speed, power, balanced) cung cấp sự linh hoạt, nhưng đòi hỏi người dùng hiểu được các đánh đổi liên quan. Các hướng thiết kế thay thế có thể bao gồm đo hiệu năng tự động, tối ưu hóa đa mục tiêu, hoặc runtime thích nghi điều chỉnh động độ phức tạp suy luận dựa trên mức pin hoặc cường độ chuyển động.

6.4 Tương đồng huấn luyện-triển khai trong edge ML

Một phát hiện quan trọng trong quá trình phát triển là tầm quan trọng thiết yếu của *tương đồng huấn luyện-triển khai*—sự đảm bảo rằng quá trình trích xuất đặc trưng trong môi trường huấn luyện (Python) tạo ra kết quả **giống hệt** với quá trình trên thiết bị biên (C++/MicroPython). Vấn đề này ít được nghiên cứu trong tài liệu edge ML nhưng có tác động nghiêm trọng đến hiệu suất triển khai thực tế^[28;34].

6.4.1 Các nguồn không khớp được xác định

Trong quá trình phát triển, đã xác định và giải quyết nhiều nguồn sai lệch:

Artifact đệm số không (zero-padding artifact) Pipeline ban đầu đệm các cửa sổ ngắn hơn kích thước mục tiêu bằng toàn số không. Trên thiết bị thực, bộ đệm trượt luôn chứa dữ liệu cảm biến thực, vì cảm biến liên tục đo gia tốc trọng trường. Sự

6.4. Tương đồng huấn luyện-triển khai trong edge ML

khác biệt này tạo ra phân phối đặc trưng hoàn toàn khác nhau giữa lúc huấn luyện ($\text{acc_mag_min} = 0$) và lúc triển khai ($\text{acc_mag_min} \approx 9,81$).

Sai lệch công thức thống kê Công thức độ nhọn (kurtosis) trong mã C++ ban đầu sử dụng độ lệch chuẩn mẫu $(n - 1)$, trong khi Python pandas sử dụng độ lệch chuẩn tổng thể (n) . Sai lệch hệ thống $(n - 1)/n)^2 \approx 0,987$ cho $n = 150$ tương đương $\sim 1,3\%$ cho mỗi đặc trưng kurtosis và skewness—nhỏ nhưng hệ thống và tích lũy qua 6 trục.

Thứ tự đặc trưng không khớp Python huấn luyện trên các cột DataFrame được sắp xếp theo thứ tự bảng chữ cái, trong khi C++ trích xuất đặc trưng theo thứ tự cố định. Bước sắp xếp lại thứ tự đặc trưng khi sinh mã đảm bảo trọng số mô hình và tham số scaler tương ứng chính xác.

6.4.2 Tác động thực tế

Trong ca kiểm chứng trên thiết bị tham chiếu Seed XIAO nRF52840, các lỗi không khớp khiến mô hình luôn dự đoán lớp `walking_downstairs` bất kể hoạt động thực tế. Bài học này không chỉ áp dụng cho XIAO mà cho toàn bộ ma trận đích của framework.

6.4.3 Danh sách kiểm tra tương đồng

Bảng 6.4.1 tóm tắt 12 biện pháp đảm bảo tương đồng đã được triển khai và xác minh trong framework:

6.4. Tương đồng huấn luyện-triển khai trong edge ML

Bảng 6.4.1: Danh sách kiểm tra tương đồng huấn luyện-triển khai

#	Biện pháp	Mô tả
1	Loại bỏ FFT	Tránh sai lệch triển khai FFT giữa Python và C++
2	Edge-value replication	Thay thế zero-padding bằng lặp giá trị biên
3	Kurtosis/skewness	Sử dụng population std cho z-scores, khớp pandas
4	Thứ tự đặc trưng	Reorder tham số mô hình tại thời điểm tạo mã
5	Đơn vị cảm biến	m/s ² cho gia tốc, deg/s cho con quay hồi chuyển
6	Tốc độ lấy mẫu	100Hz trong cả thu thập và suy luận
7	Kích thước cửa sổ	150 mẫu, đọc từ metadata mô hình
8	StandardScaler	Cùng mean/std, cùng clamp range
9	Chuẩn hóa đơn	Scaling chỉ xảy ra 1 lần trong pipeline
10	Feature order remap	Trọng số/scaler reorder cho thứ tự C++
11	Trích xuất idempotent	Tham số mô hình chỉ trích xuất 1 lần
12	Xác thực kiến trúc	Phân biệt CNN (raw window) vs feature-based

6.4.4 So sánh với nền tảng thương mại

Các nền tảng thương mại như Edge Impulse hoạt động theo mô hình hộp đen, trong đó pipeline trích xuất đặc trưng không thể kiểm tra hoặc xác minh bởi người dùng. Tính **minh bạch hoàn toàn** của framework cho phép kiểm tra trực tiếp giá trị đặc trưng, truy vết ngược đến nguyên nhân gốc, xác minh từng công thức giữa Python và C++, và sửa lỗi mà không chờ nhà cung cấp. Phát hiện này nhấn mạnh rằng **tính minh bạch không chỉ là triết lý mã nguồn mở mà là yêu cầu kỹ thuật thiết yếu** cho triển khai edge ML đáng tin cậy.

6.4.5 Bộ lọc Kalman: giải pháp parity

Việc triển khai bộ lọc Kalman constant-velocity đại diện cho một bước tiến quan trọng—đây là lần đầu tiên framework có một bộ lọc tiên xử lý với **parity gap bằng không**.

Các bộ lọc IIR Butterworth truyền thống gặp hạn chế parity cố hữu: phương pháp lọc hai chiều (non-causal, zero-phase) trong huấn luyện xử lý dữ liệu theo cả hai chiều, trong khi phương pháp lọc một chiều (causal, forward-only) trong triển khai chỉ xử lý

theo một chiều do ràng buộc thời gian thực. Khoảng cách này gây sai lệch đặc biệt nghiêm trọng với các đặc trưng nhạy cảm với pha như skewness và kurtosis.

Bộ lọc Kalman khắc phục vấn đề này từ gốc: (1) tính causal tự nhiên—chỉ dùng quan sát hiện tại và trạng thái trước đó, (2) tính thích nghi—Kalman gain tự động điều chỉnh theo prediction error, (3) mô hình vật lý phù hợp với bản chất chuyển động của dữ liệu IMU, (4) tham số trực quan—process noise (Q) và measurement noise (R) có ý nghĩa vật lý rõ ràng.

6.5 Tăng cường dữ liệu và ngưỡng tin cậy

6.5.1 Tăng cường nhận biết lớp

Năm phương pháp tăng cường được chọn (jitter, scaling, rotation, time warping, permutation) mô phỏng các nguồn biến thiên thực tế. Tuy nhiên, thực nghiệm ban đầu áp dụng tất cả phương pháp đồng nhất cho mọi lớp phát hiện vấn đề: hoạt động *still* bị phân loại sai thành *walking_downstairs* do jitter và rotation trên cửa sổ tĩnh tạo ra dao động nhân tạo.

Chiến lược nhận biết lớp giải quyết vấn đề này bằng cách tự động phân biệt hoạt động tĩnh và động: hoạt động tĩnh chỉ nhận micro-jitter ($\sigma = 0,01$) thay vì biến đổi đầy đủ, bảo toàn đặc tính phân biệt “gần bất động”. Nguyên tắc thiết kế: **tăng cường không được phá hủy đặc tính khiến một lớp khác biệt với các lớp khác.**

6.5.2 Ngưỡng tin cậy cho từ chối hoạt động không xác định

Các hệ thống phân loại thông thường gán một nhãn cho mọi đầu vào, kể cả khi đầu vào nằm ngoài phân phối huấn luyện. Framework trích xuất điểm tin cậy trực tiếp từ đầu ra phân loại hiện có (softmax/tỷ lệ bỏ phiếu) mà không cần mô hình phát hiện bất thường riêng biệt^[20]. Cách tiếp cận này có ba ưu điểm: không tốn thêm bộ nhớ, không cần dữ liệu bổ sung, và tích hợp tự nhiên trong quá trình suy luận.

Hạn chế: Điểm tin cậy softmax thường thiếu hiệu chỉnh tốt^[19]. Random Forest có độ tin cậy được hiệu chỉnh tự nhiên tốt hơn (tỷ lệ bỏ phiếu mang ý nghĩa xác suất thống kê), trong khi Neural Network/SVM qua softmax có thể quá tự tin. Các kỹ thuật hiệu chỉnh nhiệt độ (temperature scaling) có thể cải thiện chất lượng tin cậy. Ngưỡng $\tau = 0,6$ là giá trị thực nghiệm; điều chỉnh tự động dựa trên đường cong ROC của tập xác thực là hướng nghiên cứu tiếp theo.

6.6 Phân tích các phát hiện kỹ thuật quan trọng

6.6.1 Kiến trúc tạo mã đa kiến trúc

Việc hỗ trợ đồng thời các mô hình feature-based (RF/SVM/NN) và end-to-end (CNN) trong cùng một framework đặt ra thách thức kiến trúc đặc thù. Mô hình CNN sử dụng đầu vào thô (6 kênh $\times$ 150 timesteps) trong khi mô hình feature-based sử dụng vector đặc trưng thống kê (33–90 chiều). Sự khác biệt này đòi hỏi hệ thống tạo mã phải nhận biết kiến trúc mô hình: sinh metadata, logic xác minh, và cấu trúc mã đầu ra đều khác nhau tùy loại pipeline. Giải pháp là phân nhánh rõ ràng tại thời điểm sinh mã dựa trên trường kiến trúc trong gói mô hình.

6.6.2 Hạn chế TFLite và giải pháp Keras surrogate

Pipeline chuyển đổi ONNX→TFLite gặp vấn đề: công cụ chuyển đổi scikit-learn sang ONNX tạo thành công các toán tử ONNX-ML (TreeEnsembleClassifier, SVM-Classifer), nhưng công cụ chuyển đổi ONNX sang TFLite không hỗ trợ các toán tử ONNX-ML—chỉ hỗ trợ các toán tử ONNX chuẩn. Kết quả: RF/SVM không thể chuyển đổi trực tiếp sang TFLite.

Giải pháp Keras surrogate có ưu điểm (tận dụng quantization INT8, tương thích với hệ sinh thái TensorFlow) nhưng cũng có nhược điểm: tổn thất xấp xỉ (Keras MLP chỉ đạt 80–90% mức đồng thuận với RF/SVM gốc), tốn thêm bộ nhớ, và phức tạp hơn khi huấn luyện. **Khuyến nghị:** RF/SVM ưu tiên sinh mã C++ trực tiếp (chính xác, gọn, nhanh); NN/CNN dùng TFLite để tận dụng hệ sinh thái trưởng thành; Keras surrogate chỉ khi bắt buộc tích hợp với runtime chuẩn hóa.

6.7 Các hạn chế

6.7.1 Hạn chế tập dữ liệu

Xác thực hiện tại sử dụng dữ liệu từ một đối tượng duy nhất thực hiện năm hoạt động. Nghiên cứu tiếp theo nên mở rộng sang các nhóm đa đối tượng với nhân khẩu học đa dạng (tuổi, giới tính, mức độ vận động), bổ sung thêm các loại hoạt động như té ngã, các hoạt động phức tạp (nấu ăn, lái xe), chuyển đổi hoạt động, và kiểm chứng trên các tập dữ liệu benchmark công khai (UCI HAR, WISDM, PAMAP2).

6.7.2 Lựa chọn kích thước cửa sổ

Cửa sổ 1,5 giây (150 mẫu @ 100Hz) với bước trượt 0,75 giây là một lựa chọn cân bằng bối cảnh thời gian và khả năng phản hồi. Framework này hiện tại sử dụng kích thước cửa sổ cố định; các chiến lược cửa sổ thích ứng (thay đổi kích thước dựa trên cường độ hoạt động) có thể cải thiện hiệu suất nhưng thêm độ phức tạp.

6.7.3 Vị trí và hướng cảm biến

Các thử nghiệm giả định IMU được đeo ở cổ tay trong một hướng nhất quán. Triển khai thực tế phải đối mặt với nhiều biến đổi hơn: các vị trí đeo khác nhau trên cơ thể tạo ra đặc tính tín hiệu riêng, việc xoay thiết bị dẫn đến không xác định về hướng đặt, và sử dụng nhiều cảm biến đồng thời đòi hỏi tích hợp dữ liệu đa nguồn. Các hệ thống bền vững hơn cần trích xuất đặc trưng bất biến hướng hoặc các thuật toán ước lượng hướng bổ sung.

6.7.4 Sự đa dạng nền tảng tính toán

Framework hiện đã hiện thực generator cho nhiều họ đích, nhưng mức độ xác thực cần được phân tách thành hai lớp. Ở lớp thứ nhất, *khả năng sinh mã* đã được chứng minh qua kiến trúc factory và các generator đặc thù nền tảng. Ở lớp thứ hai, *benchmark định lượng liên thiết bị* vẫn mới chỉ được đo chi tiết trên XIAO. Phát biểu chính xác của luận văn là “framework cung cấp năng lực triển khai đa thiết bị ở cấp độ kiến trúc và sinh mã, với XIAO là nền tảng thực nghiệm tham chiếu đầu tiên”. Các bước tiếp theo cần benchmark thêm trên AVR (Arduino Uno), RISC-V (ESP32-C3), ARM Cortex-M0+ (Raspberry Pi Pico).

6.7.5 Tính hợp lệ bên ngoài

Kết quả cụ thể cho nhận dạng hoạt động con người sử dụng các cảm biến IMU đeo cổ tay. Khả năng áp dụng cho các lĩnh vực theo dõi chuyển động khác (nhận dạng cử chỉ, theo dõi hành vi động vật, lập hồ sơ chuyển động robot) vẫn cần được xác thực. Kiến trúc của framework được thiết kế để không phụ thuộc miền, nhưng các chiến lược tiền xử lý và bộ đặc trưng có thể yêu cầu thích ứng.

6.8 Hướng phát triển tương lai

6.8.1 Các mở rộng ngắn hạn

Các cải tiến ngay lập tức trong kiến trúc hiện tại:

1. **Mở rộng thuật toán và lựa chọn đặc trưng:** Tích hợp XGBoost, LightGBM, k-NN; triển khai phân tích tầm quan trọng đặc trưng tự động để giảm chiều và cải thiện hiệu suất triển khai
2. **Nén và tối ưu hóa mô hình:** Cắt tỉa trọng số, huấn luyện nhận thức lượng tử hóa để giảm thêm bộ nhớ và tăng tốc suy luận trên vi điều khiển
3. **Benchmark đa nền tảng:** Mở rộng xác thực định lượng trên ESP32, STM32, nRF5340 và kiểm chứng tính linh hoạt kiến trúc factory
4. **Cải tiến khả năng sử dụng:** Quy trình có hướng dẫn cho người dùng mới, trực quan hóa hoạt động thời gian thực từ thiết bị, nghiên cứu người dùng so sánh với các phương pháp truyền thống

6.8.2 Tầm nhìn dài hạn

Các cải tiến chuyển đổi yêu cầu thay đổi kiến trúc:

1. **Kiến trúc học sâu nâng cao:** Tích hợp LSTM/Transformer qua TFLite Micro, cho phép nắm bắt phụ thuộc thời gian dài hạn trong chuỗi hoạt động
2. **Học liên bang và bảo mật:** Huấn luyện cộng tác bảo vệ quyền riêng tư trên nhiều thiết bị, cho phép mô hình cải thiện liên tục mà không chia sẻ dữ liệu thô
3. **Hợp nhất đa phương thức:** Tích hợp IMU + GPS + nhịp tim + âm thanh với đồng bộ hóa đặc trưng tự động, mở rộng phạm vi ứng dụng sang các bài toán theo dõi sức khỏe tổng hợp
4. **AI có thể giải thích và hiệu chỉnh tin cậy:** Trực quan hóa LIME/SHAP cho các mẫu kích hoạt dự đoán, kết hợp temperature scaling để cải thiện độ tin cậy softmax

6.9 Kết luận

6.9.1 Tóm tắt đóng góp

Nghiên cứu này giải quyết thách thức đưa ứng dụng theo dõi chuyển động được hỗ trợ bởi AI đến gần hơn với các nhà nghiên cứu, nhà phát triển và giảng viên mà không phải đánh đổi hiệu suất hay mức độ kiểm soát trên các loại thiết bị biên đa dạng. Luận văn trình bày một framework mã nguồn mở toàn diện, tích hợp tiền xử lý dữ liệu, huấn luyện mô hình và triển khai biên vào một quy trình thân thiện với người dùng.

Bảy đóng góp kỹ thuật chính:

1. Giao diện tiền xử lý tương tác kéo thả giảm thời gian tiền xử lý 60-80%
2. Hỗ trợ đa thuật toán (RF, SVM, NN, PyTorch MLP, PyTorch 1D-CNN) với API nhất quán
3. Kiến trúc triển khai đa thiết bị nhận biết tối ưu hóa với bốn chế độ
4. Xử lý mất cân bằng lớp hệ thống với cân bằng trọng số tự động mặc định
5. Đảm bảo tương đồng huấn luyện-triển khai qua quy trình xác minh 12 điểm kiểm tra
6. Tăng cường dữ liệu nhận biết lớp với chiến lược phân biệt tĩnh/động
7. Ngưỡng tin cậy cho từ chối hoạt động không xác định mà không cần mô hình bổ sung

Xác thực thực nghiệm: Độ chính xác 97,38–99,13% trên năm thuật toán cho bài toán 5 lớp, 100% cho bài toán 3 lớp; F1-scores 0,949–1,000 trên tất cả các lớp hoạt động bao gồm các lớp thiểu số; triển khai thành công trên nền tảng tham chiếu Sseed XIAO nRF52840 với PyTorch 1D-CNN đạt 99,4% độ chính xác trên thiết bị.

6.9.2 Xem lại các mục tiêu nghiên cứu

Các mục tiêu nghiên cứu ban đầu được nêu trong Mục 1.3 đã được giải quyết toàn diện:

1. **Hệ thống tiền xử lý tương tác ✓:** Giao diện cửa sổ thời gian kéo thả được triển khai thành công

2. **Hỗ trợ đa thuật toán ✓**: Năm thuật toán tích hợp, tất cả đạt 97,38–99,13%
3. **Tạo mã không phụ thuộc nền tảng ✓**: Kiến trúc trình tạo modular theo ma trận model-platform-backend
4. **Xác thực hiệu quả framework ✓**: Độ chính xác cạnh tranh và triển khai thành công trên thiết bị tham chiếu
5. **Khả năng tiếp cận ✓**: Giao diện web trực quan, dễ sử dụng cho người dùng không chuyên về lập trình nhúng
6. **Độ tin cậy triển khai ✓**: Quy trình xác minh tương đồng 12 điểm, ngưỡng tin cậy và tăng cường nhận biết lớp

6.9.3 Các ứng dụng thực tế

Framework đã xác thực cho phép các ứng dụng thực tế đa dạng:

Y tế và phục hồi chức năng Các hệ thống đeo để giám sát mức độ hoạt động của bệnh nhân, theo dõi tiến trình phục hồi chức năng (phục hồi di động sau phẫu thuật, trị liệu đột quỵ), phát hiện té ngã trong quần thể người cao tuổi và cung cấp dữ liệu hoạt động khách quan cho các thử nghiệm lâm sàng.

Thể thao và thể dục Phát hiện và phân loại tập luyện tự động, phân tích hình thức để phòng ngừa chấn thương (phát hiện bất đối xứng đáng đi, kỹ thuật nâng không đúng) và phản hồi đào tạo cá nhân hóa. Cửa sổ trượt 1,5 giây với bước trượt 0,75 giây đảm bảo phản hồi gần thời gian thực.

Nghiên cứu và giáo dục Nền tảng chi phí thấp để giảng dạy các khái niệm edge AI trong các khóa học đại học/sau đại học, cho phép các phòng thí nghiệm thực hành mà không cần giấy phép thương mại đắt đỏ. Các nhà nghiên cứu có thể nhanh chóng tạo mẫu các thuật toán nhận dạng hoạt động mới lạ và triển khai lên phần cứng để xác thực hiện trường.

Nhà thông minh và hỗ trợ sinh hoạt hàng ngày Giám sát hoạt động cho người cao tuổi sống độc lập (phát hiện các trạng thái bất thường, theo dõi thói quen hàng ngày), tự động hóa nhà thông minh nhận biết bối cảnh (điều chỉnh ánh sáng/nhiệt độ theo hoạt động được phát hiện) và phân tích xu hướng sức khỏe dài hạn.

6.9.4 Lời kết

Luận văn này trình bày một framework toàn diện giải quyết các thách thức cốt lõi trong theo dõi chuyển động trên thiết bị biên: khả năng tiếp cận, hiệu suất và tính linh hoạt. Giá trị cốt lõi của framework không nằm ở việc tối ưu riêng cho Seeed XIAO, mà ở khả năng chuyển cùng một pipeline sang nhiều thiết bị biên khác nhau trong khi vẫn đảm bảo tính minh bạch và khả năng kiểm chứng. Xác thực thực nghiệm khẳng định rằng **tính tiếp cận rộng rãi và hiệu suất cao không mâu thuẫn nhau—chúng hoàn toàn có thể cùng tồn tại.**

Hành trình từ dữ liệu cảm biến thô đến mô hình edge AI sẵn sàng triển khai trải qua nhiều bước, mỗi bước truyền thống đòi hỏi chuyên môn chuyên biệt. Framework này giảm thiểu các rào cản đó, cho phép bất kỳ ai có kiến thức về bài toán (các hoạt động cần nhận dạng) tạo ra hệ thống sẵn sàng triển khai mà không cần trở thành chuyên gia về xử lý tín hiệu, học máy hay lập trình nhúng. Đây là bản chất của việc phổ cập công nghệ: **hạ thấp rào cản mà không hạ thấp tiêu chuẩn.**

Nhìn về tương lai, sự phát triển của edge AI sẽ phụ thuộc vào các công cụ trao quyền cho người dùng thay vì giới hạn họ, giáo dục thay vì che giấu, và tính bao trùm thay vì loại trừ. Framework này là một bước trong hướng đó.

Mã nguồn, tài liệu và tập dữ liệu ví dụ được công bố công khai nhằm hỗ trợ tái tạo kết quả, mục đích giáo dục và đóng góp cộng đồng. Các nhà nghiên cứu và nhà thực hành được khuyến khích xây dựng trên nền tảng này, mở rộng khả năng của framework và ứng dụng vào các bài toán theo dõi chuyển động mới.

Tài liệu tham khảo

- [1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, et al. Tensorflow: Large-scale machine learning on heterogeneous distributed systems, 2015.
- [2] Allied Market Research. Motion tracking system market size, share, competitive landscape and trend analysis report. <https://www.alliedmarketresearch.com/motion-tracking-system-market>, 2023. Accessed: December 2, 2025.
- [3] Saleema Amershi, Maya Cakmak, W. Bradley Knox, and Todd Kulesza. Power to the people: The role of humans in interactive machine learning. *AI Magazine*, 35(4):105–120, 2014.
- [4] Davide Anguita, Alessandro Ghio, Luca Oneto, Xavier Parra, and Jorge L. Reyes-Ortiz. A public domain dataset for human activity recognition using smartphones. *21th European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning (ESANN)*, 2013.
- [5] Colby Banbury, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kiraly, Pietro Montino, David Kanter, Sebastian Ahmed, Danilo Pau, et al. Mlperf tiny benchmark. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- [6] Oresti Banos, Juan-Manuel Galvez, Miguel Damas, Hector Pomares, and Ignacio Rojas. Window size impact in human activity recognition. *Sensors*, 14(4):6474–6499, 2014.
- [7] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [8] Mateusz Buda, Atsuto Maki, and Maciej A. Mazurowski. A systematic study of the class imbalance problem in convolutional neural networks. *Neural Networks*, 106:249–259, 2018.

- [9] Andreas Bulling, Ulf Blanke, and Bernt Schiele. A tutorial on human activity recognition using body-worn inertial sensors. *ACM Computing Surveys*, 46(3):1–33, 2014.
- [10] Han Cai, Ligeng Zhu, and Song Han. ProxylessNAS: Direct neural architecture search on target task and hardware. In *International Conference on Learning Representations (ICLR)*, 2019.
- [11] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.
- [12] Robert David, Jared Duke, Advait Jain, Vijay Janapa Reddi, Nat Jeffries, Jian Li, Nick Kreeger, Ian Nappier, Meghna Natraj, Shlomi Regev, Rocky Rhodes, Tiezhen Wang, and Pete Warden. TensorFlow Lite Micro: Embedded machine learning for TinyML systems. *Proceedings of Machine Learning and Systems*, 3:800–811, 2021.
- [13] Lipika Dutta and Swapna Bharali. Tynyml meets iot: A comprehensive survey. *Internet of Things*, 16:100461, 2021.
- [14] Edge Impulse. Edge impulse studio. <https://studio.edgeimpulse.com/>, 2024. Accessed: December 12, 2024.
- [15] Alessandro Filippeschi, Norbert Schmitz, Markus Miezal, Gabriele Bleser, Emanuele Ruffaldi, and Didier Stricker. Survey of motion tracking methods based on inertial sensors: A focus on upper limb human motion. *Sensors*, 17(6):1257, 2017.
- [16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [17] Google. Mediapipe solutions guide. <https://ai.google.dev/edge/mediapipe/solutions/guide>, 2024. Accessed: December 12, 2024.
- [18] Google. Teachable machine. <https://teachablemachine.withgoogle.com/>, 2024. Accessed: December 12, 2024.
- [19] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330, 2017.

- [20] Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017.
- [21] Brian Kenji Iwana and Seiichi Uchida. An empirical survey of data augmentation for time series classification with neural networks. *PLOS ONE*, 16(7):e0254841, 2021.
- [22] Rudolf E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.
- [23] Oscar D. Lara and Miguel A. Labrador. A survey on human activity recognition using wearable sensors. *IEEE Communications Surveys & Tutorials*, 15(3):1192–1209, 2013.
- [24] Arthur Le Guennec, Simon Malinowski, and Romain Tavenard. Data augmentation for time series classification using convolutional neural networks. In *ECML/PKDD Workshop on Advanced Analytics and Learning on Temporal Data*, 2016.
- [25] Ji Lin, Wei-Ming Chen, Yujun Lin, Chuang Gan, and Song Han. Mccunet: Tiny deep learning on iot devices. *Advances in Neural Information Processing Systems (NeurIPS)*, 33:11711–11722, 2020.
- [26] Massimo Merenda, Carlo Porcaro, and Demetrio Iero. Edge machine learning for ai-enabled iot devices: A review. *Sensors*, 20(9):2533, 2020.
- [27] Alan V. Oppenheim and Ronald W. Schaffer. *Discrete-Time Signal Processing*. Prentice Hall, 2nd edition, 1999.
- [28] Andrei Paleyes, Raoul-Gabriel Urma, and Neil D. Lawrence. Challenges in deploying machine learning: A survey of case studies. *ACM Computing Surveys*, 55(6):1–29, 2022.
- [29] Anuj Patil, Darshan Rao, Kaustubh Utturwar, Tejas Shelke, and Ekta Sarda. Body posture detection and motion tracking using ai for medical exercises and recommendation system. *ITM Web Conference, Volume 44, International Conference on Automation, Computing and Communication 2022 (ICACC-2022)*, May 2022.

- [30] Partha Pratim Ray. A review on tinyml: State-of-the-art and prospects. *Journal of King Saud University - Computer and Information Sciences*, 34(4):1595–1623, 2022.
- [31] Attila Reiss and Didier Stricker. Introducing a new benchmarked dataset for activity monitoring. In *2012 16th International Symposium on Wearable Computers (ISWC)*, pages 108–109, 2012.
- [32] Abraham Savitzky and Marcel J. E. Golay. Smoothing and differentiation of data by simplified least squares procedures. *Analytical Chemistry*, 36(8):1627–1639, 1964.
- [33] Jürgen Schmidhuber. Deep learning in neural networks: An overview. *Neural Networks*, 61:85–117, 2015.
- [34] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems (NeurIPS)*, 28, 2015.
- [35] SensiML Corporation. Sensiml analytics toolkit. <https://sensiml.com/>, 2024. Accessed: December 3, 2025.
- [36] Jaspreet Singh, Yaochu Xie, Chen Chen, and Wenchao Jiang. Attention-based human activity recognition using wearable sensors. *IEEE Sensors Journal*, 23(11):12213–12223, 2023.
- [37] Odongo Steven Eyobu and Dong Seog Han. Feature representation and data augmentation for human activity classification based on wearable IMU sensor data using a deep LSTM neural network. *Sensors*, 18(9):2892, 2018.
- [38] Terry T. Um, Franz M. J. Pfister, Daniel Pichler, Satoshi Endo, Muriel Lang, Sandra Hirche, Urban Fiber, and Dana Klückmann. Data augmentation of wearable sensor data for Parkinson’s disease monitoring using convolutional neural networks. In *Proceedings of the 19th ACM International Conference on Multimodal Interaction (ICMI)*, pages 216–220. ACM, 2017.
- [39] Shaohua Wan, Lianyong Qi, Xiaolong Xu, Chenguang Tong, and Zongming Gu. Deep learning models for real-time human activity recognition with smartphones. *Mobile Networks and Applications*, 25:743–755, 2020.

- [40] An Wang, Guangyu Chen, Chuang Shang, Mingli Zhang, and Likun Liu. Human activity recognition in a smart home environment with stacked denoising autoencoders. *IEEE Transactions on Industrial Informatics*, 19(2):2071–2080, 2023.
- [41] An Wang, Guangyu Chen, Jian Yang, Shihua Zhao, and Ching-Yao Chang. A comparative study on human activity recognition using inertial sensors in a smartphone. *IEEE Sensors Journal*, 24(3):3234–3247, 2024.
- [42] Pete Warden and Daniel Situnayake. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O’Reilly Media, 2019.
- [43] Michael W. Whittle. *Gait Analysis: An Introduction*. Butterworth-Heinemann, 5th edition, 2014.
- [44] Ke Xia, Jianguang Huang, and Hanyu Wang. Lstm-cnn architecture for human activity recognition. *IEEE Access*, 8:56855–56866, 2020.
- [45] Huiyu Zhou and Huosheng Hu. Human motion tracking for rehabilitation—a survey. *Biomedical Signal Processing and Control*, 3:1–18, 01 2008.